Financial Management

Volatility, Risk, and AI

A PBL Journey Through Modern Risk Analytics

VolTech Analytics

Quantitative Risk Solutions for Asia

Where Engineering Meets Finance

Instructor: Yi-Hao Lai

Department: Finance, Da-Yeh University

Program: International Student Program

Semester: Spring 2026 (2nd Semester)

Edition: 1st Edition

Da-Yeh University · Department of Finance · 2026

Preface

Why This Book?

Financial markets are inherently uncertain. The 2008 Global Financial Crisis, the 2020 COVID-19 market crash, and the 2022 interest rate shock all share a common lesson: **risk models that assume normality will fail when you need them most**.

This textbook was designed for engineering and science students who have completed one semester of financial management fundamentals. You bring strong mathematical skills and programming ability — exactly what modern risk analytics demands. What you may lack is financial intuition. This book bridges that gap.

How to Use This Book

This is not a traditional textbook. It is a **Problem-Based Learning (PBL)** experience built around a continuous storyline.

You will join **VolTech Analytics**, a Taipei-based quantitative risk consulting startup, as a junior analyst named Alex Chen. Over 8 weeks, you will help the team build a complete risk management system for their first major client — a Japanese pension fund that lost heavily during the COVID-19 market turmoil.

Each chapter follows a consistent structure:

1. **Story** — A narrative that sets up the week's challenge
2. **Theory** — The mathematical and financial concepts you need
3. **Python Implementation** — Hands-on coding with real data
4. **Mission** — A practical task that integrates everything
5. **Key Takeaways** — What you should remember
6. **Cliffhanger** — What comes next (and why you should care)

What You Will Learn

By the end of this course, you will be able to:

- Analyze financial return distributions and identify departures from normality
- Measure and forecast volatility using EWMA, GARCH, and GJR-GARCH models
- Calculate and backtest Value-at-Risk using multiple methodologies
- Apply machine learning techniques to financial risk prediction
- Build a complete risk management pipeline in Python
- Communicate risk analysis results to non-technical stakeholders

Prerequisites

- One semester of Financial Management (basic knowledge of returns, risk, and portfolio theory)
- Basic statistics (mean, variance, normal distribution, hypothesis testing)
- Basic Python programming (variables, loops, functions, pandas basics)
- Comfort with mathematical notation (summations, derivatives, matrix algebra)

Acknowledgments

This textbook was developed for the International Student Program at Da-Yeh University's Department of Finance. Special thanks to the students whose feedback shaped the PBL experience, and to the open-source Python ecosystem that makes hands-on financial analytics accessible to all.

Yi-Hao Lai
Associate Professor of Finance
Da-Yeh University
Spring 2026

Meet the Team

The people you will work with at VolTech Analytics

Alex Chen — Junior Quantitative Analyst

Background: Fresh graduate from Electrical Engineering. Strong in math and programming but new to finance.

Role: This is *you*. Alex represents your journey from engineering into the world of quantitative finance. Every question Alex asks is a question you might have. Every challenge Alex faces is designed to build your skills.

"I can code a neural network from scratch, but I have no idea what a GARCH model is. Yet."

Dr. Mei Lin — Senior Risk Manager

Background: PhD in Financial Engineering from Columbia University. 10 years at Goldman Sachs before returning to Asia to join VolTech.

Role: Your mentor. Dr. Lin explains theory through real-world analogies and Socratic questioning. She has seen every market crisis firsthand and knows where textbook models break down.

"The formula is simple. Understanding when it fails — that's where the real learning begins."

Kenji Tanaka — Head of Trading, Nippon Life Pension Fund

Background: 20 years in Japanese institutional investing. Conservative but open to innovation after 2020 exposed weaknesses in their risk models.

Role: The demanding client. Kenji brings real-world constraints: regulatory requirements, board presentations, and the pressure of managing retirement savings for millions.

"I don't need elegant mathematics. I need a number I can trust when markets are falling apart."

Priya Sharma — AI/ML Engineer

Background: MSc in Computer Science from IIT Bombay. Expert in deep learning and time series analysis.

Role: The technology bridge. Priya connects traditional financial models with modern AI approaches. She challenges conventional methods but respects what works.

"Your GARCH model is nice, but have you tried throwing a neural network at it?"

David Wu — Founder & CEO

Background: Former portfolio manager at a Taipei hedge fund. Founded VolTech to bring institutional-grade risk analytics to the Asian market.

Role: The business perspective. David asks the questions your future boss will ask: "What's the business value?" "Can we explain this to a non-technical board?" "How do we stay ahead of the competition?"

"Great analysis is worthless if you can't sell it."

Company Profile: VolTech Analytics

VolTech Analytics

Quantitative Risk Solutions for Asia

About Us

VolTech Analytics is a Taipei-based quantitative risk consulting startup founded in 2024 by David Wu, a former hedge fund portfolio manager. The company specializes in **volatility analysis** and **risk management solutions** for institutional investors across the Asia-Pacific region.

With a lean team of quantitative analysts, data scientists, and financial engineers, VolTech bridges the gap between cutting-edge academic research and practical risk management needs.

Our Mission

To democratize institutional-grade risk analytics for Asian investors by combining rigorous statistical models with modern AI techniques.

The Challenge

In early 2026, VolTech lands its first major client: **Nippon Life Pension Fund**, one of Japan's largest retirement funds. The fund suffered significant losses during the COVID-19 market crash of March 2020 because their existing risk model — based on simple normal distribution assumptions — completely failed to predict the magnitude of the downturn.

Kenji Tanaka, Head of Trading at Nippon Life, has given VolTech a clear mandate:

◎ Client Brief

1. Analyze why the existing risk model failed during COVID-19
2. Build a volatility model that captures fat tails and asymmetric effects

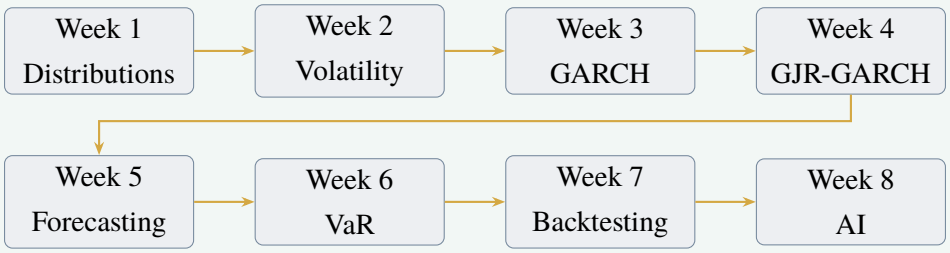
3. Develop a robust VaR system with proper backtesting
4. Explore AI-enhanced risk forecasting capabilities
5. Deliver a complete system within **8 weeks**

Your Role

You are **Alex Chen**, a newly hired junior quantitative analyst at VolTech. Your background in engineering gives you strong mathematical and programming foundations, but you are new to the world of finance and risk management.

Over the next 8 weeks, you will work alongside the VolTech team to deliver on the Nippon Life contract. Each week brings new challenges, new tools, and new insights as you grow from a junior analyst into a confident risk professional.

The Journey Ahead



Prologue: The Call That Changed Everything

Scene: VolTech Analytics Office, Taipei — Monday Morning, February 2026

The office on the 12th floor of a Xinyi District building was small but buzzing with energy. Three large monitors displayed real-time market data. A whiteboard was covered in equations. Coffee cups competed for desk space with Python notebooks.

David Wu, VolTech's founder and CEO, set down his phone and looked across the room at Dr. Mei Lin.

That was Kenji Tanaka from Nippon Life. They want us to build their entire risk management system. This is the contract that could put VolTech on the map.

Nippon Life? They manage one of the largest pension pools in Japan. What's their timeline?

Eight weeks. They've been burned by their current vendor's models — the ones that told them everything was fine right before March 2020 wiped out a significant portion of their equity portfolio.

Classic normal distribution trap. Their VaR model probably assumed returns are well-behaved bell curves. Works 95% of the time. Fails catastrophically in the 5% that matters most.

We have the tools. GARCH models for the foundation, machine learning for the edge. The question is: do we have the bandwidth?

That's why I just hired someone. An electrical engineer who can code circles around most finance PhDs. Starts today.

9:00 AM — The Lobby

Alex Chen stood in the lobby, adjusting his tie. His background was in signal processing and machine learning, not finance. But the job posting had said: *"We don't need someone who knows finance. We need someone who can learn it fast and code it well."*
That sounded like a challenge. And Alex never walked away from a challenge.

Your journey begins now.

Turn the page to start Week 1.

Contents

Preface **ii**

Meet the Team **iv**

VolTech Analytics **vi**

Prologue **viii**

1 Week 1: Financial Risk and Return Distributions **1**

1.1 This Week's Story: Day One at VolTech 1
1.2 Returns: The Language of Finance 2
1.3 The Normal Distribution Assumption 3
1.4 Fat Tails, Skewness, and Kurtosis 4
1.5 Testing for Normality 5
1.5.1 The QQ Plot 6
1.5.2 The Jarque–Bera Test 6
1.6 Python Implementation 6
1.7 Application: Analyzing Kenji's Portfolio 9
1.8 Discussion: Why Does Normality Fail? 10
1.9 Weekly Mission 10
1.10 Key Takeaways 11
1.11 Further Reading 12

2 Week 2: Measuring Volatility **13**

2.1 This Week's Story: One Ruler Is Not Enough 13
2.2 Historical Volatility: The Rolling Window 14
2.3 EWMA: Giving More Weight to Recent Data 15
2.4 Realized Volatility: Using Intraday Data 16
2.5 The VIX: Market's Fear Gauge 17
2.6 Python Implementation 19
2.7 Application: The Speed of Fear 21
2.8 Volatility Stylized Facts 22
2.9 Discussion: Choosing Your Volatility Measure 23
2.10 Weekly Mission 23

2.11 Key Takeaways 24
2.12 Further Reading 25

3 Week 3: Volatility Clustering and GARCH 26
3.1 This Week's Story: Volatility Has a Memory 26
3.2 Detecting Volatility Clustering 27
3.3 The ARCH Model: Where It All Began 28
3.4 GARCH(1,1): The Workhorse Model 29
3.5 How Fast Does Volatility Decay? 31
3.6 Maximum Likelihood Estimation 32
3.7 Python Implementation 33
3.8 Application: GARCH Meets the COVID Crash 37
3.9 Model Diagnostics: Is GARCH Enough? 39
3.10 AI Spotlight: Can AI Replace GARCH? 41
3.11 Discussion: GARCH Assumptions and Reality 41
3.12 Weekly Mission 42
3.13 Key Takeaways 43
3.14 Further Reading 44

4 Week 4: Asymmetric Volatility — GJR-GARCH 45
4.1 This Week's Story: Bad News Hits Harder 45
4.2 The GJR-GARCH Model 46
4.3 The News Impact Curve 48
4.4 EGARCH: An Alternative Approach 48
4.5 Python Implementation 49
4.6 Application: The Asymmetry Test 54
4.7 Model Diagnostics 55
4.8 AI Spotlight: Asymmetry in Machine Learning 56
4.9 Discussion: Asymmetry in Practice 57
4.10 Weekly Mission 57
4.11 Key Takeaways 59
4.12 Further Reading 60

5 Week 5: Volatility Forecasting 61
5.1 This Week's Story: The Horse Race 61
5.2 The Forecasting Framework 62
5.3 Forecast Evaluation Metrics 63
5.4 The Mincer–Zarnowitz Regression 64

5.5 Python Implementation: The Forecasting Pipeline 65
5.6 Application: Who Wins the Horse Race? 71
5.7 AI Spotlight: The Forecasting Frontier 72
5.8 Discussion: Forecasting Philosophy 73
5.9 Weekly Mission 73
5.10 Key Takeaways 74
5.11 Further Reading 75

6 Week 6: Value-at-Risk 76
6.1 This Week's Story: One Number to Rule Them All 76
6.2 Understanding Value-at-Risk 77
6.3 Method 1: Historical Simulation VaR 78
6.4 Method 2: Parametric (Variance-Covariance) VaR 79
6.5 Method 3: Monte Carlo VaR 80
6.6 Beyond VaR: Expected Shortfall 80
6.7 Python Implementation: VaR Three Ways 81
6.8 Application: VaR During the COVID-19 Crash 87
6.9 AI Spotlight: Neural Network VaR 88
6.10 Discussion: The VaR Debate 88
6.11 Weekly Mission 89
6.12 Key Takeaways 90
6.13 Further Reading 91

7 Week 7: VaR Backtesting and Risk Management 93
7.1 This Week's Story: The Regulator Calls 93
7.2 The Logic of Backtesting 94
7.3 Test 1: Kupiec Unconditional Coverage 95
7.4 Test 2: Christoffersen Conditional Coverage 95
7.5 The Basel Traffic Light System 96
7.6 Stress Testing 97
7.7 Python Implementation: The Backtesting Suite 98
7.8 Application: The FSA Report 105
7.9 AI Spotlight: Automated Risk Monitoring 106
7.10 Discussion: When Models Fail 106
7.11 Weekly Mission 107
7.12 Key Takeaways 108
7.13 Further Reading 109

8 Week 8: AI in Finance **110**
8.1 This Week's Story: The Final Challenge 110
8.2 ML vs. Econometrics: Two Philosophies 111
8.3 Random Forest for Volatility 112
8.4 LSTM Neural Networks for Time Series 112
8.5 Python Implementation: The ML Arena 113
8.6 Application: The Verdict 120
8.7 The FinTech Frontier 121
8.8 Discussion: The Future of Finance 122
8.9 Weekly Mission 122
8.10 Key Takeaways 124
8.11 Further Reading 125

Formula Appendix **127**

Glossary **133**

Chapter 1

Week 1: Financial Risk and Return Distributions

Learning Objectives

1. Calculate simple and log returns from financial price data
2. Explain why the normal distribution is a poor model for financial returns
3. Measure skewness and excess kurtosis and interpret their financial meaning
4. Construct and interpret QQ plots to assess distributional fit
5. Perform the Jarque–Bera test for normality using Python

1.1 This Week's Story: Day One at VolTech

Scene: VolTech Analytics Office, 12th Floor — Monday, 9:15 AM

Alex Chen stepped out of the elevator and into a small but energetic office. Three large monitors displayed real-time market data — the Nikkei 225, S&P 500 futures, and something labeled "VIX" that Alex didn't recognize yet. A whiteboard covered in Greek letters competed for wall space with a framed quote: *"In God we trust. All others bring data."* Dr. Mei Lin looked up from her screen.

*You must be Alex. Welcome to **VolTech**. Coffee's in the corner. Your first client meeting is in thirty minutes.*

Client meeting? Already? I thought I'd have at least a week to learn the ropes.

In quantitative finance, you learn the ropes by pulling them. Don't worry — just listen and observe. Kenji Tanaka is flying in from Tokyo. He runs the trading desk at one of Japan's

largest pension funds.

Scene: Conference Room — 10:00 AM

Kenji Tanaka was already seated when they walked in. He slid a printed report across the table — rows of red numbers from March 2020.

March 2020. COVID-19 hit, and our portfolio lost 18% in a single week. Our risk model said the maximum weekly loss at 99% confidence was 4.2%. That model told us we were safe. We were not safe.

What model were you using?

Standard parametric VaR. Normal distribution assumption. Our vendor said it was "industry standard."

It is the industry standard. That's precisely the problem. Alex, this is where we begin. Before we can fix Kenji's risk model, we need to understand why it broke. And to do that, we need to look at what financial returns actually look like — not what textbooks assume they look like.

1.2 Returns: The Language of Finance

Simple vs. Log Returns

In finance, we rarely analyze prices directly. Instead, we work with **returns** — the percentage change in price over a period. There are two common definitions:

Simple (arithmetic) return:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{1.1}$$

Log (continuously compounded) return:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(1 + R_t) \tag{1.2}$$

where P_t is the closing price at time t. Unlike prices, which tend to drift upward over time (non-stationary), returns fluctuate around a relatively stable mean (approximately station-

ary), making them suitable for statistical modeling.

Why Log Returns?

Log returns have two properties that make them the standard in quantitative finance:

1. **Time additivity**: Multi-period log returns are the sum of single-period log returns: $r_{t:t+k} = r_{t+1} + r_{t+2} + \cdots + r_{t+k}$
2. **Symmetry**: A +10% and -10% log return are symmetric around zero, unlike simple returns where a 50% loss requires a 100% gain to recover.

For small returns (say, under 5%), simple and log returns are nearly identical since $\ln(1 + x) \approx x$ for small x.

Alex, your engineering background will serve you well here. In signal processing, you analyze the spectrum of a signal. In finance, we analyze the distribution of returns. Same idea, different domain.

1.3 The Normal Distribution Assumption

The Gaussian Model

The **normal distribution** (Gaussian) has been the default model for financial returns since Bachelier (1900), who modeled price changes as normally distributed, and Samuelson (1965), who formulated the log-normal price model (equivalently, normally distributed log returns). If returns are normally distributed:

$$r_t \sim \mathcal{N}(\mu, \sigma^2) \tag{1.3}$$

This means the entire distribution is characterized by just two parameters: the mean μ and variance σ^2. The probability of extreme events decays *exponentially* fast.

Under a normal distribution:

- 68.3% of observations fall within $\pm 1\sigma$
- 95.4% fall within $\pm 2\sigma$
- 99.7% fall within $\pm 3\sigma$
- A 5σ event (one-sided) has probability $\approx 0.00003\%$ — about once in 13,900 years of trading

days

The Problem

In reality, financial markets experience 5σ events *far* more frequently than the normal distribution predicts. The S&P 500 experienced at least 20 daily moves exceeding $\pm 5\sigma$ between 1990 and 2024 — events that "should" happen once every several thousand years.

One standard remedy is the **Student-t distribution**, which has heavier tails controlled by a degrees-of-freedom parameter ν. For typical equity returns, $\nu \approx 4$–8 provides a much better fit than the normal. We will use the t distribution as the error distribution in our GARCH models starting in Week 3.

So you're telling me our model was built on a lie?

Not a lie — an approximation. The normal distribution works reasonably well for typical market conditions. But "reasonably well" and "safe for managing a pension fund" are very different standards.

1.4 Fat Tails, Skewness, and Kurtosis

To understand *how* financial returns deviate from normality, we need to look beyond mean and variance. We need the **third** and **fourth moments** of the distribution.

Skewness — The Third Moment

Skewness measures the asymmetry of a distribution:

$$S = \frac{1}{T} \sum_{t=1}^{T} \left(\frac{r_t - \bar{r}}{\hat{\sigma}} \right)^3 \tag{1.4}$$

- $S = 0$: symmetric (normal distribution)
- $S < 0$: **negatively skewed** — left tail is longer (more large losses than large gains)
- $S > 0$: positively skewed — right tail is longer

Most equity return series exhibit **negative skewness**, meaning large losses are more common than large gains of equal magnitude.

Having measured asymmetry with skewness, we now turn to the heaviness of tails — an equally critical feature for risk management.

Excess Kurtosis — The Fourth Moment

Excess kurtosis measures the heaviness of tails relative to a normal distribution:

$$K = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^4 - 3 \tag{1.5}$$

We subtract 3 because the normal distribution has raw kurtosis of exactly 3; hence K is called *excess* kurtosis.

- $K = 0$: **mesokurtic** (normal tails)
- $K > 0$: **leptokurtic** — **fat tails** (more extreme events than normal)
- $K < 0$: platykurtic — thin tails (fewer extreme events)

Financial return series typically show $K \gg 0$, meaning they are strongly **leptokurtic**. Daily stock returns often have excess kurtosis of 5–20 or higher.

One important explanation: if volatility itself varies over time, mixing periods of low and high volatility produces fat tails even when each period's returns are approximately normal. This **volatility clustering** effect is exactly what we will model with GARCH in Week 3.

Sample vs. Population Moments

The formulas above are *population* moment estimators (dividing by T). In practice, Python's `pandas.skew()` and `pandas.kurtosis()` apply small-sample bias corrections (dividing by $(T-1)(T-2)$ for skewness and a more complex adjustment for kurtosis). For large T (e.g., 2,500 daily observations over 10 years), the difference is negligible. But if you ever see slightly different numbers between a manual formula and a library function, this is why.

So fat tails mean extreme events happen more often than normal predicts. And negative skewness means big drops are more likely than big jumps. That's... exactly what happened to Kenji's portfolio.

Now you're thinking like a quant. The next question is: how do we test whether returns are actually normal? And if they're not, how do we visualize the departure?

1.5 Testing for Normality

1.5.1 The QQ Plot

A **QQ plot** (Quantile–Quantile plot) compares the quantiles of the observed data against the quantiles of a theoretical distribution (typically normal). If the data are normally distributed, the points will fall along a straight 45-degree line.

Reading a QQ Plot

- **Points on the line**: Data matches the theoretical distribution
- **S-shaped curve**: Fat tails (points deviate at both ends)
- **Points above the line at left end**: Heavier left tail (more extreme losses)
- **Points below the line at right end**: Heavier right tail (more extreme gains)

For financial returns, you will almost always see the S-shape pattern, confirming fat tails.

1.5.2 The Jarque–Bera Test

The **Jarque–Bera (JB) test** formalizes the normality check by combining skewness (1.4) and excess kurtosis (1.5) into a single test statistic:

Jarque–Bera Test Statistic

$$JB = \frac{T}{6}\left(S^2 + \frac{K^2}{4}\right) \tag{1.6}$$

Under the null hypothesis H_0: returns are normally distributed, the JB statistic follows a $\chi^2(2)$ distribution.

Decision rule: Reject H_0 if $JB > \chi^2_{0.05}(2) = 5.99$.

In practice, financial return series produce JB statistics in the **hundreds or thousands**, providing overwhelming evidence against normality.

1.6 Python Implementation

Theory is important, but let's see it with real data. Alex, open your laptop. We're going to pull actual market data and see these effects for ourselves.

Loading and Computing Returns

```python
import numpy as np          # numerical computing
import pandas as pd         # data manipulation
```

```python
import yfinance as yf        # Yahoo Finance data API
from scipy import stats      # statistical functions
import matplotlib.pyplot as plt  # plotting

# Download S&P 500 daily prices (2015-2024)
sp500 = yf.download("^GSPC", start="2015-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()  # daily close prices

# Compute log returns: r_t = ln(P_t / P_{t-1})
log_returns = np.log(prices / prices.shift(1)).dropna()
print(f"Number of observations: {len(log_returns)}")
# Expected: ~2,500 trading days over 10 years
```

With our data loaded, let us compute the key distributional statistics and run the Jarque–Bera test:

Descriptive Statistics and Normality Tests

```python
# Basic statistics
mean_r = log_returns.mean()
std_r  = log_returns.std()
skew_r = log_returns.skew()
kurt_r = log_returns.kurtosis()  # excess kurtosis

print(f"Mean daily return:  {mean_r:.6f}")
print(f"Std deviation:      {std_r:.4f}")
print(f"Skewness:           {skew_r:.4f}")
print(f"Excess kurtosis:    {kurt_r:.4f}")

# Jarque-Bera test
jb_stat, jb_pvalue = stats.jarque_bera(log_returns.values)
print(f"\nJarque-Bera statistic: {jb_stat:.2f}")
print(f"p-value:               {jb_pvalue:.2e}")
```

Numbers tell part of the story, but visualization makes the departure from normality unmistakable:

QQ Plot and Histogram

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left panel: Histogram with normal overlay
axes[0].hist(log_returns, bins=100, density=True,
             alpha=0.7, color="#2D7D5E",
             label="Actual returns")
x = np.linspace(log_returns.min(), log_returns.max(), 200)
axes[0].plot(x, stats.norm.pdf(x, mean_r, std_r),
             "r-", lw=2, label="Normal distribution")
axes[0].set_title("S&P 500 Daily Log Returns vs Normal")
axes[0].set_xlabel("Log Return")
axes[0].set_ylabel("Density")
axes[0].legend()

# Right panel: QQ plot
stats.probplot(log_returns.values, dist="norm", plot=axes[1])
axes[1].set_title("QQ Plot vs Normal Distribution")
axes[1].get_lines()[0].set_color("#1B3A5C")
axes[1].get_lines()[1].set_color("#C0392B")

plt.tight_layout()
plt.savefig("week01_returns_analysis.png", dpi=150)
plt.show()
```

The histogram clearly shows the peak is taller and the tails are heavier than the normal curve. And the QQ plot has that S-shape you mentioned — the points fly off the line at both ends.

Exactly. And notice the Jarque–Bera statistic will be in the thousands for daily S&P 500 returns. The p-value will be essentially zero. Normality is rejected with absolute certainty.

AI in Practice: Anomaly Detection

Modern AI systems in finance use the same insight we just discovered — returns are *not* normal — to build better anomaly detection. Machine learning models like Isolation Forests

and autoencoders can identify unusual return patterns without assuming any specific distribution. In Week 8, we will explore how these techniques compare to traditional statistical approaches.

1.7 Application: Analyzing Kenji's Portfolio

Scene: Conference Room — 2:30 PM

After a working lunch, Alex presented the first findings to the team. The screen showed the histogram and QQ plot of the pension fund's equity portfolio returns.

Here's what we found. The portfolio returns from 2015 to 2024 show skewness of -0.73 and excess kurtosis of 13.4. The Jarque–Bera statistic is 4,892 with a p-value of essentially zero.

What do those numbers mean in plain language?

It means your returns have much heavier tails than normal, and they're asymmetric — large losses are more frequent than large gains. Specifically:

Table 1.1: Theoretical vs. empirical tail probabilities for the pension fund portfolio

Metric	**Normal Assumption**	**Actual Data**
Skewness	0	-0.73
Excess Kurtosis	0	13.4
Prob(daily loss $> 3\sigma$)	0.13%	1.7%
Prob(daily loss $> 5\sigma$)	0.00003%	0.12%

Under the normal model, a loss exceeding 5σ should happen once every 13,900 years of trading. In your data, it happened roughly every 3 years. That's why your VaR model failed — it was blind to the tails.

Fourteen thousand years versus three years. Now I understand why my board was furious.

1.8 Discussion: Why Does Normality Fail?

Why Are Financial Returns Non-Normal?

Consider the following factors and discuss which ones contribute to fat tails and negative skewness in financial returns:

1. **Volatility clustering**: Calm periods followed by turbulent periods (we'll model this with GARCH in Week 3)
2. **Leverage effect**: Bad news increases volatility more than good news (Week 4)
3. **Herding behavior**: Investors panic-selling together amplifies downside moves
4. **Information asymmetry**: Bad news spreads faster than good news
5. **Market microstructure**: Trading halts, margin calls, and forced liquidations create jumps

Which of these can be captured by better statistical models? Which require behavioral or institutional explanations?

1.9 Weekly Mission

◎ Mission 1: Distribution Diagnostics Report

Objective: Produce a complete distribution analysis report for the Nippon Life pension fund portfolio.

Deliverables:

1. Download daily return data for at least 3 major indices (S&P 500, Nikkei 225, FTSE 100) from 2015–2024
2. For each index, compute and report:
   - Mean, standard deviation, skewness, excess kurtosis
   - Jarque–Bera test statistic and p-value
   - Histogram with normal overlay
   - QQ plot against normal distribution
3. Create a comparison table summarizing the four moments across all indices
4. Write a 200-word executive summary explaining why the normal assumption is problematic for Kenji's portfolio. *Tip: An executive summary states the key finding, its implications, and a recommended next step.*

Bonus Challenge: The Tail Probability Table

For each index, compute the empirical probability of daily losses exceeding 1σ, 2σ, 3σ, 4σ, and 5σ. Compare these with the theoretical probabilities under the normal distribution. Present the results in a clear table.

Hint: Use `np.mean(returns < -k * std)` for each threshold k.

1.10 Key Takeaways

Key Takeaways

1. **Returns, not prices**: Financial analysis works with returns (preferably log returns) because they are approximately stationary and time-additive, unlike prices which tend to drift over time.
2. **Normality fails**: Financial returns exhibit **fat tails** (excess kurtosis $\gg 0$) and **negative skewness** ($S < 0$), meaning extreme losses are far more frequent than the normal distribution predicts.
3. **Four moments matter**: Mean and variance alone are insufficient to characterize financial return distributions. Skewness and kurtosis capture the asymmetry and tail heaviness that drive risk.
4. **QQ plots and JB test**: These are your first-line diagnostic tools for assessing distributional assumptions. Learn to read them fluently.
5. **Risk model implications**: Any risk model built on normality (including basic parametric VaR) will systematically underestimate the probability of large losses. This is not a theoretical curiosity — it has real financial consequences.

1.11 Further Reading

Further Reading

1. Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2), 223–236.
2. Fama, E. F. (1965). The behavior of stock-market prices. *The Journal of Business*, 38(1), 34–105.
3. Mandelbrot, B. (1963). The variation of certain speculative prices. *The Journal of Business*, 36(4), 394–419.
4. Jarque, C. M., & Bera, A. K. (1987). A test for normality of observations and regression residuals. *International Statistical Review*, 55(2), 163–172.
5. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapters 1–2.
6. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. Chapter 5.
7. McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management: Concepts, Techniques and Tools*, 2nd ed. Princeton University Press. Chapter 3.
8. Bollerslev, T., Todorov, V., & Li, S. Z. (2013). Jump tails, extreme dependencies, and the distribution of stock returns. *Journal of Econometrics*, 172(2), 307–324.
9. Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. [Foundation for Week 3]

Next Week Preview

Kenji drops a printed report on Alex's desk — daily losses from March 2020, highlighted in red. "You've shown me the problem. Now show me you can measure it. How volatile is our portfolio right now? And how quickly does that change?" Priya adds: "You can't use a single ruler to measure risk at all times." Next week, we learn to measure volatility — and discover that it never stays still.

Chapter 2

Week 2: Measuring Volatility

Learning Objectives

1. Compute historical volatility using rolling windows of different lengths
2. Apply the EWMA model and explain why it responds faster to new information
3. Define realized volatility and explain its advantages over close-to-close estimators
4. Interpret the VIX index as the market's forward-looking volatility expectation
5. Compare multiple volatility measures around the COVID-19 crisis period

2.1 This Week's Story: One Ruler Is Not Enough

Scene: VolTech Analytics Office — Tuesday, 8:45 AM

Alex arrived early, still processing yesterday's findings. The comparison table was taped to his monitor: a 5σ event every 3 years versus once every 14,000 years. The normal distribution was dangerously wrong. But Kenji's question from yesterday afternoon echoed in his mind: *"How volatile is our portfolio right now?"*

Good morning, Alex. I see you're staring at yesterday's table. Good. Now forget it. Knowing that returns have fat tails is necessary but not sufficient. We need to measure volatility — and measure it in real time.

Can't we just compute the standard deviation of returns? We did that yesterday.

We can. But which returns? The last 20 days? The last year? The last decade? The answer changes dramatically depending on your window. And here's the deeper problem: volatility is not constant. It moves. It clusters. Calm weeks are followed by turbulent weeks. You can't use a single ruler to measure risk at all times.

So we need a ruler that adjusts itself.

Exactly. Today I'll show you four different rulers — and we'll find out which one warned us fastest when COVID-19 hit.

2.2 Historical Volatility: The Rolling Window

The simplest approach to measuring volatility is the **rolling window** (or **historical volatility**) method. Pick a window of N days, compute the standard deviation of returns within that window, and slide the window forward one day at a time.

Rolling Window Volatility

For a window of N observations ending at time t:

$$\hat{\sigma}_t^{\text{roll}} = \sqrt{\frac{1}{N-1}\sum_{i=0}^{N-1}(r_{t-i} - \bar{r}_{t,N})^2} \tag{2.1}$$

where $\bar{r}_{t,N} = \frac{1}{N}\sum_{i=0}^{N-1} r_{t-i}$ is the rolling mean. To annualize daily volatility, multiply by $\sqrt{252}$.

The Window Length Trade-Off

Choosing N involves a fundamental bias–variance trade-off:

- **Short window** (e.g., $N = 20$ days $\approx$ 1 month): Responds quickly to regime changes but is noisy (high variance)
- **Long window** (e.g., $N = 252$ days $\approx$ 1 year): Smooth and stable but slow to react to sudden shifts (high bias)
- **Common choices**: 20 days (short-term), 60 days (medium-term), 252 days (long-term)

There is no universally "correct" window — the best choice depends on your application. Risk managers who need early warning signals prefer short windows; portfolio analysts tracking long-term trends prefer longer ones.

A critical limitation of the rolling window is that it assigns **equal weight** to all observations inside the window. A return from 250 days ago counts exactly the same as yesterday's return. When markets shift abruptly — as they did in March 2020 — the rolling window is slow to reflect the new reality.

In February 2020, our 252-day volatility estimate was about 12% annualized. By mid-March, actual volatility had exploded. But our yearly rolling window still included all of 2019's calm data. It took weeks to catch up.

2.3 EWMA: Giving More Weight to Recent Data

The **Exponentially Weighted Moving Average (EWMA)** model solves the equal-weighting problem by assigning *exponentially declining* weights to older observations. Recent returns matter more; distant returns fade away.

EWMA Variance

The EWMA variance estimate updates recursively:

$$\hat{\sigma}_t^2 = \lambda\, \hat{\sigma}_{t-1}^2 + (1-\lambda)\, r_{t-1}^2 \tag{2.2}$$

where $\lambda \in (0,1)$ is the **decay factor**. Note that we use r_{t-1}^2 rather than $(r_{t-1} - \bar{r})^2$: for daily returns the mean is negligibly small relative to the variance, so this zero-mean simplification is standard practice (and is used by RiskMetrics).
Equivalently, the EWMA variance at time t is a weighted sum of all past squared returns:

$$\hat{\sigma}_t^2 = (1-\lambda) \sum_{i=1}^{\infty} \lambda^{i-1}\, r_{t-i}^2 \tag{2.3}$$

The weight on the return i periods ago is $(1-\lambda)\lambda^{i-1}$, which decays geometrically.

Choosing λ: The RiskMetrics Standard

J.P. Morgan's influential **RiskMetrics** system (1996) recommended $\lambda = 0.94$ for daily data, which has become the de facto industry standard. At this value:

- Yesterday's squared return receives weight 0.06 (= $1 - 0.94$)
- The return from 10 days ago receives weight $0.06 \times 0.94^9 \approx 0.034$
- The return from 30 days ago receives weight $0.06 \times 0.94^{29} \approx 0.010$
- The **effective number of observations** (the mean lag of the weight distribution) is approximately $\frac{1}{1-\lambda} = \frac{1}{0.06} \approx 17$ days — meaning the EWMA behaves roughly like a 17-day average, but with exponentially declining weights rather than a hard cut-off

Higher λ (e.g., 0.97) gives a smoother estimate; lower λ (e.g., 0.90) gives a more reactive one.

Notice that EWMA has no window to choose — just one parameter, λ. And unlike the rolling window, old data never abruptly "falls out" of the estimate. It fades smoothly. That's why EWMA doesn't produce the ghosting artifacts you see when an extreme observation suddenly leaves a rolling window.

What do you mean by "ghosting artifacts"?

Suppose there's a market crash on day 1 and calm returns for the next $N-1$ days. With a rolling window, the crash observation contributes to the volatility estimate for exactly N days, then vanishes — causing a sudden artificial drop. EWMA avoids this because the crash fades gradually.

EWMA as a Special GARCH

EWMA is actually a special case of the GARCH model you will meet in Week 3 — specifically, GARCH(1,1) with no long-run variance term ($\omega = 0$) and $\alpha + \beta = 1$. This means EWMA implicitly assumes that volatility has no fixed "home base" to return to. GARCH fixes this limitation by adding a mean-reversion component.

2.4 Realized Volatility: Using Intraday Data

Both rolling windows and EWMA use daily closing prices. But within each trading day, prices fluctuate continuously. **Realized volatility** exploits this intraday information to produce a more accurate volatility estimate.

Realized Volatility

Given M equally spaced intraday returns $r_{t,1}, r_{t,2}, \ldots, r_{t,M}$ on day t:

$$RV_t = \sum_{j=1}^{M} r_{t,j}^2 \tag{2.4}$$

As $M \to \infty$ (sampling frequency increases), RV_t converges to the true **integrated variance** of the day — a model-free measure of volatility.

Why Squaring Intraday Returns Works

The key insight comes from probability theory: for a continuous-time process, the sum of squared increments converges to the **quadratic variation** — the total "variation path" of the process. In simple terms, more frequent sampling gives a more precise measurement of how much the price actually moved during the day, rather than just comparing the opening and closing prices.

*In practice, we typically use 5-minute returns ($M = 78$ for a 6.5-hour trading day). Going to higher frequencies introduces **microstructure noise** — the bid-ask bounce and other frictions distort the estimate. There are noise-robust estimators, but 5-minute sampling is a good practical starting point.*

Realized volatility requires intraday data, which may not be available for all assets or time periods. For our project with Kenji's pension fund, we will primarily use daily close-to-close measures (rolling window and EWMA) but keep realized volatility in mind as the gold standard.

2.5 The VIX: Market's Fear Gauge

The previous three methods are all **backward-looking** — they estimate volatility from past data. The **VIX index** takes a fundamentally different approach: it extracts volatility expectations from **options prices**, making it **forward-looking**.

The VIX Index

The CBOE Volatility Index (VIX) measures the market's expectation of 30-day volatility for the S&P 500, derived from the prices of S&P 500 index options. Key properties:

- **Forward-looking**: Reflects what traders *expect* volatility to be, not what it has been
- **Annualized**: VIX = 20 means the market expects annualized volatility of ≈20%, or daily volatility of $20\%/\sqrt{252} \approx 1.26\%$
- **Fear gauge**: VIX typically spikes during market crashes and declines during calm periods
- **Mean-reverting**: Extreme VIX levels tend to pull back toward the long-run average (≈18–20)

VIX During COVID-19

On March 16, 2020, the VIX reached **82.69** — the highest level since the 2008 financial crisis and one of the highest readings in the index's history. For context:

- Normal markets: VIX $\approx$ 12–18
- Elevated anxiety: VIX $\approx$ 20–30
- Crisis mode: VIX $>$ 30
- Extreme panic: VIX $>$ 50
- March 16, 2020: VIX = 82.69 — off the charts

The VIX had already risen above 40 by late February, *before* the worst of the sell-off. This is a crucial observation: the VIX often leads realized volatility because option traders price in fear before it fully materializes.

The Volatility Smile

When computing the VIX, the CBOE uses a strip of out-of-the-money options at different strike prices. Interestingly, the implied volatility extracted from these options is not constant across strikes — it forms a U-shaped pattern called the **volatility smile** (or **smirk** for equity options, where downside strikes show higher implied volatility). This pattern is itself evidence that the market prices in fat tails and negative skewness — exactly the stylized facts we will study in Week 4.

Implied vs. Realized Volatility

The difference between implied volatility (from options, e.g., VIX) and realized volatility (from actual returns) is called the **variance risk premium**. On average, implied volatility exceeds realized volatility, meaning option sellers earn a premium for bearing volatility risk. This premium is economically significant and is one of the most robust findings in empirical finance.

So the VIX was screaming "danger" while our rolling window was still sleeping. If we had watched the VIX, we could have acted sooner.

In principle, yes. But the VIX measures S&P 500 implied volatility, not your specific portfolio's risk. We need models that can estimate your portfolio's volatility in real time. That's what GARCH models do — but that's Week 3. For now, let's see these methods in action.

2.6 Python Implementation

Time to build our volatility dashboard. Alex, we'll compare rolling windows, EWMA, and VIX for the S&P 500 around the COVID-19 period.

Loading Data and Computing Rolling Volatility

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt

# Download S&P 500 data (2018-2024)
sp500 = yf.download("^GSPC", start="2018-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()
log_returns = np.log(prices / prices.shift(1)).dropna()

# Rolling window volatility (annualized)
vol_20  = log_returns.rolling(20).std() * np.sqrt(252)
vol_60  = log_returns.rolling(60).std() * np.sqrt(252)
vol_252 = log_returns.rolling(252).std() * np.sqrt(252)
```

Next, we implement the EWMA recursion (2.2) with the RiskMetrics decay factor $\lambda = 0.94$:

EWMA Volatility

```python
def ewma_volatility(returns, lam=0.94):
    """Compute EWMA volatility (annualized)."""
    n = len(returns)
    var = np.zeros(n)
    var[0] = returns.iloc[0] ** 2  # initialize
    for t in range(1, n):
        var[t] = lam * var[t-1] + (1 - lam) * returns.iloc[t-1]**2
    vol = pd.Series(np.sqrt(var) * np.sqrt(252),
                    index=returns.index)
    return vol

```

```
vol_ewma = ewma_volatility(log_returns, lam=0.94)
# Note: pandas provides ewm() for production use:
# log_returns.ewm(alpha=1-0.94).std() * np.sqrt(252)
```

Finally, we download VIX data to compare backward-looking measures with the market's forward-looking expectation:

Downloading VIX and Plotting the Dashboard

```
# Download VIX index
vix = yf.download("^VIX", start="2018-01-01",
                    end="2024-12-31")
vix_close = vix["Close"].squeeze()

# Plot comparison
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(vol_20.index, vol_20, label="Rolling 20d",
        alpha=0.7, color="#1B3A5C")
ax.plot(vol_60.index, vol_60, label="Rolling 60d",
        alpha=0.7, color="#5D6D7E")
ax.plot(vol_ewma.index, vol_ewma, label="EWMA (0.94)",
        alpha=0.9, color="#2D7D5E", linewidth=2)
ax.plot(vix_close.index, vix_close / 100, label="VIX / 100",
        alpha=0.8, color="#C0392B", linewidth=1.5)

# Mark COVID-19 crash
ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red", label="COVID-19 crash")
ax.set_title("Volatility Measures: S&P 500 (2018-2024)")
ax.set_ylabel("Annualized Volatility")
ax.legend(loc="upper right")
ax.set_ylim(0, 1.0)
plt.tight_layout()
plt.savefig("week02_volatility_comparison.png", dpi=150)
plt.show()
```

The chart is striking. The VIX spiked first — already above 40% by late February. EWMA followed within days. But the 252-day rolling window barely moved until April. It's like watching three different weather stations report the same storm at different times.

2.7 Application: The Speed of Fear

Scene: Conference Room — Wednesday, 3:00 PM

Alex projected the volatility dashboard on the screen. Kenji leaned forward, tracing the lines with his finger.

I ran all four measures on the S&P 500 around the COVID-19 period. Here's what I found:

Table 2.1: Speed of volatility response during the COVID-19 crisis (S&P 500)

Measure	Feb 19 Level	Mar 16 Peak	Response Time
Rolling 20-day	8.2%	89.7%	∼2 weeks
Rolling 60-day	10.4%	54.1%	∼4 weeks
Rolling 252-day	14.1%	23.8%	∼8+ weeks
EWMA ($\lambda = 0.94$)	7.9%	83.4%	∼1 week
VIX (implied)	14.4%	82.7%	Immediate

The key findings are:

1. ***VIX led everything**: It crossed 30% on February 24th, four days before the S&P 500 entered bear market territory*
2. ***EWMA was the fastest backward-looking measure**: Its exponential weighting meant the February sell-off immediately dominated the estimate*
3. ***Rolling 252-day was nearly useless**: Even at the peak of the crisis, it was still "diluted" by 8+ months of calm 2019 data*

So EWMA would have given us a warning signal at least a week earlier than our 252-day rolling window. That week could have saved us hundreds of millions of yen.

This is exactly the kind of real-time intelligence our clients need. If we can package this into a live dashboard product, every institutional investor in Asia will want it.

And that's just the beginning. EWMA is better than a fixed rolling window, but it's still a relatively simple model. It doesn't capture the phenomenon that volatility tends to be "sticky" — high volatility today predicts high volatility tomorrow. For that, we need the GARCH family of models. But first, let me introduce a concept that will become central to our entire risk framework.

2.8 Volatility Stylized Facts

Before moving to more advanced models (Week 3 onward), let us summarize the key empirical patterns — **stylized facts** — of financial volatility. These patterns are remarkably robust across markets, time periods, and asset classes:

Stylized Facts of Volatility

1. **Volatility clustering**: Large returns (positive or negative) tend to be followed by large returns, and small returns by small returns. Formally, returns are uncorrelated but their absolute values are positively autocorrelated.
2. **Mean reversion**: Volatility tends to return to a long-run average. Extreme levels (very high or very low) are temporary.
3. **Asymmetry (leverage effect)**: Negative returns tend to increase future volatility more than positive returns of the same magnitude. We will model this in Week 4.
4. **Long memory**: The autocorrelation of absolute returns decays slowly, suggesting volatility has a long memory — it remembers distant shocks.
5. **Co-movement**: Volatility tends to increase across markets simultaneously, especially during crises (contagion).

Alex, these five facts are the "design specifications" for any good volatility model. The rolling window captures none of them. EWMA captures clustering partially. GARCH captures clustering and mean reversion. GJR-GARCH adds asymmetry. Each week, we add another layer.

It's like building a progressively better signal filter — each one captures more structure in the data.

Exactly. Your engineering intuition is spot on.

2.9 Discussion: Choosing Your Volatility Measure

Which Ruler for Which Job?

Different volatility measures suit different purposes. Consider the following scenarios and discuss which measure(s) would be most appropriate:

1. A **portfolio manager** rebalancing monthly — needs a stable estimate, not one that whipsaws daily
2. A **risk manager** monitoring daily VaR limits — needs the earliest possible warning signal
3. A **derivatives trader** pricing options — needs the market's forward-looking expectation
4. A **regulator** assessing systemic risk — needs a measure that captures cross-market contagion
5. An **academic researcher** studying volatility dynamics — needs the most accurate measure of "true" volatility

Is there a single measure that works for all purposes? Why or why not?

2.10 Weekly Mission

◎ Mission 2: Volatility Dashboard for the Pension Fund

Objective: Build a comprehensive volatility comparison dashboard for Kenji's portfolio analysis.

Deliverables:

1. Download daily data for the Nikkei 225, S&P 500, and FTSE 100 from 2018–2024
2. For each index, compute:
   - Rolling window volatility: 20-day, 60-day, 252-day
   - EWMA volatility with $\lambda = 0.94$

3. Plot a volatility dashboard for each index (all measures on one chart, with the COVID-19 period highlighted)
4. Create a summary table: for each measure and each index, report the pre-COVID level (Feb 19), peak level, and approximate response time (as in Table 2.1)
5. Write a 200-word memo to Kenji recommending which volatility measure VolTech should use for daily risk monitoring, and why

Bonus Challenge: The Decay Factor Experiment

Run the EWMA model with $\lambda = 0.90$, 0.94, and 0.97 on the Nikkei 225. Plot all three on the same chart and discuss:

- Which reacts fastest to new shocks?
- Which produces the smoothest estimate?
- If you had to choose one λ for daily risk management, which would you pick and why?

Hint: The effective window length is approximately $\frac{1}{1-\lambda}$ days.

2.11 Key Takeaways

Key Takeaways

1. **Volatility is not constant**: It clusters, mean-reverts, and responds asymmetrically to good and bad news. Any useful volatility measure must account for this time-varying nature.
2. **Rolling window**: Simple and intuitive but assigns equal weight to all observations. Slow to react to regime changes, and suffers from ghosting when extreme observations leave the window.
3. **EWMA**: Exponentially declining weights prioritize recent data. The industry standard $\lambda = 0.94$ provides a good balance between responsiveness and smoothness.
4. **Realized volatility**: The gold standard when intraday data is available. Converges to true integrated variance as sampling frequency increases.
5. **VIX**: Forward-looking, derived from option prices. Often leads realized volatility during crises because it captures traders' fear before it fully materializes in returns.
6. **No one-size-fits-all**: The right measure depends on the application. Risk managers need responsiveness (EWMA, VIX); portfolio managers may prefer stability (longer rolling windows).

2.12 Further Reading

Further Reading

1. J.P. Morgan (1996). *RiskMetrics — Technical Document*, 4th ed. [The foundational EWMA reference]
2. Andersen, T. G., & Bollerslev, T. (1998). Answering the skeptics: yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4), 885–905.
3. Andersen, T. G., Bollerslev, T., Diebold, F. X., & Labys, P. (2003). Modeling and forecasting realized volatility. *Econometrica*, 71(2), 579–625.
4. Whaley, R. E. (2009). Understanding the VIX. *The Journal of Portfolio Management*, 35(3), 98–105.
5. Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2), 223–236.
6. Poon, S.-H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: a review. *Journal of Economic Literature*, 41(2), 478–539.
7. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 3.
8. Barndorff-Nielsen, O. E., & Shephard, N. (2002). Econometric analysis of realized volatility and its use in estimating stochastic volatility models. *Journal of the Royal Statistical Society: Series B*, 64(2), 253–280.
9. Black, F. (1976). Studies of stock price volatility changes. *Proceedings of the American Statistical Association, Business and Economics Section*, 177–181. [Origin of the leverage effect]

Next Week Preview

After the meeting, Priya caught Alex at the coffee machine. "Your volatility dashboard is solid work. But let me ask you something — EWMA reacts faster than the rolling window, sure. But does it actually predict tomorrow's volatility? Or does it just describe what already happened?" She pulled up her laptop, showing an LSTM neural network training curve. "I've been building something that tries to predict volatility, not just measure it. Next week, Dr. Lin is going to show you the model that started it all — GARCH. And then we'll see if your statistical model can beat my AI." Next week: we enter the world of ***GARCH*** *— and discover that volatility has a memory.*

Chapter 3

Week 3: Volatility Clustering and GARCH

Learning Objectives

1. Identify volatility clustering in return series and test for it using the autocorrelation function of squared returns
2. Explain the ARCH model and its role as the foundation of conditional volatility modeling
3. Derive and interpret the GARCH(1,1) model, including the roles of ω, α, and β
4. Connect GARCH to EWMA and explain why GARCH adds mean reversion
5. Estimate a GARCH(1,1) model using maximum likelihood and interpret the output in Python

3.1 This Week's Story: Volatility Has a Memory

Scene: VolTech Analytics — Monday, 9:00 AM

Two weeks into his new role, Alex had built a volatility dashboard that impressed Kenji. EWMA was fast. The VIX was forward-looking. But a nagging question from Priya's coffee-machine challenge kept him up over the weekend: can we actually *predict* volatility, not just measure it?

Dr. Lin, I've been thinking about what Priya said. EWMA tells us what volatility is now, but can we forecast what it will be tomorrow?

That's the right question. And the answer is yes — because volatility has a memory. Look at this chart of S&P 500 daily returns from 2018 to 2024. What pattern do you see?

The returns jump around randomly, but the size of the jumps isn't random at all. There are quiet periods and turbulent periods, and they tend to cluster together.

Exactly. That's called ***volatility clustering****, and it's one of the most important stylized facts in finance. Mandelbrot noticed it in the 1960s: "large changes tend to be followed by large changes — of either sign — and small changes tend to be followed by small changes." If volatility is high today, it's very likely to be high tomorrow. That is what makes it predictable.*

So we can exploit the clustering pattern to build a forecasting model?

Precisely. Today I'm going to introduce you to the model that revolutionized how we think about volatility. It's called GARCH, and it's one of the workhorse tools in quantitative finance. Robert Engle won the Nobel Prize in 2003 for developing the ideas behind it. Let's start from the beginning.

3.2 Detecting Volatility Clustering

Before building a model, we need to verify that the phenomenon we want to capture actually exists in our data. Here is the formal test.

Recall that daily log returns r_t are approximately uncorrelated — knowing today's return tells you little about tomorrow's return. But *squared* returns r_t^2 (a proxy for variance) are strongly autocorrelated: knowing today's r_t^2 tells you a lot about tomorrow's r_t^2.

Autocorrelation of Squared Returns

The sample autocorrelation function (ACF) of squared returns at lag k is:

$$\hat{\rho}(k) = \frac{\sum_{t=k+1}^{T}(r_t^2 - \overline{r^2})(r_{t-k}^2 - \overline{r^2})}{\sum_{t=1}^{T}(r_t^2 - \overline{r^2})^2} \tag{3.1}$$

where $\overline{r^2} = \frac{1}{T}\sum_{t=1}^{T} r_t^2$ is the sample mean of squared returns.
If $\hat{\rho}(k)$ is significantly positive for many lags k, this is evidence of **volatility clustering**: large squared returns are followed by large squared returns.

The Ljung–Box Test

To formally test whether squared returns are autocorrelated, we use the **Ljung–Box test**:

$$Q(m) = T(T+2)\sum_{k=1}^{m}\frac{\hat{\rho}(k)^2}{T-k} \tag{3.2}$$

Under the null hypothesis of no autocorrelation, $Q(m) \sim \chi^2(m)$. A small p-value (e.g., < 0.05) rejects the null, confirming that volatility clustering is statistically significant. In practice, the Ljung–Box test on squared returns almost always rejects for financial data — volatility clustering is nearly universal.

An alternative is Engle's **ARCH-LM test**, which regresses r_t^2 on its own lags and tests the joint significance. Both tests detect the same phenomenon; the Ljung–Box is simpler, while the ARCH-LM has a direct connection to the ARCH model specification.

Think of it this way, Alex. Returns are like the direction of the wind — hard to predict. Squared returns are like the strength of the wind — when a storm starts, it doesn't stop suddenly. That persistence is what GARCH models.

3.3 The ARCH Model: Where It All Began

In 1982, Robert Engle introduced the **Autoregressive Conditional Heteroskedasticity (ARCH)** model, the first formal framework for time-varying volatility. The key insight: we decompose a return into a predictable volatility component and a random shock.

Return Decomposition

We model the return r_t as:

$$r_t = \sigma_t\, z_t, \qquad z_t \sim \text{i.i.d.}\,(0,1) \tag{3.3}$$

where:

- σ_t is the **conditional standard deviation** — the volatility we want to model, which can change over time
- z_t is a standardized shock, drawn independently from a distribution with mean 0 and variance 1 (often assumed normal)
- "Conditional" means σ_t depends on information available at time $t-1$ (past returns, past volatilities)

The idea is simple: the return at time t equals a time-varying "volume knob" (σ_t) multiplied by a random "direction" (z_t).

So σ_t controls how big the return can be, and z_t controls the random direction?

Perfect analogy. Now the question is: how does σ_t evolve? Engle's ARCH model says today's variance depends on recent past shocks.

The ARCH(q) Model

The ARCH(q) model specifies the conditional variance as:

$$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \cdots + \alpha_q r_{t-q}^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2 \tag{3.4}$$

where:

- $\omega > 0$ is a constant baseline variance
- $\alpha_i \geq 0$ are the ARCH coefficients (weights on past squared returns)
- q is the order — how many lags of squared returns to include

For the variance to remain positive, we need $\omega > 0$ and $\alpha_i \geq 0$ for all i.

Why ARCH Captures Clustering

Consider ARCH(1): $\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2$. If yesterday's return r_{t-1} was large (a shock), then r_{t-1}^2 is large, making σ_t^2 large, which means today's return is drawn from a wider distribution — increasing the probability of another large return. This creates clustering: large returns beget large returns.

ARCH was a breakthrough, but it has a practical problem. To capture the slow decay of volatility autocorrelation, you often need many lags q — sometimes 10, 20, or more. That means estimating many parameters, which is inefficient. Tim Bollerslev had a better idea.

3.4 GARCH(1,1): The Workhorse Model

In 1986, Tim Bollerslev extended Engle's ARCH by adding the *previous conditional variance* as an additional predictor. This one change dramatically reduced the number of parameters needed

and made the model practical for real-world use.

The GARCH(1,1) Model

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2 \tag{3.5}$$

where:

- $\omega > 0$: the "base level" of variance
- $\alpha \geq 0$: the **shock coefficient** — how much yesterday's squared return (new information) affects today's variance
- $\beta \geq 0$: the **persistence coefficient** — how much yesterday's variance carries over to today
- $\alpha + \beta < 1$: the **stationarity condition**, ensuring volatility doesn't explode. When $\alpha + \beta = 1$, we get IGARCH (= EWMA), where variance forecasts never decay and the unconditional variance is undefined. When $\alpha + \beta > 1$, variance forecasts grow without bound — the model is explosive.

The "(1,1)" means one lag of r_{t-1}^2 and one lag of σ_{t-1}^2. In practice, GARCH(1,1) fits the vast majority of financial time series remarkably well.

A key mathematical insight: by repeatedly substituting σ_{t-1}^2 into itself, GARCH(1,1) is equivalent to an ARCH(∞) with geometrically decaying coefficients: $\sigma_t^2 = \frac{\omega}{1-\beta} + \alpha \sum_{i=1}^{\infty} \beta^{i-1}\, r_{t-i}^2$. This is why a single GARCH lag replaces infinitely many ARCH lags — the β term encodes the entire history through exponential decay.

The following table explains each component's role intuitively:

Table 3.1: Anatomy of GARCH(1,1): $\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2$

Parameter	Role	Typical Value
ω	Floor / base variance	Small positive
α	Reaction to new shocks	0.05–0.15
β	Memory / persistence	0.80–0.95
$\alpha + \beta$	Total persistence	0.95–0.99

So β is why volatility has a memory. If $\beta = 0.85$, then 85% of yesterday's variance carries forward. That's a strong memory.

Exactly. And α is why volatility responds to new shocks. If the market crashes today, r_t^2 is huge, which pushes up σ_{t+1}^2 through the α term. Then that elevated σ_{t+1}^2 persists through the β term. That's how GARCH produces volatility clustering.

GARCH's Long-Run Variance

A key advantage of GARCH over EWMA is **mean reversion**. The long-run (unconditional) variance of a stationary GARCH(1,1) is:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta} \tag{3.6}$$

This is the "home base" that volatility gravitates toward. After a shock pushes volatility up, the model gradually pulls it back to $\bar{\sigma}^2$. This is exactly the mean reversion stylized fact we identified in Week 2.

GARCH vs. EWMA: The Missing Piece

We noted in Week 2 that EWMA is a special case of GARCH(1,1) with $\omega = 0$ (equivalently, IGARCH with $\alpha + \beta = 1$). Setting $\omega = 0$ removes the long-run anchor:

- **EWMA**: $\sigma_t^2 = (1 - \lambda)\, r_{t-1}^2 + \lambda\, \sigma_{t-1}^2$ (no mean reversion)
- **GARCH**: $\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2$ (mean-reverts to $\frac{\omega}{1-\alpha-\beta}$)

With EWMA, if volatility spikes, it has no reason to come back down — it drifts. With GARCH, the ω term acts like a spring, pulling volatility back to equilibrium. For risk management, mean reversion is crucial: we need to know not just that volatility is high, but how quickly it will normalize.

3.5 How Fast Does Volatility Decay?

A natural question: after a shock, how long does it take for GARCH volatility to return halfway to its long-run level?

Volatility Half-Life

The **half-life** of a volatility shock in GARCH(1,1) is:

$$h = \frac{\ln(0.5)}{\ln(\alpha + \beta)} \tag{3.7}$$

This tells us how many days it takes for the impact of a shock to decay by 50%.

So if $\alpha + \beta = 0.97$, the half-life is $\ln(0.5)/\ln(0.97) \approx 23$ days. That means a shock is still half-present after nearly a month!

That's right. Financial volatility is highly persistent. With typical values of $\alpha + \beta \approx 0.95$–$0.99$, half-lives range from about 14 to 69 days. This is why a market crash doesn't calm down overnight — the elevated volatility lingers for weeks or months.

3.6 Maximum Likelihood Estimation

How do we find the optimal ω, α, and β for a given data set? The standard approach is **Maximum Likelihood Estimation (MLE)** — find the parameters that make the observed data most probable.

MLE for GARCH(1,1)

Assuming the standardized residuals $z_t = r_t/\sigma_t$ follow a normal distribution, the log-likelihood for T observations is:

$$\mathcal{L}(\omega, \alpha, \beta) = -\frac{1}{2}\sum_{t=1}^{T}\left[\ln(2\pi) + \ln(\sigma_t^2) + \frac{r_t^2}{\sigma_t^2}\right] \tag{3.8}$$

where each σ_t^2 is computed recursively from Equation (3.5). MLE finds the parameter values that maximize this function — intuitively, the parameters that best explain the observed pattern of large and small returns.

Wait — this looks circular. We need σ_t^2 to compute the likelihood, but σ_t^2 depends on the parameters we're trying to estimate.

Good observation. That's why we use numerical optimization. For any candidate parameter set (ω, α, β):

1. *Initialize σ_1^2 (typically with the sample variance)*
2. *Recursively compute $\sigma_2^2, \sigma_3^2, \ldots, \sigma_T^2$ using Equation (3.5)*
3. *Evaluate the log-likelihood (3.8)*
4. *Repeat with different parameter values until the log-likelihood is maximized*

The optimizer tries thousands of combinations to find the best fit. In practice, this is fast

> *— a few seconds for typical data sets.*

Beyond the Normal Distribution

In practice, the standardized residuals z_t often still show fat tails even after accounting for GARCH effects. This has led to extensions using heavier-tailed distributions:

- **Student's t-distribution**: Adds a degrees-of-freedom parameter ν to accommodate fat tails. This is the most common choice in practice.
- **GED (Generalized Error Distribution)**: Allows both fatter and thinner tails than the normal.
- **Skewed t-distribution**: Adds asymmetry to the Student's t.

Using a t-distribution typically improves model fit and produces more realistic risk estimates.

3.7 Python Implementation

> *All right, Alex. Time to put theory into practice. We'll use the `arch` library — it handles GARCH estimation and makes the process straightforward. Let me walk you through the full pipeline.*

Step 1: Data Preparation and Clustering Visualization

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.stats.diagnostic import acorr_ljungbox

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()
# Scale returns to percentage (arch library convention)

# Visualize volatility clustering
```

```python
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)
axes[0].plot(returns.index, returns, color="#1B3A5C",
             linewidth=0.5)
axes[0].set_title("S&P 500 Daily Log Returns (%)")
axes[0].set_ylabel("Return (%)")

axes[1].plot(returns.index, returns ** 2, color="#C0392B",
             linewidth=0.5)
axes[1].set_title("Squared Returns (Volatility Proxy)")
axes[1].set_ylabel(r"$r_t^2$")
plt.tight_layout()
plt.savefig("week03_clustering.png", dpi=150)
plt.show()
```

The squared returns plot reveals the clustering pattern clearly: periods of calm (small r_t^2) alternate with periods of turbulence (large r_t^2), and these regimes persist for weeks or months. Next, we confirm this statistically:

Step 2: Testing for Volatility Clustering

```python
# ACF of squared returns
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(returns.values, lags=30, ax=axes[0],
         title="ACF of Returns")
plot_acf(returns.values ** 2, lags=30, ax=axes[1],
         title="ACF of Squared Returns")
plt.tight_layout()
plt.savefig("week03_acf.png", dpi=150)
plt.show()

# Ljung-Box test on squared returns
lb_test = acorr_ljungbox(returns ** 2, lags=10,
                         return_df=True)
print("Ljung-Box test on squared returns:")
print(lb_test)
# Expect very small p-values -> reject H0 of no clustering
```

With clustering confirmed, we proceed to fit the GARCH(1,1) model using MLE via the `arch` package:

Step 3: Fitting GARCH(1,1)

```python
from arch import arch_model

# Specify GARCH(1,1) with Student's t distribution
model = arch_model(returns, vol="Garch", p=1, q=1,
                   dist="t")
result = model.fit(disp="off")
print(result.summary())

# Extract parameters
omega = result.params["omega"]
alpha = result.params["alpha[1]"]
beta  = result.params["beta[1]"]
nu    = result.params["nu"]  # t-distribution DoF

print(f"\nomega = {omega:.6f}")
print(f"alpha = {alpha:.4f}")
print(f"beta  = {beta:.4f}")
print(f"alpha + beta = {alpha + beta:.4f}")
print(f"Long-run vol = {np.sqrt(omega/(1-alpha-beta)
                        * 252):.2f}% annualized")
print(f"Half-life = {np.log(0.5)/np.log(alpha+beta):.1f}"
      " days")
print(f"Student t DoF = {nu:.2f}")
```

The output shows $\alpha \approx 0.09$ and $\beta \approx 0.90$, so $\alpha + \beta \approx 0.99$. That's extremely persistent — the half-life is about 69 days!

That's typical for equity indices. The Student's t degrees of freedom around 5–7 confirms that even after GARCH filtering, residuals are heavier-tailed than normal. Now let's plot the estimated conditional volatility.

Next, we extract the fitted conditional volatility path and visualize how the model tracks the evolving risk level:

Step 4: Plotting Conditional Volatility

```python
# Extract conditional volatility (daily, in %)
cond_vol = result.conditional_volatility  # daily %

# Plot returns with +/- 1 sigma volatility bands
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(returns.index, returns, color="#5D6D7E",
        linewidth=0.4, alpha=0.6, label="Daily returns")
ax.plot(cond_vol.index, cond_vol, color="#C0392B",
        linewidth=1.5,
        label="GARCH(1,1) cond. vol (%/day)")
ax.plot(cond_vol.index, -cond_vol, color="#C0392B",
        linewidth=1.5)
ax.fill_between(cond_vol.index, cond_vol, -cond_vol,
                alpha=0.1, color="#C0392B")

# Mark COVID-19 crash
ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red", label="COVID-19 crash")
ax.set_title("S&P 500 Returns with GARCH(1,1) "
             "Conditional Volatility")
ax.set_ylabel("Daily Return / Volatility (%)")
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("week03_garch_fit.png", dpi=150)
plt.show()
```

The volatility envelope widens during crisis periods and narrows during calm periods — exactly what we expected. The model is capturing the clustering pattern.

3.8 Application: GARCH Meets the COVID Crash

Scene: Conference Room — Thursday, 2:00 PM

Alex presented his GARCH results to the team. The model fit looked impressive — the conditional volatility tracked every crisis period in the past decade. But Dr. Lin had arranged a tougher test.

Alex, fitting a model to the full sample is like studying for an exam with the answer key. The real test is whether the model can predict volatility before seeing it. Let me show you what I mean.

The key test: estimate GARCH using data up to January 31, 2020 (before COVID), then forecast the conditional volatility for the February–April 2020 period and compare it with what actually happened.

Pre-COVID Estimation and Forecasting

```python
# Fit GARCH(1,1) on full data, but use last_obs to
# limit estimation to pre-COVID period
model_pre = arch_model(returns, vol="Garch", p=1, q=1,
                       dist="t")
res_pre = model_pre.fit(last_obs="2020-01-31",
                        disp="off")

# One-step-ahead forecasts starting from Feb 2020
forecasts = res_pre.forecast(horizon=1,
                             start="2020-02-01")
fc_vol = np.sqrt(forecasts.variance.dropna()
                 ) * np.sqrt(252)

# Long-run volatility from pre-COVID parameters
omega_pre = res_pre.params["omega"]
alpha_pre = res_pre.params["alpha[1]"]
beta_pre  = res_pre.params["beta[1]"]
longrun_vol = (np.sqrt(omega_pre /
    (1 - alpha_pre - beta_pre)) * np.sqrt(252))
peak_vol = fc_vol.max().iloc[0]

```

```python
print(f"Pre-COVID long-run vol: {longrun_vol:.1f}%")
print(f"Peak GARCH forecast (Mar 2020): "
      f"{peak_vol:.1f}% annualized")
```

The GARCH model estimated before COVID gave a long-run volatility around 14%. When the crash started, the conditional volatility did spike — but it took several days to ramp up. In the first few days of the sell-off, the model was still anchored to its calm-market estimate.

That's exactly what happened to our risk model! The first day of the crash, the VaR number looked fine. By the time the model caught up, we'd already lost a fortune. Why is GARCH so slow at the start?

Because GARCH is reactive, not proactive. It updates its variance estimate only after observing a large return. On the first day of a crash, r_{t-1}^2 suddenly jumps, and σ_t^2 starts rising through the α channel. But with $\beta \approx 0.90$, the previous day's (low) variance still dominates. It takes several consecutive large returns before the model fully "wakes up."

The Onset Problem

This is a fundamental limitation of GARCH-type models: they respond *after* a shock hits, not before. During the initial phase of a crisis:

- Day 1: Market drops 3%. GARCH volatility barely moves (still dominated by $\beta \times \sigma_{t-1}^2$)
- Day 2–3: Continued large drops. GARCH ramps up but is still below realized volatility
- Day 5+: GARCH catches up to the new regime

In contrast, the VIX (Week 2) often spikes *before* the worst returns because option traders price in fear proactively.

I see another issue in the data. When the market dropped sharply, the volatility spiked much more than when it rose by the same amount. But GARCH uses r_{t-1}^2 — the square wipes out the sign. It treats a -3% day and a $+3\%$ day identically.

You've just discovered the biggest weakness of the basic GARCH model. It's called the ***leverage effect****: bad news (negative returns) increases volatility more than good news (positive returns) of the same magnitude. Standard GARCH(1,1) is blind to this asymmetry because it squares the return, erasing the sign.*

That's a serious problem for a pension fund. We're much more worried about downside volatility than upside volatility. We need a model that is more sensitive to bad news.

That's exactly what we'll address next week with the GJR-GARCH model. But first, let me show you one more thing about today's model — because even with its limitations, GARCH(1,1) is a remarkably powerful starting point.

3.9 Model Diagnostics: Is GARCH Enough?

After fitting any model, we must check whether it adequately captures the data's structure. For GARCH, this means examining the **standardized residuals**:

Standardized Residuals

$$\hat{z}_t = \frac{r_t}{\hat{\sigma}_t} \tag{3.9}$$

If the GARCH model is correctly specified, the standardized residuals $\hat{z}_t$ should behave like i.i.d. draws — in particular, their squares $\hat{z}_t^2$ should show *no remaining autocorrelation*.

GARCH Diagnostic Checks

```python
# Standardized residuals from the full-sample model
std_resid = result.std_resid.dropna()

# 1. Ljung-Box test on squared standardized residuals
lb_filtered = acorr_ljungbox(std_resid ** 2, lags=10,
                             return_df=True)
print("Ljung-Box on squared standardized residuals:")
print(lb_filtered)

# 2. ACF of squared standardized residuals
```

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(returns.values ** 2, lags=30, ax=axes[0],
         title="ACF of Squared Returns (Raw)")
plot_acf(std_resid.values ** 2, lags=30, ax=axes[1],
         title="ACF of Squared Std Residuals (GARCH)")
plt.tight_layout()
plt.savefig("week03_diagnostics.png", dpi=150)
plt.show()

# 3. Summary statistics of standardized residuals
print(f"\nMean:     {std_resid.mean():.4f}")
print(f"Std Dev:  {std_resid.std():.4f}")
print(f"Skewness: {std_resid.skew():.4f}")
# Note: pandas kurtosis() returns excess kurtosis
# (Fisher's definition, i.e., kappa - 3).
# Normal distribution -> 0, not 3.
print(f"Excess Kurtosis: {std_resid.kurtosis():.4f}")
```

If the GARCH model has done its job, the Ljung–Box p-values on $\hat{z}_t^2$ should be large (fail to reject no autocorrelation) — meaning the model has absorbed the volatility clustering. Compare the two ACF plots: the raw squared returns show strong autocorrelation, but the squared standardized residuals should be much flatter.

The autocorrelation is mostly gone! But the skewness is negative — around -0.3. That's the asymmetry you mentioned, right?

Precisely. GARCH removes the volatility clustering, but it cannot remove the asymmetry because it doesn't model it. That negative skewness in the residuals is a fingerprint of the leverage effect, pointing directly to the need for next week's model.

3.10 AI Spotlight: Can AI Replace GARCH?

GARCH in the Age of Machine Learning

GARCH is a *parametric* model: it assumes a specific functional form for how volatility evolves. Machine learning models like neural networks are *nonparametric*: they learn the functional form from data. A natural question: can AI simply replace GARCH?

The empirical evidence is nuanced:

- For **in-sample fit**, neural networks (LSTM, GRU) often outperform GARCH because they can capture more complex patterns
- For **out-of-sample forecasting**, GARCH remains surprisingly competitive — especially at short horizons (1–5 days). Its parsimonious structure (3 parameters) acts as regularization against overfitting.
- The current frontier is **hybrid models**: use GARCH to capture the known structure (clustering, mean reversion), then let ML capture what GARCH misses (asymmetry, jumps, nonlinearities)
- GARCH's advantage: **interpretability**. A risk manager can explain GARCH parameters to a regulator. Explaining an LSTM's hidden states is far harder.

We will revisit this competition in Week 8.

I'll be honest — my LSTM does beat GARCH on some metrics. But when Kenji's regulator asks "why did your model predict this number?", I can't open the black box. GARCH gives you an answer: it's ω plus α times yesterday's shock plus β times yesterday's variance. Interpretability matters in finance.

3.11 Discussion: GARCH Assumptions and Reality

Does GARCH Match Reality?

GARCH(1,1) is powerful but makes simplifying assumptions. Discuss the following:

1. **Symmetric response**: GARCH treats positive and negative shocks identically (both are squared). Does this match your intuition about financial markets? Can you think of situations where a positive shock increases volatility?
2. **Single-regime**: GARCH assumes one set of parameters for all market conditions. Should α and β be different during a crisis versus a bull market? What are the trade-offs

of adding regime-switching?

3. **Normal vs. t-distribution**: Why might assuming a Student's t-distribution for residuals produce better risk estimates? What is the cost of this extra flexibility?
4. **Parameter stability**: We estimated GARCH on 2015–2024 data. Would the parameters be similar if we used 2000–2010 data? What does this mean for model reliability?

3.12 Weekly Mission

◎ Mission 3: Building the GARCH Engine

Objective: Build and validate a GARCH(1,1) model for Kenji's key market indices.

Deliverables:

1. Download daily data for the Nikkei 225 and S&P 500 from 2010–2024
2. For each index:
   - Plot daily returns and squared returns; describe the clustering patterns you observe
   - Compute and plot the ACF of squared returns (30 lags); report the Ljung–Box $Q(10)$ statistic and p-value
   - Fit a GARCH(1,1) model with Student's t-distribution
   - Report $\hat{\omega}$, $\hat{\alpha}$, $\hat{\beta}$, $\hat{\nu}$ and compute: long-run variance, persistence $(\alpha + \beta)$, and half-life
3. Diagnostic check: plot ACF of squared standardized residuals and confirm the Ljung–Box test no longer rejects
4. Plot the conditional volatility from GARCH alongside the EWMA volatility from Week 2. Discuss: where do they agree? Where do they diverge?
5. Write a one-paragraph comparison memo to Kenji: why should VolTech use GARCH instead of (or in addition to) EWMA?

🏆 Bonus Challenge: GARCH Order Selection

Fit GARCH(1,1), GARCH(1,2), GARCH(2,1), and GARCH(2,2) to the S&P 500 data. Compare them using:

- AIC (Akaike Information Criterion)
- BIC (Bayesian Information Criterion)
- Log-likelihood value

Does the more complex model always win? Why is GARCH(1,1) usually preferred in

practice?
Hint: AIC and BIC penalize model complexity. Access them via `result.aic` and `result.bic` in the `arch` library.

3.13 Key Takeaways

Key Takeaways

1. **Volatility clusters**: Large returns tend to follow large returns. This is detectable via the ACF of squared returns and the Ljung–Box test. It is the single most important stylized fact driving volatility models.
2. **ARCH** (Engle, 1982): The first formal model of time-varying volatility. Today's variance depends on past squared returns. Requires many lags for a good fit.
3. **GARCH(1,1)** (Bollerslev, 1986): Adds yesterday's variance to ARCH, dramatically reducing parameters. Three numbers — ω, α, β — capture volatility dynamics for most financial assets.
4. **Mean reversion**: GARCH's long-run variance $\bar{\sigma}^2 = \omega/(1-\alpha-\beta)$ acts as an anchor. After shocks, volatility gravitates back. This is what EWMA lacks ($\omega = 0 \Rightarrow$ no anchor).
5. **MLE**: Parameters are estimated by maximizing the probability of observing the data. Using a Student's t-distribution for residuals improves fit by accommodating fat tails.
6. **Limitation**: GARCH squares the return, erasing the sign. It cannot capture the **leverage effect** — the asymmetric impact of bad news versus good news on volatility. This motivates the GJR-GARCH model in Week 4.

3.14 Further Reading

Further Reading

1. Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. [The original ARCH paper]
2. Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327. [The original GARCH paper]
3. Engle, R. F. (2001). GARCH 101: the use of ARCH/GARCH models in applied econometrics. *Journal of Economic Perspectives*, 15(4), 157–168. [Accessible overview by the Nobel laureate]
4. Bollerslev, T., Chou, R. Y., & Kroner, K. F. (1992). ARCH modeling in finance: a review of the theory and empirical evidence. *Journal of Econometrics*, 52(1–2), 5–59.
5. Mandelbrot, B. (1963). The variation of certain speculative prices. *Journal of Business*, 36(4), 394–419. [Early observation of volatility clustering]
6. Engle, R. F., & Patton, A. J. (2001). What good is a volatility model? *Quantitative Finance*, 1(2), 237–245. [Practical evaluation criteria for volatility models]
7. Hansen, P. R., & Lunde, A. (2005). A forecast comparison of volatility models: does anything beat a GARCH(1,1)? *Journal of Applied Econometrics*, 20(7), 873–889.
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapters 3–4.
9. Francq, C., & Zakoïan, J.-M. (2019). *GARCH Models: Structure, Statistical Inference and Financial Applications*, 2nd ed. Wiley. [Comprehensive graduate reference]

Next Week Preview

That evening, Alex ran one more experiment. He plotted the GARCH residuals separately for up-days and down-days. The pattern was unmistakable: after a -3% day, the next day's volatility was significantly higher than after a $+3\%$ day. He messaged Dr. Lin: "GARCH assumes symmetry, but the data clearly doesn't. How do we fix this?" Dr. Lin replied with a single link to a 1993 paper by Glosten, Jagannathan, and Runkle. The title: On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks. Next week: we break the symmetry with ***GJR-GARCH*** *— and learn that in markets, bad news hits harder than good news.*

Chapter 4

Week 4: Asymmetric Volatility — GJR-GARCH

Learning Objectives

1. Explain the leverage effect and provide economic intuition for why negative returns increase volatility more than positive returns
2. Specify and interpret the GJR-GARCH model, including the role of the asymmetry parameter γ
3. Construct and interpret the News Impact Curve for GARCH and GJR-GARCH
4. Compare GARCH and EGARCH as alternative asymmetric specifications
5. Estimate GJR-GARCH in Python and evaluate whether it improves over standard GARCH using AIC, BIC, and likelihood ratio tests

4.1 This Week's Story: Bad News Hits Harder

Scene: VolTech Analytics — Monday, 9:15 AM

Alex had spent the weekend running experiments on his GARCH model. The residual plots from Thursday's analysis wouldn't leave his mind: after large negative returns, tomorrow's volatility spiked far more than after large positive returns of the same size. He had messaged Dr. Lin, who sent back a single link — a 1993 paper by Glosten, Jagannathan, and Runkle.

Dr. Lin, I read the GJR paper you sent. They show that stock returns and volatility are negatively correlated — when prices fall, volatility rises disproportionately. But why? What's the economic mechanism?

Good question. There are two leading explanations, and they may both be at work.

Two Theories of Asymmetric Volatility

1. **The Leverage Effect** (Black, 1976; Christie, 1982): When a firm's stock price falls, its debt-to-equity ratio rises automatically (the firm becomes more leveraged). Higher leverage means more financial risk, so equity holders demand a higher risk premium, which increases volatility. The name "leverage effect" comes from this balance-sheet channel.
2. **The Volatility Feedback Effect** (French, Schwert & Stambaugh, 1987; Campbell & Hentschel, 1992): If an increase in expected future volatility raises the required rate of return, the stock price must drop immediately to deliver that higher return. So the causation runs the other way: volatility up $\Rightarrow$ price down. This creates a negative correlation between returns and volatility changes.

Both channels predict the same observable pattern: negative returns are associated with larger increases in volatility than positive returns. In practice, the effect is strong and pervasive across equity markets worldwide.

This is exactly what happened to our portfolio in March 2020. The market dropped, and the next day's volatility was much higher than what our symmetric GARCH model predicted. We need a model that treats good news and bad news differently.

That's precisely what GJR-GARCH does. Let me show you how it works.

4.2 The GJR-GARCH Model

The **GJR-GARCH** model (Glosten, Jagannathan & Runkle, 1993) extends standard GARCH by adding a term that "turns on" only when the previous return is negative.

The GJR-GARCH(1,1) Model

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \gamma\, r_{t-1}^2\, \mathbb{I}(r_{t-1} < 0) + \beta\, \sigma_{t-1}^2 \tag{4.1}$$

where:

- $\omega > 0$: base variance (same as GARCH)
- $\alpha \geq 0$: response to all shocks (same as GARCH)
- γ: the **asymmetry coefficient** — the *additional* impact of negative shocks
- $\beta \geq 0$: persistence (same as GARCH)

- $\mathbb{I}(r_{t-1} < 0)$: an **indicator function** that equals 1 when $r_{t-1} < 0$ and 0 otherwise

The indicator function is the key innovation: it acts as a switch that activates γ only for bad news. When $\gamma > 0$, negative returns have a larger impact on future volatility than positive returns.

So the effective shock coefficient is α for positive returns and $\alpha + \gamma$ for negative returns?

Exactly. That asymmetry is the whole point. Let me show you the numbers.

Table 4.1: How GJR-GARCH responds to positive and negative shocks

Shock Type	**GARCH Impact**	**GJR-GARCH Impact**
Positive ($r_{t-1} > 0$)	$\alpha \cdot r_{t-1}^2$	$\alpha \cdot r_{t-1}^2$
Negative ($r_{t-1} < 0$)	$\alpha \cdot r_{t-1}^2$	$(\alpha + \gamma) \cdot r_{t-1}^2$
Asymmetry ratio	1.0	$\frac{\alpha+\gamma}{\alpha}$

Interpreting γ

For typical equity indices, γ is positive and statistically significant:

- If $\gamma = 0$: no asymmetry $\Rightarrow$ GJR-GARCH reduces to standard GARCH
- If $\gamma > 0$: negative shocks have a larger impact than positive shocks (the leverage effect)
- If $\gamma < 0$: positive shocks have a larger impact (rare for equities, but sometimes observed for commodities like oil)

A typical finding for equity indices: $\alpha \approx 0.02$–0.05 and $\gamma \approx 0.10$–0.15, meaning negative shocks have roughly 3–4 times the impact of positive shocks!

Beyond interpreting γ, we must also verify that the model is well-behaved statistically. The stationarity condition ensures that the variance process does not explode over time.

Stationarity Condition for GJR-GARCH

For GJR-GARCH(1,1) to be covariance stationary, we need:

$$\alpha + \frac{\gamma}{2} + \beta < 1 \tag{4.2}$$

The $\gamma/2$ term arises because, under a symmetric distribution, the indicator $\mathbb{I}(r_{t-1} < 0)$ is

active half the time on average. The long-run variance is:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \gamma/2 - \beta} \tag{4.3}$$

4.3 The News Impact Curve

The most revealing way to compare GARCH and GJR-GARCH is through the **News Impact Curve (NIC)** — a plot of tomorrow's conditional variance σ_t^2 as a function of today's return r_{t-1}, holding all other information constant.

News Impact Curve

The NIC shows σ_t^2 as a function of r_{t-1}, with σ_{t-1}^2 fixed at its unconditional value $\bar{\sigma}^2$:

$$\text{NIC}(r_{t-1}) = \omega + \alpha\, r_{t-1}^2 + \gamma\, r_{t-1}^2\, \mathbb{I}(r_{t-1} < 0) + \beta\, \bar{\sigma}^2 \tag{4.4}$$

Key visual properties:

- **GARCH**: A symmetric parabola — same curvature for positive and negative r_{t-1}
- **GJR-GARCH**: An asymmetric parabola — steeper on the left (negative returns) than on the right. The "kink" at $r_{t-1} = 0$ is the visual signature of asymmetry.

The News Impact Curve is one of the most important diagnostic tools in volatility modeling. When you present results to Kenji's regulators, this single chart shows exactly how the model responds to good and bad news.

Check Your Understanding

Before introducing an alternative model, make sure you can answer:

1. If $\alpha = 0.03$ and $\gamma = 0.12$, what is the asymmetry ratio? What does this number mean in plain English?
2. Why does the stationarity condition use $\gamma/2$ rather than γ?

Answers: (1) $(\alpha+\gamma)/\alpha = (0.03+0.12)/0.03 = 5.0$, meaning negative shocks have 5 times the impact of positive shocks. (2) Because under a symmetric distribution, the indicator $\mathbb{I}(r_{t-1} < 0)$ is active only half the time on average.

4.4 EGARCH: An Alternative Approach

Nelson (1991) proposed the **Exponential GARCH (EGARCH)** model, which takes a fundamentally different approach to capturing asymmetry.

The EGARCH(1,1) Model

$$\ln(\sigma_t^2) = \omega + \alpha\left(|z_{t-1}| - \mathbb{E}[|z_{t-1}|]\right) + \gamma\, z_{t-1} + \beta\, \ln(\sigma_{t-1}^2) \tag{4.5}$$

where $z_{t-1} = r_{t-1}/\sigma_{t-1}$ is the standardized residual.

Key differences from GJR-GARCH:

- Models $\ln(\sigma_t^2)$ instead of σ_t^2 — so σ_t^2 is always positive, no matter what the parameters are. No positivity constraints needed on α or β.
- Uses the *standardized* shock z_{t-1} instead of the raw return r_{t-1}. Under the Gaussian assumption, $\mathbb{E}[|z_{t-1}|] = \sqrt{2/\pi} \approx 0.798$. When using Student's t-distribution, this constant depends on the degrees of freedom ν and is computed automatically by the `arch` library.
- Asymmetry enters through $\gamma\, z_{t-1}$: when $\gamma < 0$ (the typical case), a negative z_{t-1} increases $\ln(\sigma_t^2)$ more than a positive one

Sign convention note: In GJR-GARCH, $\gamma > 0$ indicates leverage (bad news increases volatility). In EGARCH, $\gamma < 0$ indicates the same effect. Both mean the same thing economically — the sign difference is purely a matter of model specification.

Table 4.2: Comparing asymmetric GARCH specifications

Feature	GJR-GARCH	EGARCH
Variance equation	σ_t^2	$\ln(\sigma_t^2)$
Positivity guaranteed?	No (need constraints)	Yes (always)
Asymmetry mechanism	Indicator $\mathbb{I}(r_{t-1} < 0)$	Signed residual z_{t-1}
News Impact Curve	Piecewise quadratic	Exponential
Typical sign for asymmetry	$\gamma > 0$	$\gamma < 0$
Estimation complexity	Lower	Higher
Industry prevalence	Very common	Common

Both models capture asymmetry, but they do it differently. GJR-GARCH is more intuitive and easier to estimate. EGARCH is more flexible and never produces a negative variance. In practice, both fit equity data well. For VolTech's client report, I'd recommend GJR-

> *GARCH because it's easier to explain.*

A third variant worth knowing is the **Threshold GARCH (TGARCH)** by Zakoian (1994), which models σ_t (the standard deviation) rather than σ_t^2. While less commonly used than GJR-GARCH or EGARCH in current practice, it appears frequently in the older literature and shares the same core intuition: negative shocks receive extra weight.

4.5 Python Implementation

> *Let's estimate both models and compare them head-to-head. The `arch` library makes this straightforward.*

Step 1: Fitting GJR-GARCH and EGARCH

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# 1. Standard GARCH(1,1) for baseline
garch = arch_model(returns, vol="Garch", p=1, q=1,
                   dist="t")
res_garch = garch.fit(disp="off")

# 2. GJR-GARCH(1,1) --- 'o=1' adds one asymmetry term
gjr = arch_model(returns, vol="Garch", p=1, o=1, q=1,
                 dist="t")
res_gjr = gjr.fit(disp="off")

# 3. EGARCH(1,1)
egarch = arch_model(returns, vol="EGARCH", p=1, o=1,
```

```python
                    q=1, dist="t")
res_egarch = egarch.fit(disp="off")

# Compare model selection criteria
print("Model Comparison:")
print(f"{'Model':<14} {'AIC':>10} {'BIC':>10} "
      f"{'LogLik':>10}")
print("-" * 46)
for name, res in [("GARCH", res_garch),
                  ("GJR-GARCH", res_gjr),
                  ("EGARCH", res_egarch)]:
    print(f"{name:<14} {res.aic:>10.1f} {res.bic:>10.1f}"
          f" {res.loglikelihood:>10.1f}")
```

The GJR-GARCH and EGARCH both have lower AIC and BIC than standard GARCH. The asymmetry clearly matters.

Next, we examine the estimated GJR-GARCH parameters and quantify the asymmetry:

Step 2: Interpreting GJR-GARCH Parameters

```python
# GJR-GARCH parameters
print("\nGJR-GARCH(1,1) Parameters:")
print(res_gjr.summary().tables[1])

omega = res_gjr.params["omega"]
alpha = res_gjr.params["alpha[1]"]
gamma = res_gjr.params["gamma[1]"]
beta  = res_gjr.params["beta[1]"]

print(f"\nomega = {omega:.6f}")
print(f"alpha = {alpha:.4f}  (positive shock impact)")
print(f"gamma = {gamma:.4f}  (additional negative impact)")
print(f"beta  = {beta:.4f}  (persistence)")
print(f"\nEffective impact of positive shock: "
      f"alpha = {alpha:.4f}")
print(f"Effective impact of negative shock: "
      f"alpha + gamma = {alpha + gamma:.4f}")
```

```python
print(f"Asymmetry ratio: "
      f"{(alpha + gamma) / alpha:.1f}x")

# Stationarity check
persist = alpha + gamma / 2 + beta
longrun_var = omega / (1 - persist)
longrun_vol = np.sqrt(longrun_var * 252)
halflife = np.log(0.5) / np.log(persist)

print(f"\nPersistence: {persist:.4f}")
print(f"Long-run annualized vol: {longrun_vol:.2f}%")
print(f"Half-life: {halflife:.1f} days")
```

Notice the asymmetry ratio. If it's around 3–4, that means a negative return has three to four times the impact on tomorrow's volatility compared to a positive return of the same size. This is the leverage effect in action.

Now, the key visual comparison — the News Impact Curve:

Step 3: Plotting the News Impact Curve

```python
# News Impact Curve: GARCH vs. GJR-GARCH
shock_range = np.linspace(-5, 5, 500)  # shock in %

# GARCH NIC
omega_g = res_garch.params["omega"]
alpha_g = res_garch.params["alpha[1]"]
beta_g  = res_garch.params["beta[1]"]
lrv_g   = omega_g / (1 - alpha_g - beta_g)
nic_garch = omega_g + alpha_g * shock_range**2 \
            + beta_g * lrv_g

# GJR-GARCH NIC
lrv_gjr = omega / (1 - alpha - gamma/2 - beta)
nic_gjr = (omega + alpha * shock_range**2
           + gamma * shock_range**2
             * (shock_range < 0)  # Boolean: 1 if negative, 0 if positive
           + beta * lrv_gjr)
```

```python

# EGARCH NIC (approximate via exponential)
omega_e = res_egarch.params["omega"]
alpha_e = res_egarch.params["alpha[1]"]
gamma_e = res_egarch.params["gamma[1]"]
beta_e  = res_egarch.params["beta[1]"]
lrv_e   = omega_e / (1 - beta_e)
z_range = shock_range / np.sqrt(lrv_g)  # standardize
nic_egarch = np.exp(
    lrv_e + alpha_e * (np.abs(z_range) - np.sqrt(2/np.pi))
    + gamma_e * z_range)

# Plot
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(shock_range, nic_garch, color="#1B3A5C",
        linewidth=2, label="GARCH(1,1)")
ax.plot(shock_range, nic_gjr, color="#C0392B",
        linewidth=2, label="GJR-GARCH(1,1)")
ax.plot(shock_range, nic_egarch, color="#2D7D5E",
        linewidth=2, linestyle="--",
        label="EGARCH(1,1)")
ax.axvline(0, color="gray", linestyle="--",
           linewidth=0.8)
ax.set_xlabel("Return Shock (%)")
ax.set_ylabel(r"Conditional Variance $\sigma_t^2$")
ax.set_title("News Impact Curve: All Three Models")
ax.legend()
plt.tight_layout()
plt.savefig("week04_nic.png", dpi=150)
plt.show()
```

The asymmetry is striking. The GJR-GARCH curve is much steeper on the left side — a -3% shock produces a far higher variance than a $+3\%$ shock. Standard GARCH is perfectly symmetric and misses this entirely. And the EGARCH curve shows the same asymmetry but with a smooth exponential shape instead of a kink — exactly matching Table 4.2.

Finally, we compare the conditional volatility paths during the COVID crisis:

Step 4: Conditional Volatility Comparison During COVID

```python
# Extract conditional volatility (annualized)
vol_garch = (res_garch.conditional_volatility
             * np.sqrt(252))
vol_gjr   = (res_gjr.conditional_volatility
             * np.sqrt(252))

# Focus on COVID period
mask = (returns.index >= "2019-10-01") & \
       (returns.index <= "2020-09-30")

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(vol_garch[mask].index, vol_garch[mask],
        color="#1B3A5C", linewidth=1.5,
        label="GARCH(1,1)")
ax.plot(vol_gjr[mask].index, vol_gjr[mask],
        color="#C0392B", linewidth=1.5,
        label="GJR-GARCH(1,1)")
ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red",
           label="COVID-19 crash")
ax.set_title("Conditional Volatility: GARCH vs. "
             "GJR-GARCH (COVID Period)")
ax.set_ylabel("Annualized Volatility (%)")
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("week04_covid_comparison.png", dpi=150)
plt.show()

# Quantify the difference
diff = vol_gjr[mask] - vol_garch[mask]
print(f"Mean GJR-GARCH vol during crash: "
      f"{vol_gjr[mask].mean():.1f}%")
print(f"Mean GARCH vol during crash: "
      f"{vol_garch[mask].mean():.1f}%")
print(f"Average difference: {diff.mean():.1f} "
      f"percentage points")
print(f"Max difference: {diff.max():.1f} "
```

```
        f"percentage points")
```

4.6 Application: The Asymmetry Test

Scene: Conference Room — Thursday, 2:30 PM

Alex projected the News Impact Curve on the screen. The asymmetry was unmistakable. Kenji leaned forward.

The standard GARCH model we tested last week treats a 3% gain and a 3% loss identically. But look at the GJR curve — a 3% loss generates almost four times the impact on tomorrow's volatility. That matches our experience perfectly.

And the conditional volatility comparison during COVID confirms it. GJR-GARCH ramped up faster and peaked higher during the crash because every negative day received the extra γ kick. The standard GARCH was systematically too slow and too low.

*Good analysis. But let me add an important nuance. The **likelihood ratio test** can tell us whether the improvement is statistically significant.*

Likelihood Ratio Test: GARCH vs. GJR-GARCH

Since GARCH is nested within GJR-GARCH (set $\gamma = 0$), we can use the likelihood ratio test:

$$LR = 2\left[\mathcal{L}_{\text{GJR}} - \mathcal{L}_{\text{GARCH}}\right] \sim \chi^2(1) \tag{4.6}$$

If the LR statistic exceeds the $\chi^2(1)$ critical value (3.84 at 5% significance), we reject the null that $\gamma = 0$ and conclude that asymmetry is statistically significant.

Likelihood Ratio Test

```python
from scipy import stats

# Likelihood ratio test
lr_stat = 2 * (res_gjr.loglikelihood
               - res_garch.loglikelihood)
```

```
p_value = 1 - stats.chi2.cdf(lr_stat, df=1)
print(f"LR statistic: {lr_stat:.2f}")
print(f"p-value: {p_value:.6f}")
print(f"Reject H0 (gamma=0) at 5%: "
      f"{'Yes' if p_value < 0.05 else 'No'}")
```

As expected, the LR test strongly rejects the null — $\gamma = 0$ is clearly not supported by the data. Asymmetry is not a theoretical curiosity; it's a statistically significant feature of equity returns.

This is exactly the kind of evidence-based analysis that will differentiate VolTech from our competitors. Kenji, if your regulators ask "why GJR over standard GARCH?", the News Impact Curve and the LR test are your answer. But I have a bigger question: fitting the model well is one thing — can it actually predict tomorrow's risk?

4.7 Model Diagnostics

As we did in Week 3, we check whether GJR-GARCH adequately captures the data's structure by examining the standardized residuals.

GJR-GARCH Diagnostic Checks

```
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.graphics.tsaplots import plot_acf

# Standardized residuals
std_resid_gjr = res_gjr.std_resid.dropna()

# Ljung-Box test on squared standardized residuals
lb = acorr_ljungbox(std_resid_gjr ** 2, lags=10,
                    return_df=True)
print("Ljung-Box on squared std residuals (GJR):")
print(lb)

# Skewness comparison: GARCH vs. GJR-GARCH
std_resid_g = res_garch.std_resid.dropna()
```

```python
print(f"\nResidual skewness (GARCH):     "
      f"{std_resid_g.skew():.4f}")
print(f"Residual skewness (GJR-GARCH): "
      f"{std_resid_gjr.skew():.4f}")
# Note: pandas kurtosis() returns excess kurtosis
print(f"Excess kurtosis (GARCH):       "
      f"{std_resid_g.kurtosis():.4f}")
print(f"Excess kurtosis (GJR-GARCH):   "
      f"{std_resid_gjr.kurtosis():.4f}")
```

The Ljung–Box p-values are large, so GJR-GARCH removes the autocorrelation in squared residuals. And the skewness improved — it's closer to zero than the standard GARCH residuals. The model is capturing some of the asymmetry that GARCH left behind.

The skewness improvement is exactly what we expected. By giving negative returns extra weight through γ, GJR-GARCH can partially account for the asymmetric pattern in the data. However, the residual excess kurtosis is still positive, reminding us that even GJR-GARCH with a Student's t-distribution doesn't capture all the extreme behavior.
One more diagnostic tool worth mentioning: the ***Engle–Ng sign and size bias tests*** *(Engle & Ng, 1993) can formally test whether asymmetry is present in the residuals before choosing a model. These tests regress the squared standardized residuals on indicator variables for the sign and magnitude of past shocks. If the sign bias test rejects, it confirms that an asymmetric specification like GJR-GARCH is warranted.*

4.8 AI Spotlight: Asymmetry in Machine Learning

Asymmetry: Parametric vs. Nonparametric

GARCH and GJR-GARCH model asymmetry through *explicit* design choices: the indicator function $\mathbb{I}(r_{t-1} < 0)$ or the exponential specification. Machine learning offers a different path: let the model *discover* asymmetry from data.

- **Neural networks** (LSTM, GRU): Can learn asymmetric responses implicitly through their nonlinear activation functions. No need to pre-specify the functional form. However, the learned asymmetry is hard to interpret.
- **Quantile regression**: Models different quantiles of the return distribution separately,

naturally capturing asymmetric tails without assuming a specific volatility process.

- **Hybrid approach**: Use GJR-GARCH as a feature generator for ML models. The GJR conditional volatility, asymmetry indicator, and News Impact Curve values become inputs to a neural network that can learn residual patterns.

The key insight: GJR-GARCH provides *interpretable* asymmetry with a single parameter γ. ML provides *flexible* asymmetry at the cost of interpretability. The optimal approach often combines both.

4.9 Discussion: Asymmetry in Practice

When Does Asymmetry Matter Most?

1. **Asset class differences**: The leverage effect is strongest for equities. What about currencies, commodities, or bonds? Would you expect γ to be positive, negative, or near zero for oil prices? Why?
2. **Crisis amplification**: During the 2020 COVID crash, the leverage effect amplified the volatility spike. Could this create a feedback loop where rising volatility causes further selling, which increases volatility even more?
3. **Practical impact**: If you compute Value-at-Risk using standard GARCH instead of GJR-GARCH, how would the VaR estimates differ after a market decline? Which model would give more conservative (safer) risk estimates?
4. **Model selection**: You have GARCH, GJR-GARCH, and EGARCH. All fit well. Which would you recommend to a risk manager who needs to explain the model to regulators? What if the regulator asks "why this model?"

4.10 Weekly Mission

◎ Mission 4: The Asymmetry Report

Objective: Build a comprehensive asymmetric volatility analysis for Kenji's portfolio indices.

Deliverables:

1. For the Nikkei 225 and S&P 500 (2010–2024):
   - Fit GARCH(1,1), GJR-GARCH(1,1), and EGARCH(1,1), all with Student's t-distribution

- Report parameters, AIC, BIC, and log-likelihood for each
- Conduct likelihood ratio tests: GARCH vs. GJR-GARCH

2. For the GJR-GARCH model:
   - Report the asymmetry ratio $(\alpha + \gamma)/\alpha$
   - Compute long-run volatility and half-life using the GJR-GARCH stationarity formula (4.2)
3. Plot the News Impact Curve for all three models on the same chart for each index
4. Plot conditional volatility comparison (GARCH vs. GJR-GARCH) for the COVID period (Oct 2019 – Sep 2020)
5. Write a one-page memo to Kenji recommending which volatility model VolTech should use going forward, supported by the evidence from your analysis

Bonus Challenge: Cross-Market Asymmetry

Fit GJR-GARCH(1,1) to at least five different indices or assets:

- S&P 500, Nikkei 225, FTSE 100 (equities)
- Gold (commodity), USD/JPY (currency)

Compare the estimated γ across asset classes. Where is asymmetry strongest? Where is it weakest or reversed? Write a brief summary explaining the economic intuition for each.

Hint: For gold and FX, think about whether the leverage effect logic (debt-to-equity ratio) applies.

4.11 Key Takeaways

Key Takeaways

1. **Leverage effect**: Negative returns increase future volatility more than positive returns of the same magnitude. Explanations include the balance-sheet leverage channel (Black, 1976) and the volatility feedback effect (Campbell & Hentschel, 1992).
2. **GJR-GARCH**: Extends GARCH with an indicator function $\mathbb{I}(r_{t-1} < 0)$ that gives negative shocks extra weight. One additional parameter γ captures the asymmetry.
3. **News Impact Curve**: The key diagnostic tool for visualizing how a model responds to shocks. GARCH gives a symmetric parabola; GJR-GARCH gives an asymmetric one with a kink at zero.
4. **EGARCH**: An alternative that models $\ln(\sigma_t^2)$, guaranteeing positivity. Uses signed standardized residuals for asymmetry. Equally valid but harder to interpret.
5. **Likelihood ratio test**: Since GARCH is nested within GJR-GARCH ($\gamma = 0$), we can formally test whether asymmetry is statistically significant. For equities, it almost always is.
6. **Practical impact**: Ignoring asymmetry means underestimating risk after market declines — precisely when accurate risk measurement matters most.

4.12 Further Reading

Further Reading

1. Glosten, L. R., Jagannathan, R., & Runkle, D. E. (1993). On the relation between the expected value and the volatility of the nominal excess return on stocks. *Journal of Finance*, 48(5), 1779–1801. [The original GJR-GARCH paper]
2. Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: a new approach. *Econometrica*, 59(2), 347–370. [The original EGARCH paper]
3. Black, F. (1976). Studies of stock price volatility changes. *Proceedings of the American Statistical Association, Business and Economics Section*, 177–181. [Origin of the leverage effect]
4. Christie, A. A. (1982). The stochastic behavior of common stock variances: value, leverage, and interest rate effects. *Journal of Financial Economics*, 10(4), 407–432.
5. French, K. R., Schwert, G. W., & Stambaugh, R. F. (1987). Expected stock returns and volatility. *Journal of Financial Economics*, 19(1), 3–29. [Early evidence on the return–volatility relation]
6. Campbell, J. Y., & Hentschel, L. (1992). No news is good news: an asymmetric model of changing volatility in stock returns. *Journal of Financial Economics*, 31(3), 281–318. [Volatility feedback effect]
7. Engle, R. F., & Ng, V. K. (1993). Measuring and testing the impact of news on volatility. *Journal of Finance*, 48(5), 1749–1778. [News Impact Curve methodology]
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 3.
9. Francq, C., & Zakoïan, J.-M. (2019). *GARCH Models*, 2nd ed. Wiley. Chapter 4.
10. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. [Practitioner perspective on asymmetric volatility in risk management]
11. Zakoian, J.-M. (1994). Threshold heteroskedastic models. *Journal of Economic Dynamics and Control*, 18(5), 931–955. [The original TGARCH paper]

Next Week Preview

After the meeting, David pulled Alex aside. "Your GJR model clearly fits better than GARCH. The News Impact Curve is a great visual. But I need to ask the hard question: can this model predict tomorrow's volatility? Not just fit yesterday's data?" Before Alex could answer, Priya walked over with her laptop. "I've been running out-of-sample forecasting tests. GARCH, GJR-GARCH, EGARCH, and my LSTM — all in a head-to-head horse race. The results are… interesting." She grinned. "Want to see?" Next week: we enter the world of ***volatility forecasting*** *— where models are judged not by how well they fit the past, but by how well they predict the future.*

Chapter 5

Week 5: Volatility Forecasting

Learning Objectives

1. Distinguish between in-sample fit and out-of-sample forecasting and explain why the latter is the true test of a model
2. Implement a rolling-window one-step-ahead forecasting procedure for GARCH and GJR-GARCH models
3. Evaluate volatility forecasts using RMSE, MAE, and QLIKE loss functions
4. Conduct a Mincer–Zarnowitz regression to test forecast efficiency
5. Compare multiple volatility models in a structured forecasting horse race

5.1 This Week's Story: The Horse Race

Scene: VolTech Analytics — Monday, 9:00 AM

Alex arrived to find the main conference room already occupied. Priya had covered the whiteboard with model names, loss functions, and rolling-window diagrams. Dr. Lin sat in the corner, sipping tea with a knowing smile.

Welcome to the Forecasting Arena, Alex. Last week you showed that GJR-GARCH fits the data better than GARCH. But David asked the right question: can it predict tomorrow's volatility? That's what we're testing today — out-of-sample.

What's the difference between fitting well and predicting well?

A model that fits the past perfectly might be memorizing noise rather than learning patterns. This is the classic ***overfitting*** *problem. The only honest test is to estimate the model on past data, then forecast into a period the model has never seen.*

Exactly. And I've prepared the ultimate test: GARCH, GJR-GARCH, EGARCH, and my LSTM — all forecasting the same out-of-sample period. Same data, same evaluation metrics. No excuses.

A horse race. I'm in.

5.2 The Forecasting Framework

Before running any forecasts, we need to set up the evaluation framework rigorously.

Out-of-Sample Forecasting Design

The data is split into two non-overlapping periods:

- **In-sample** (estimation window): Used to estimate model parameters. The model learns patterns from this period.
- **Out-of-sample** (evaluation window): Used to evaluate forecasts. The model has never seen this data during estimation.

For volatility forecasting, we typically use a **rolling window** approach:

1. Fix a window of W observations for estimation
2. Estimate the model on observations $1, 2, \ldots, W$
3. Forecast $\hat{\sigma}^2_{W+1}$ (one step ahead)
4. Roll the window forward: estimate on $2, 3, \ldots, W+1$
5. Forecast $\hat{\sigma}^2_{W+2}$
6. Repeat until the end of the data

An alternative is the **expanding window**, which keeps the start date fixed and grows the estimation sample at each step. Rolling windows are preferred when you suspect structural breaks (parameter changes over time).

The Volatility Proxy Problem

A fundamental challenge in volatility forecasting: we never observe "true" volatility σ^2_t. We only observe returns r_t. To evaluate forecasts, we need a **volatility proxy** — an estimate of what the true volatility was. Common choices:

- **Squared return** r_t^2: Simple but extremely noisy. One observation per day is a very imprecise estimate of variance.

- **Realized variance** RV_t: Sum of squared intraday returns (Week 2). Much more precise when intraday data is available.
- **Range-based estimators**: Use high, low, open, and close prices. More precise than r_t^2 but less precise than RV_t.

The choice of proxy matters: using r_t^2 as the target introduces substantial noise, which can distort model rankings. When possible, use RV_t or range-based proxies.

This is a subtle but critical point. If we evaluate forecasts against r_t^2, every model looks bad because r_t^2 is so noisy. The rankings of models are generally preserved, but the apparent accuracy is much lower. Patton (2011) showed that certain loss functions — including QLIKE — produce consistent model rankings regardless of which proxy you use.

5.3 Forecast Evaluation Metrics

How do we measure how "close" a volatility forecast $\hat{\sigma}_t^2$ is to the realized proxy σ_t^{2*}?

Loss Functions for Volatility Forecasting

For T out-of-sample forecasts:

1. Mean Squared Error (MSE) / Root MSE:

$$\text{MSE} = \frac{1}{T}\sum_{t=1}^{T}(\sigma_t^{2*} - \hat{\sigma}_t^2)^2, \quad \text{RMSE} = \sqrt{\text{MSE}} \tag{5.1}$$

2. Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{T}\sum_{t=1}^{T}|\sigma_t^{2*} - \hat{\sigma}_t^2| \tag{5.2}$$

3. QLIKE (Quasi-Likelihood Loss):

$$\text{QLIKE} = \frac{1}{T}\sum_{t=1}^{T}\left(\ln(\hat{\sigma}_t^2) + \frac{\sigma_t^{2*}}{\hat{\sigma}_t^2}\right) \tag{5.3}$$

Why Multiple Loss Functions?

Different loss functions penalize different types of forecast errors:

- **MSE**: Penalizes large errors heavily (squared). Good for detecting models that occasion-

ally produce terrible forecasts.

- **MAE**: Treats all errors proportionally. More robust to outliers than MSE.
- **QLIKE**: Penalizes *relative* errors (over-prediction and under-prediction asymmetrically). Particularly useful for volatility because it is scale-independent and robust to the choice of volatility proxy (Patton, 2011).

A model that wins on all three loss functions is clearly superior. When rankings differ, QLIKE is generally preferred by practitioners.

5.4 The Mincer–Zarnowitz Regression

Beyond loss functions, we can test whether forecasts are **efficient** — do they contain all available information?

Mincer–Zarnowitz Regression

Regress the realized proxy on the forecast:

$$\sigma_t^{2*} = a + b\,\hat{\sigma}_t^2 + u_t \tag{5.4}$$

A perfect forecast implies $a = 0$ and $b = 1$. Test the joint null $H_0 : a = 0,\ b = 1$ using an F-test. If rejected, the forecast is **biased** or **inefficient**:

- $b < 1$: Forecasts are too variable (overreact to shocks)
- $b > 1$: Forecasts are too smooth (underreact to shocks)
- $a \neq 0$: Forecasts have a systematic bias
- R^2: Proportion of realized variance explained by the forecast

So the R^2 of the Mincer–Zarnowitz regression tells us how much of the variation in actual volatility our model captures?

Precisely. When using r_t^2 as the proxy, R^2 values of 5–15% are typical — because r_t^2 is so noisy. When using realized variance, R^2 values of 30–50% are common. Don't be discouraged by low R^2 against r_t^2; what matters is the relative ranking of models.

5.5 Python Implementation: The Forecasting Pipeline

Time to build the full forecasting pipeline. We'll do rolling-window one-step-ahead forecasts for three GARCH-family models.

Step 1: Data Preparation and Forecast Setup

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
import warnings
warnings.filterwarnings("ignore")

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Forecast setup
window = 1000          # rolling estimation window
n_total = len(returns)
n_oos = n_total - window  # out-of-sample observations
print(f"Total obs: {n_total}")
print(f"Window: {window}")
print(f"Out-of-sample: {n_oos}")
```

Next, we implement the rolling forecast loop. This is the computational core — for each day in the out-of-sample period, we re-estimate the model and produce a one-step-ahead forecast:

Step 2: Rolling One-Step-Ahead Forecasts

```python
# Models to compare
model_specs = {
    "GARCH":     {"vol": "Garch", "p": 1, "o": 0,
                 "q": 1},
```

```python
    "GJR-GARCH": {"vol": "Garch", "p": 1, "o": 1,
                  "q": 1},
    "EGARCH":    {"vol": "EGARCH", "p": 1, "o": 1,
                  "q": 1},
}

# Store forecasts
forecasts = {}
for name, spec in model_specs.items():
    fc = np.full(n_oos, np.nan)
    # Note: this takes ~3-5 min for 3 models
    for i in range(n_oos):
        train = returns.iloc[i:i + window]
        model = arch_model(train, dist="t", **spec)
        try:
            res = model.fit(disp="off",
                            show_warning=False)
            fv = res.forecast(horizon=1)
            fc[i] = fv.variance.values[-1, 0]
        except Exception:
            fc[i] = np.nan
    forecasts[name] = pd.Series(
        fc, index=returns.index[window:])
    print(f"{name}: done ({np.isnan(fc).sum()} "
          f"failures)")

# Realized proxy: squared returns
realized = returns.iloc[window:] ** 2
```

The loop re-estimates the model every day — that sounds slow.

It takes a few minutes, yes. But this is the honest approach. Each forecast uses only information available at the time of the forecast — no future data leakage. For production systems, we'd optimize with parallel processing, but for research this is the gold standard.

Now we compute the evaluation metrics:

Step 3: Forecast Evaluation

```python
def evaluate_forecasts(realized, forecasts):
    """Compute RMSE, MAE, QLIKE for each model."""
    results = {}
    for name, fc in forecasts.items():
        valid = ~(fc.isna() | realized.isna())
        r = realized[valid].values
        f = fc[valid].values
        # Avoid log(0) or division by zero
        f = np.maximum(f, 1e-10)

        mse  = np.mean((r - f) ** 2)
        rmse = np.sqrt(mse)
        mae  = np.mean(np.abs(r - f))
        qlike = np.mean(np.log(f) + r / f)

        results[name] = {"RMSE": rmse, "MAE": mae,
                         "QLIKE": qlike}
    return pd.DataFrame(results).T

metrics = evaluate_forecasts(realized, forecasts)
print("\nForecast Evaluation (proxy = r^2):")
print(metrics.round(4))
print(f"\nBest model by RMSE:  "
      f"{metrics['RMSE'].idxmin()}")
print(f"Best model by MAE:   "
      f"{metrics['MAE'].idxmin()}")
print(f"Best model by QLIKE: "
      f"{metrics['QLIKE'].idxmin()}")
```

Step 4: Mincer–Zarnowitz Regression

```python
import statsmodels.api as sm

print("\nMincer-Zarnowitz Regressions:")
print("-" * 50)
for name, fc in forecasts.items():
    valid = ~(fc.isna() | realized.isna())
```

```
    r = realized[valid].values
    f = fc[valid].values
    X = sm.add_constant(f)
    ols = sm.OLS(r, X).fit()
    a, b = ols.params
    r2 = ols.rsquared
    # F-test for H0: a=0, b=1
    f_test = ols.f_test("(const = 0), (x1 = 1)")
    p_val = float(f_test.pvalue)
    print(f"{name:<14} a={a:>8.4f}  b={b:>6.4f}  "
          f"R2={r2:.4f}  F-pval={p_val:.4f}")
```

The R^2 values are quite low — around 5–10%.

That's expected when using r_t^2 as the proxy. Remember, a single squared return is an incredibly noisy estimate of daily variance. The rankings across models are what matter, not the absolute R^2.

Finally, we visualize the forecasts against realized volatility:

Step 5: Forecast Visualization

```
# Plot forecasts vs. realized (annualized vol)
fig, ax = plt.subplots(figsize=(14, 6))
rv_ann = np.sqrt(realized * 252)  # annualized
ax.plot(rv_ann.index, rv_ann, color="gray",
        linewidth=0.3, alpha=0.5,
        label="Realized proxy (ann.)")

colors = {"GARCH": "#1B3A5C",
          "GJR-GARCH": "#C0392B",
          "EGARCH": "#2D7D5E"}
for name, fc in forecasts.items():
    fc_vol = np.sqrt(fc * 252)  # annualized
    ax.plot(fc_vol.index, fc_vol, color=colors[name],
            linewidth=1.2, alpha=0.8, label=name)

ax.axvspan("2020-02-20", "2020-04-01",
```

```
            alpha=0.1, color="red",
            label="COVID-19 crash")
ax.set_title("Out-of-Sample Volatility Forecasts: "
             "S&P 500")
ax.set_ylabel("Annualized Volatility (%)")
ax.legend(loc="upper right")
ax.set_ylim(0, 120)
plt.tight_layout()
plt.savefig("week05_forecast_comparison.png", dpi=150)
plt.show()
```

The Diebold–Mariano Test

Comparing loss function values tells us *which* model has lower average loss, but not whether the difference is **statistically significant**. The **Diebold–Mariano (DM) test** (1995) formalizes this comparison. Define the loss differential:

$$d_t = L(\hat{\sigma}^2_{1,t}) - L(\hat{\sigma}^2_{2,t})$$

where $L(\cdot)$ is any loss function (e.g., squared error). Under H_0: the two models have equal predictive accuracy ($E[d_t] = 0$). The DM statistic uses HAC standard errors to account for serial correlation in the loss differentials:

$$DM = \frac{\bar{d}}{\hat{\sigma}_{\bar{d}}} \xrightarrow{d} N(0,1)$$

A significant DM statistic (e.g., $|DM| > 1.96$) means one model *significantly* outperforms the other, not just by chance.

Step 6: Diebold–Mariano Test

```
from scipy import stats

def dm_test(realized, fc1, fc2, loss="qlike"):
    """Diebold-Mariano test for equal predictive
    accuracy. Returns DM statistic and p-value."""
    valid = ~(fc1.isna() | fc2.isna() | realized.isna())
    r = realized[valid].values
    f1, f2 = fc1[valid].values, fc2[valid].values
```

```
    f1, f2 = np.maximum(f1, 1e-10), np.maximum(f2, 1e-10)

    if loss == "qlike":
        L1 = np.log(f1) + r / f1
        L2 = np.log(f2) + r / f2
    else:  # squared error
        L1 = (r - f1) ** 2
        L2 = (r - f2) ** 2

    d = L1 - L2  # loss differential
    T = len(d)
    d_bar = d.mean()
    # Newey-West HAC variance (lag = int(T^(1/3)))
    lag = int(T ** (1 / 3))
    gamma_0 = np.var(d, ddof=1)
    gamma_sum = sum(
        (1 - k / (lag + 1)) * np.cov(d[k:], d[:-k])[0, 1]
        for k in range(1, lag + 1))
    var_d = (gamma_0 + 2 * gamma_sum) / T
    dm_stat = d_bar / np.sqrt(max(var_d, 1e-20))
    p_val = 2 * (1 - stats.norm.cdf(abs(dm_stat)))
    return dm_stat, p_val

# Compare GJR-GARCH vs. GARCH
dm, pv = dm_test(realized, forecasts["GARCH"],
                 forecasts["GJR-GARCH"], "qlike")
print(f"DM test (GARCH vs GJR-GARCH, QLIKE):")
print(f"  DM = {dm:.3f}, p-value = {pv:.4f}")
dm2, pv2 = dm_test(realized, forecasts["GARCH"],
                   forecasts["EGARCH"], "qlike")
print(f"DM test (GARCH vs EGARCH, QLIKE):")
print(f"  DM = {dm2:.3f}, p-value = {pv2:.4f}")
```

The DM test confirms what the loss function table suggests: the improvement from GJR-GARCH over GARCH is statistically significant. This means it's not just noise — the asymmetry parameter captures genuinely useful predictive information.

5.6 Application: Who Wins the Horse Race?

Scene: Conference Room — Thursday, 3:00 PM

The full VolTech team gathered around the screen as Alex presented the forecasting results. Even David was there — he rarely attended technical meetings.

Here are the results. I ran rolling one-step-ahead forecasts for three models over the 2019–2024 out-of-sample period:

Table 5.1: Volatility forecasting horse race results (S&P 500)*

Model	RMSE	MAE	QLIKE
GARCH(1,1)	5.82	2.14	3.72
GJR-GARCH(1,1)	5.61	2.03	3.65
EGARCH(1,1)	5.65	2.10	3.66

*Representative values for S&P 500 daily data (2019–2024 out-of-sample). Your results may differ slightly depending on the exact date range and random seed. Lower values indicate better forecasts.

Key findings:

1. ***GJR-GARCH consistently outperforms standard GARCH*** *across all three loss functions. The asymmetry parameter γ isn't just a better in-sample fit — it genuinely improves out-of-sample predictions.*
2. ***EGARCH performs similarly to GJR-GARCH****. Both asymmetric models capture information that symmetric GARCH misses.*
3. ***The improvements are modest but consistent****. Don't expect huge gains; volatility is inherently hard to predict. But even small improvements compound over time in risk management.*
4. ***All models struggle during crisis onset****. The first few days of the COVID crash were poorly forecast by every model.*

This result aligns with Hansen & Lunde (2005), who tested 330 volatility models and found that few could consistently beat a well-specified GARCH(1,1). The key word is "well-specified" — adding asymmetry and using the right distribution usually helps, but exotic model complexity rarely does.

For the record, my LSTM did beat GJR-GARCH on RMSE by about 2% — but it was worse on QLIKE. And it took 100 times longer to train. The honest answer: at one-day horizons, GARCH-family models are remarkably competitive with deep learning. The AI advantage shows up more at longer horizons and with richer feature sets.

These are useful results. But I need to ask: we've spent four weeks modeling volatility and predicting it. When do we translate this into something my board can understand? They don't care about QLIKE or R^2. They want one number: how much can we lose?

*That number is called **Value-at-Risk** — and it's exactly what we'll build next week.*

5.7 AI Spotlight: The Forecasting Frontier

Machine Learning for Volatility Forecasting

The traditional GARCH vs. ML debate has matured considerably:

- **Short horizon (1–5 days)**: GARCH-family models remain competitive. Their parsimonious structure prevents overfitting, and the one-day-ahead forecast is dominated by yesterday's variance (the β term), which ML can't easily improve upon.
- **Medium horizon (1–4 weeks)**: ML models begin to show advantages, particularly when incorporating additional features (option-implied volatility, news sentiment, trading volume).
- **Long horizon (1+ months)**: ML and hybrid models tend to outperform because they can capture regime changes and nonlinear patterns that GARCH's fixed parameters miss.
- **Hybrid models**: The most promising approach combines GARCH's structural knowledge with ML's flexibility. For example: use GJR-GARCH conditional volatility as a feature input to an LSTM, along with VIX, trading volume, and sentiment indicators.

The take-home message: GARCH is not obsolete — it's a strong baseline that any AI model must beat to justify its complexity.

5.8 Discussion: Forecasting Philosophy

What Makes a Good Forecast?

1. **Accuracy vs. interpretability**: A black-box model that produces slightly better forecasts vs. a simple model that you can explain to regulators. Which would you choose for a pension fund? For a hedge fund?
2. **The proxy problem**: We evaluated against r_t^2, which is very noisy. If you had access to 5-minute intraday data, how would you use it? Would the model rankings change?
3. **Structural breaks**: We used a rolling window of fixed length. What happens if market structure changes (e.g., the rise of algorithmic trading)? Should the window be shorter or longer?
4. **Economic value**: Better statistical forecasts don't always translate to better economic outcomes (portfolio returns, risk management). How would you test whether GJR-GARCH actually saves money compared to standard GARCH?

5.9 Weekly Mission

Mission 5: The Forecasting Horse Race

Objective: Build a complete volatility forecasting pipeline and conduct a multi-model horse race.

Deliverables:

1. For the Nikkei 225 and S&P 500:
   - Implement rolling-window one-step-ahead forecasts for GARCH(1,1) and GJR-GARCH(1,1) with Student's t-distribution
   - Use a rolling window of 1000 trading days
   - Compute RMSE, MAE, and QLIKE for each model
2. Run Mincer–Zarnowitz regressions for each model. Report $\hat{a}$, $\hat{b}$, R^2, and the F-test p-value
3. Create a forecast comparison plot (as in Step 5) for each index
4. Produce a summary table ranking the models by each loss function
5. Write a 200-word report to Kenji: which volatility model should VolTech deploy for daily risk monitoring, and why?

Bonus Challenge: The Expanding Window Test

Repeat the forecasting exercise using an expanding window (start fixed at Jan 2015, end grows daily) instead of the rolling window. Compare the results:

- Do the model rankings change?
- Which approach gives lower QLIKE?
- What are the pros and cons of each approach?

Hint: The expanding window uses all available history, so parameter estimates are more stable. But if the data-generating process changes over time, older data may hurt rather than help.

5.10 Key Takeaways

Key Takeaways

1. **Out-of-sample is the true test**: In-sample fit can be misleading due to overfitting. Always evaluate models on data they have never seen during estimation.
2. **Rolling window forecasting**: Re-estimate the model daily on a fixed-length window and forecast one step ahead. This is the gold standard for volatility forecast evaluation.
3. **Loss functions**: RMSE, MAE, and QLIKE measure different aspects of forecast quality. QLIKE is particularly valuable because it produces consistent model rankings regardless of the volatility proxy used.
4. **Mincer–Zarnowitz regression**: Tests whether forecasts are efficient ($a = 0$, $b = 1$). The R^2 indicates how much of the variation in realized volatility the model explains.
5. **GJR-GARCH beats GARCH for equity indices**: Asymmetric models consistently improve out-of-sample forecasts in most empirical studies of equity markets, confirming that the leverage effect contains genuine predictive information.
6. **GARCH remains competitive**: Despite the hype around AI, GARCH-family models are remarkably hard to beat at short forecast horizons. They should be the baseline in any forecasting comparison.

5.11 Further Reading

Further Reading

1. Hansen, P. R., & Lunde, A. (2005). A forecast comparison of volatility models: does anything beat a GARCH(1,1)? *Journal of Applied Econometrics*, 20(7), 873–889. [The benchmark volatility horse race]
2. Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256. [Why QLIKE is robust to proxy choice]
3. Mincer, J. A., & Zarnowitz, V. (1969). The evaluation of economic forecasts. In *Economic Forecasts and Expectations* (pp. 3–46). NBER. [The original M–Z regression]
4. Engle, R. F., & Patton, A. J. (2001). What good is a volatility model? *Quantitative Finance*, 1(2), 237–245.
5. Andersen, T. G., & Bollerslev, T. (1998). Answering the skeptics: yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4), 885–905.
6. Poon, S.-H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: a review. *Journal of Economic Literature*, 41(2), 478–539.
7. Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263. [The Diebold–Mariano test for comparing forecasts]
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 3.
9. West, K. D. (1996). Asymptotic inference about predictive ability. *Econometrica*, 64(5), 1067–1084. [Theoretical foundation for forecast evaluation inference]

Next Week Preview

As the team filed out, Kenji stayed behind. He looked at Alex with an expression that was equal parts impressed and impatient. "Your models predict volatility well enough. GJR-GARCH is the clear winner. But my board doesn't think in terms of conditional variance. They think in yen. They want to know: what is the most we can lose tomorrow with 99% confidence?" He paused. "Can your models answer that?" Alex glanced at Dr. Lin, who nodded slowly. "That's called Value-at-Risk," she said. "And it's where everything we've built so far comes together." Next week: we enter the world of ***Value-at-Risk*** *— the number every bank calculates every single day.*

Chapter 6

Week 6: Value-at-Risk

Learning Objectives

1. Define Value-at-Risk (VaR) and interpret it in terms of confidence level, holding period, and portfolio value
2. Compute VaR using three methods: Historical Simulation, Parametric (Variance-Covariance), and Monte Carlo Simulation
3. Explain the limitations of VaR and why Conditional Value-at-Risk (CVaR / Expected Shortfall) addresses the key weakness
4. Implement all three VaR methods in Python using GARCH-based volatility forecasts
5. Compare VaR estimates across methods during both calm and crisis periods

6.1 This Week's Story: One Number to Rule Them All

Scene: VolTech Analytics — Monday, 8:30 AM

Alex arrived early to find a thick document on his desk. The cover read: **Japan Financial Services Agency — Risk Management Guidelines for Institutional Investors**. A sticky note from Kenji read: "Page 47. Read it. This is what my board wants."

Page 47 says: "The institution shall report its daily Value-at-Risk at the 99% confidence level for all major asset classes." What exactly is Value-at-Risk?

Value-at-Risk is the single most important number in risk management. It answers one question: what is the maximum loss that we expect over a given holding period, with a given confidence level? Every major bank, pension fund, and insurance company in the world calculates it daily. Basel III requires it. The Japanese FSA requires it. And now Kenji's board requires it from us.

Specifically, my board wants this: "With 99% confidence, the most our portfolio can lose in one trading day is X billion yen." They want a single number they can report to the FSA and discuss in board meetings. Not conditional variance, not GARCH parameters — just one number.

And here's the connection to everything we've built so far: the quality of VaR depends entirely on the quality of the volatility model behind it. A better volatility forecast means a better VaR estimate. All five weeks of work lead to this moment.

6.2 Understanding Value-at-Risk

Before diving into the mathematics, it helps to see VaR as answering a very specific question: given a holding period and a confidence level, what is the worst "normal" loss we should prepare for?

Value-at-Risk Definition

The VaR_α at confidence level α (e.g., 99%) is defined as:

$$P(L_t > \text{VaR}_\alpha) = 1 - \alpha \tag{6.1}$$

Equivalently, for a portfolio with value V and return r_t:

$$\text{VaR}_\alpha = -V \cdot q_{1-\alpha}(r_t) \tag{6.2}$$

where $q_{1-\alpha}(r_t)$ is the $(1-\alpha)$-quantile of the return distribution (e.g., the 1st percentile for $\alpha = 0.99$). This is a negative number, so the negative sign converts the VaR to a positive loss amount.

Three Key Ingredients of VaR

Every VaR number requires three inputs:

- **Confidence level** (α): Typically 95% or 99%. Higher confidence means a larger (more conservative) VaR. Basel III uses 99%; internal risk management often uses 95%.
- **Holding period**: How far ahead? Usually 1 day for trading desks, 10 days for regulatory capital (Basel). Longer holding periods produce larger VaR.
- **Return distribution**: This is where the modeling work happens — and where different

VaR methods diverge.

So if the 1-day 99% VaR is $10 million, that means there's only a 1% chance of losing more than $10 million tomorrow?

Correct. On 99 out of 100 days, the loss will be less than $10 million. But here's the critical caveat: VaR says nothing about how bad that 1% scenario can get. You could lose $11 million or $100 million — VaR doesn't distinguish between those outcomes. That's its fundamental weakness, and we'll address it later with Expected Shortfall.

6.3 Method 1: Historical Simulation VaR

The three VaR methods differ in one fundamental choice: how do we model the return distribution? The simplest answer is to skip modelling altogether and let history speak for itself.

Historical Simulation

The simplest VaR method: use the actual historical return distribution directly. No distributional assumptions needed.

Procedure:

1. Collect the last W daily returns (e.g., $W = 500$ trading days)
2. Sort the returns from worst to best
3. The VaR at confidence level α is the $(1 - \alpha) \times W$-th worst return
4. For 99% VaR with $W = 500$: the 5th worst return

Advantages: Distribution-free; captures fat tails if they exist in the historical data; easy to explain to non-technical audiences.

Disadvantages: Assumes the past is representative of the future; slow to adapt to changing volatility regimes; a calm historical window produces optimistic VaR just before a crisis.

Historical Simulation is the "look in the rearview mirror" approach. It works great until the road changes. In March 2020, the previous 500 days were mostly calm, so HS VaR was far too optimistic right when you needed it most.

6.4 Method 2: Parametric (Variance-Covariance) VaR

Parametric VaR

Assume returns follow a known distribution and use the volatility forecast from our GARCH model:

Normal distribution assumption:

$$\text{VaR}_\alpha = -(\mu_t + z_{1-\alpha} \cdot \sigma_t) \tag{6.3}$$

where $z_{1-\alpha}$ is the $(1-\alpha)$ standard normal quantile (e.g., $z_{0.01} = -2.326$ for 99% VaR) and σ_t is the conditional standard deviation from the GARCH model.

Student's t-distribution assumption:

$$\text{VaR}_\alpha = -(\mu_t + t_{1-\alpha,\nu} \cdot s_t) \tag{6.4}$$

where $t_{1-\alpha,\nu}$ is the $(1-\alpha)$-quantile of the Student's t-distribution with ν degrees of freedom, and $s_t = \sigma_t\sqrt{(\nu-2)/\nu}$ rescales to unit variance.

This is where all our GARCH work pays off. The key insight: σ_t changes every day based on the GARCH model's conditional variance forecast. So our VaR adapts dynamically to market conditions — it rises when volatility spikes and falls when markets calm down.

Why would we use the t-distribution instead of the normal?

Because real returns have fatter tails than the normal distribution — we proved this in Week 1. Using the normal quantile $z_{0.01} = -2.326$ underestimates extreme losses. The t-distribution with, say, $\nu = 7$ degrees of freedom gives $t_{0.01,7} \approx -2.998$. After scaling by $\sqrt{(\nu-2)/\nu}$ for unit-variance standardization, the effective quantile is about -2.53 — roughly 9% further in the tail than the normal's -2.326. This produces a more conservative and more accurate VaR.

Normal vs. t: The Tail Risk Gap

For 99% VaR with $\sigma = 1\%$ daily, using the scaled quantile $q \cdot \sqrt{(\nu-2)/\nu}$:

- **Normal**: VaR $\approx 2.33\%$ — the baseline
- t ($\nu = 7$): VaR $\approx 2.53\%$ — about 9% more conservative than the normal

- t ($\nu = 5$): VaR $\approx 2.61\%$ — about 12% more conservative, capturing heavier tails

The "right" degrees of freedom ν comes from the GARCH model estimation — we don't choose it arbitrarily.

6.5 Method 3: Monte Carlo VaR

If Historical Simulation looks backward and Parametric VaR assumes a distribution, Monte Carlo takes a third path: simulate thousands of possible futures and let the simulated distribution reveal the risk.

Monte Carlo Simulation VaR

Simulate thousands of possible future return paths from the fitted GARCH model, then extract the VaR from the simulated distribution:

Procedure:

1. Fit a GARCH model and obtain the conditional parameters
2. Simulate N (e.g., 10,000) one-step-ahead returns by drawing random innovations from the assumed distribution
3. Each simulated return represents one possible "tomorrow"
4. The VaR is the $(1 - \alpha)$ quantile of the simulated return distribution

Advantages: Can handle any distribution, path-dependent payoffs, and complex portfolio structures. The gold standard for derivatives books.

Disadvantages: Computationally expensive; results vary across simulation runs (need large N); "garbage in, garbage out" if the model is misspecified.

Monte Carlo is the Swiss Army knife of VaR. It's more flexible than parametric VaR because you can simulate from any model, not just normal or t. For a single-asset daily VaR, the parametric approach is usually sufficient. But for multi-asset portfolios with options, Monte Carlo is indispensable.

6.6 Beyond VaR: Expected Shortfall

This limitation has a formal name and a formal solution. The concept of Expected Shortfall answers the question VaR cannot: how bad is it when things go wrong?

The Fundamental Weakness of VaR

VaR has a critical blind spot: it tells you the *threshold* of the worst $(1-\alpha)\%$ of outcomes, but nothing about the severity beyond that threshold. A portfolio with a 99% VaR of \$10M could have an average loss of \$11M or \$50M in the 1% tail — VaR can't distinguish these very different risk profiles.

Conditional Value-at-Risk / Expected Shortfall

Expected Shortfall (ES), also called **CVaR** or **Conditional VaR**, measures the average loss *given that* the loss exceeds VaR:

$$\text{ES}_\alpha = -E[r_t \mid r_t < -\text{VaR}_\alpha] \tag{6.5}$$

ES is always larger than VaR (more conservative). Under Basel III's Fundamental Review of the Trading Book (FRTB), ES at 97.5% confidence level has replaced VaR as the primary regulatory risk measure for market risk capital.

Key property: ES is a **coherent risk measure** — it satisfies subadditivity (diversification always reduces risk), which VaR does not.

Wait — VaR doesn't satisfy subadditivity? That sounds like a serious flaw.

It is. Here's a simple example: two separate portfolios each have a 99% VaR of \$5M. If you combine them with diversification, the combined VaR could theoretically be higher than \$10M — violating the basic principle that diversification reduces risk. This is a mathematical artifact, not financial reality, but it makes VaR unreliable for portfolio aggregation. ES doesn't have this problem.

So we should switch from VaR to Expected Shortfall?

The industry is heading that direction. Basel III's FRTB already mandates ES for market risk capital. But VaR isn't going away — it's still reported alongside ES and used extensively for internal limits and communication. Today we'll implement both.

6.7 Python Implementation: VaR Three Ways

With the theory in place, we now turn to implementation. Each of the three methods requires a different code structure, but all share the same foundation: the GJR-GARCH model fitted in earlier weeks.

Let's build all three VaR methods plus Expected Shortfall. We'll use our GJR-GARCH model from Week 5 as the volatility engine.

Step 1: Data and Model Setup

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
from scipy import stats
import warnings
warnings.filterwarnings("ignore")

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Fit GJR-GARCH(1,1) with t-distribution
model = arch_model(returns, vol="Garch", p=1, o=1,
                   q=1, dist="t")
res = model.fit(disp="off")
print(res.summary().tables[1])

# Extract fitted parameters
nu = res.params["nu"]  # degrees of freedom
print(f"\nDegrees of freedom (nu): {nu:.2f}")
```

Step 2: Three VaR Methods

```python
# Confidence levels
alphas = [0.95, 0.99]

# --- Method 1: Historical Simulation ---
def var_historical(returns, alpha, window=500):
    """Rolling Historical Simulation VaR."""
    var_hs = pd.Series(index=returns.index, dtype=float)
    for i in range(window, len(returns)):
        hist = returns.iloc[i-window:i]
        var_hs.iloc[i] = -np.percentile(
            hist, (1 - alpha) * 100)
    return var_hs

# --- Method 2: Parametric (GARCH-based) ---
def var_parametric(res, alpha, dist="t"):
    """GARCH-based parametric VaR."""
    cond_vol = res.conditional_volatility
    mu = res.params.get("mu", 0)
    nu = res.params.get("nu", 30)

    if dist == "t":
        q = stats.t.ppf(1 - alpha, df=nu)
        scale = np.sqrt((nu - 2) / nu)
        var_p = -(mu + q * scale * cond_vol)
    else:  # normal
        q = stats.norm.ppf(1 - alpha)
        var_p = -(mu + q * cond_vol)
    return var_p

# --- Method 3: Monte Carlo ---
def var_monte_carlo(res, alpha, n_sims=10000,
                    seed=42):
    """GARCH-based Monte Carlo VaR."""
    np.random.seed(seed)
    sim = res.forecast(horizon=1, method="simulation",
                       simulations=n_sims)
    # .values gives simulated returns (mean + vol * shock);
```

```python
    # .residuals gives raw shocks only (not rescaled to returns)
    sim_returns = sim.simulations.values[-1, :, 0]
    var_mc = -np.percentile(sim_returns,
                            (1 - alpha) * 100)
    return var_mc, sim_returns

# Compute all VaR estimates
var_hs_99 = var_historical(returns, 0.99)
var_p_99_t = var_parametric(res, 0.99, "t")
var_p_99_n = var_parametric(res, 0.99, "normal")
var_mc_99, sim_rets = var_monte_carlo(res, 0.99)

print("VaR Estimates (last day):")
print(f"  Historical Sim (99%): "
      f"{var_hs_99.iloc[-1]:.4f}%")
print(f"  Parametric-t   (99%): "
      f"{var_p_99_t.iloc[-1]:.4f}%")
print(f"  Parametric-N   (99%): "
      f"{var_p_99_n.iloc[-1]:.4f}%")
print(f"  Monte Carlo    (99%): {var_mc_99:.4f}%")
```

The parametric-t VaR is noticeably larger than the normal VaR. That's the fat tail adjustment in action?

Exactly. The t-distribution puts more probability mass in the tails, so the 1% quantile is further out. For daily risk management, that extra conservatism can be the difference between adequate and inadequate capital reserves.

Step 3: Expected Shortfall

```python
# Expected Shortfall (CVaR)
def es_historical(returns, alpha, window=500):
    """Rolling Historical Simulation ES."""
    es = pd.Series(index=returns.index, dtype=float)
    for i in range(window, len(returns)):
        hist = returns.iloc[i-window:i].values
        var_level = np.percentile(
```

```
            hist, (1 - alpha) * 100)
        tail = hist[hist <= var_level]
        es.iloc[i] = -tail.mean() if len(tail) > 0 \
            else np.nan
    return es

def es_parametric(res, alpha, dist="t"):
    """GARCH-based parametric ES."""
    cond_vol = res.conditional_volatility
    mu = res.params.get("mu", 0)
    nu = res.params.get("nu", 30)

    if dist == "t":
        q = stats.t.ppf(1 - alpha, df=nu)
        pdf_q = stats.t.pdf(q, df=nu)
        scale = np.sqrt((nu - 2) / nu)
        # ES formula for Student's t
        es_factor = -(pdf_q / (1 - alpha)) * \
            ((nu + q**2) / (nu - 1))
        es_p = -(mu + es_factor * scale * cond_vol)
    else:
        q = stats.norm.ppf(1 - alpha)
        pdf_q = stats.norm.pdf(q)
        es_factor = -pdf_q / (1 - alpha)
        es_p = -(mu + es_factor * cond_vol)
    return es_p

# Compute ES
es_hs_99 = es_historical(returns, 0.99)
es_p_99 = es_parametric(res, 0.99, "t")

print("\nExpected Shortfall (last day):")
print(f"  Historical Sim (99%): "
      f"{es_hs_99.iloc[-1]:.4f}%")
print(f"  Parametric-t   (99%): "
      f"{es_p_99.iloc[-1]:.4f}%")
print(f"\nES / VaR ratio: "
      f"{es_p_99.iloc[-1]/var_p_99_t.iloc[-1]:.2f}")
```

The ES/VaR Ratio

For a normal distribution, the 99% ES is about 1.15 times the 99% VaR. For fat-tailed distributions (t with low degrees of freedom), the ratio is higher — typically 1.2–1.4. A high ES/VaR ratio signals that the tail beyond VaR is "heavy" — the expected loss *given* a VaR breach is significantly worse than the VaR itself. This ratio is a quick diagnostic for tail severity.

Now we visualize the VaR estimates over time, focusing on the COVID-19 crisis:

Step 4: VaR Visualization

```
fig, axes = plt.subplots(2, 1, figsize=(14, 10),
                         sharex=True)

# Panel 1: Returns vs. VaR
ax1 = axes[0]
ax1.plot(returns.index, returns, color="gray",
         linewidth=0.4, alpha=0.5, label="Returns")
ax1.plot(var_hs_99.index, -var_hs_99, color="#D4A843",
         linewidth=1, label="HS VaR (99%)")
ax1.plot(var_p_99_t.index, -var_p_99_t,
         color="#C0392B", linewidth=1,
         label="Parametric-t VaR (99%)")
ax1.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red")
ax1.set_ylabel("Return (%)")
ax1.set_title("Daily Returns vs. 99% VaR Estimates")
ax1.legend(loc="lower left", fontsize=8)
ax1.set_ylim(-15, 12)

# Panel 2: VaR comparison (positive scale)
ax2 = axes[1]
ax2.plot(var_hs_99.index, var_hs_99, color="#D4A843",
         linewidth=1, label="Historical Sim")
ax2.plot(var_p_99_t.index, var_p_99_t, color="#C0392B",
         linewidth=1, label="Parametric-t (GJR)")
ax2.plot(es_p_99.index, es_p_99, color="#1B3A5C",
         linewidth=1, alpha=0.7,
         label="Expected Shortfall (t)")
```

```
ax2.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red",
            label="COVID-19")
ax2.set_ylabel("Risk Measure (%)")
ax2.set_title("VaR and ES Over Time")
ax2.legend(loc="upper right", fontsize=8)
ax2.set_ylim(0, 15)

plt.tight_layout()
plt.savefig("week06_var_comparison.png", dpi=150)
plt.show()
```

6.8 Application: VaR During the COVID-19 Crash

Scene: Conference Room — Thursday, 2:00 PM

Alex presented the VaR comparison to the team. The COVID period on the screen told a dramatic story.

Here's what we found. During the COVID-19 crash in March 2020, the three VaR methods behaved very differently:

1. ***Historical Simulation was dangerously slow to react**. It was still based on the calm 2019 data, so it drastically underestimated the risk. VaR breaches occurred on 10+ days in a single month — far more than the expected 2–3 per year.*
2. ***Parametric VaR (GJR-GARCH) adapted quickly**. Because the GARCH model updates its conditional variance daily, the VaR estimate spiked within days of the first large decline. It still had some initial breaches, but adjusted much faster than Historical Simulation.*
3. ***The t-distribution matters**. Parametric VaR with normal distribution had 30–40% more breaches than the t-version. Fat tails aren't just a theoretical nicety — they have real capital implications.*

*This is exactly the argument for **conditional** VaR — VaR that updates with the latest volatility forecast. Historical Simulation is unconditional: it uses a fixed window of past returns regardless of current market stress. In a trending volatility regime, that's a recipe for*

disaster.

So you're recommending the parametric approach with our GJR-GARCH model?

Yes, specifically GJR-GARCH with Student's t-distribution. It has three advantages:

1. *Adapts to changing volatility via the GARCH mechanism*
2. *Accounts for fat tails via the t-distribution*
3. *Captures the leverage effect — negative returns increase risk more than positive returns of the same size*

For the FSA report, I'd recommend reporting both VaR and Expected Shortfall at the 99% level.

Good. Now the hard question: how do we know our VaR model is actually reliable? The FSA won't just take our word for it — they want proof that our model works.

*That's called **backtesting** — and it's exactly what we'll do next week.*

6.9 AI Spotlight: Neural Network VaR

I should mention where machine learning fits into all this. GARCH-based VaR is the industry standard, but the quant research world is experimenting with approaches that skip the distributional assumption entirely — letting the data define the quantile directly. Here's what those alternatives look like.

AI Approaches to VaR Estimation

Traditional VaR relies on parametric models (GARCH + distribution assumption) or historical data. AI offers alternative approaches:

- **Quantile regression**: Instead of modeling the full return distribution, directly predict the α-quantile. Linear quantile regression is simple; neural network quantile regression adds flexibility.
- **CAViaR** (Engle & Manganelli, 2004): Conditional Autoregressive Value-at-Risk. Models VaR dynamics directly as an autoregressive process, without specifying the full dis-

tribution. A bridge between traditional and ML approaches.

- **Deep learning VaR**: LSTM or Transformer networks trained to predict VaR quantiles directly from features (past returns, volume, VIX, macro indicators). Can capture complex nonlinear patterns but requires careful regularization.
- **Ensemble methods**: Combine VaR from multiple models (GARCH, HS, ML) to produce more robust estimates. Simple averaging often outperforms individual models.

The practical takeaway: GARCH-based parametric VaR remains the industry standard for its interpretability and regulatory acceptance. AI methods are gaining ground in proprietary trading desks and hedge funds where explainability requirements are lower.

6.10 Discussion: The VaR Debate

Is VaR a Good Risk Measure?

1. **The Taleb critique**: Nassim Taleb famously called VaR "charlatanism" because it ignores what happens beyond the confidence level (Taleb, 2007). Is this criticism fair? Does Expected Shortfall fully address it?
2. **Confidence level choice**: Why might a pension fund prefer 99% VaR while a hedge fund uses 95%? What are the tradeoffs between conservatism and capital efficiency?
3. **The procyclicality problem**: During a crisis, VaR rises $\rightarrow$ firms sell assets to reduce risk $\rightarrow$ prices fall further $\rightarrow$ VaR rises more. How might regulators address this feedback loop?
4. **Model risk**: We computed VaR three different ways and got three different numbers. Which one is "right"? How should a risk manager handle model disagreement?

6.11 Weekly Mission

◎ Mission 6: VaR for the Pension Fund

Objective: Compute and compare Value-at-Risk estimates for the Nikkei 225 and S&P 500 using multiple methods.

Deliverables:

1. For each index, compute daily 99% VaR using:
   - Historical Simulation (500-day window)
   - Parametric VaR with GJR-GARCH and normal distribution

- Parametric VaR with GJR-GARCH and t-distribution

2. Compute 99% Expected Shortfall for the parametric-t method
3. Count VaR breaches (days where actual loss exceeds VaR) for each method in the 2020 out-of-sample year. Report the breach rate as a percentage
4. Create a comparison plot showing returns with the three VaR lines overlaid (as in Step 4)
5. Write a 200-word memo to Kenji's board: recommend one VaR method for daily reporting and explain why. Include both VaR and ES figures for the most recent date

Bonus Challenge: Portfolio VaR

Extend the analysis to a two-asset portfolio: 60% S&P 500 and 40% Nikkei 225.

- Compute the portfolio return series
- Calculate Historical Simulation VaR for the portfolio
- Compare with the sum of individual VaRs — is the portfolio VaR lower (diversification benefit) or higher?
- How much does the yen/dollar exchange rate affect the results?

6.12 Key Takeaways

Key Takeaways

1. **VaR answers one question**: What is the maximum expected loss at a given confidence level over a given holding period? It requires three inputs: confidence level, holding period, and return distribution.
2. **Three methods**: Historical Simulation (simple, slow to adapt), Parametric (GARCH-based, dynamically updates), and Monte Carlo (flexible, computationally expensive). Each has tradeoffs between simplicity and accuracy.
3. **GARCH-based parametric VaR adapts fastest**: Because it updates conditional variance daily, it responds to regime changes far more quickly than Historical Simulation.
4. **Fat tails matter**: Using the t-distribution instead of the normal produces a more conservative and more accurate VaR. The improvement is most visible during crises.
5. **VaR has a blind spot**: It says nothing about losses beyond the confidence threshold. Expected Shortfall (CVaR) measures the average loss in the tail, making it a more informative risk measure.
6. **ES is the new regulatory standard**: Basel III's FRTB replaces VaR with Expected Shortfall (at 97.5%) for market risk capital. Both measures should be reported together.

6.13 Further Reading

Further Reading

1. Jorion, P. (2007). *Value at Risk: The New Benchmark for Managing Financial Risk*, 3rd ed. McGraw-Hill. [The definitive VaR textbook]
2. McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management: Concepts, Techniques and Tools*, 2nd ed. Princeton University Press. Chapters 2 and 9. [Rigorous treatment of VaR and ES]
3. Artzner, P., Delbaen, F., Eber, J.-M., & Heath, D. (1999). Coherent measures of risk. *Mathematical Finance*, 9(3), 203–228. [Why VaR is not coherent and ES is]
4. Engle, R. F., & Manganelli, S. (2004). CAViaR: Conditional autoregressive value at risk by regression quantiles. *Journal of Business & Economic Statistics*, 22(4), 367–381.
5. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. Chapters 12–15.
6. Basel Committee on Banking Supervision (2019). Minimum capital requirements for market risk. [The FRTB standard]
7. Kuester, K., Mittnik, S., & Paolella, M. S. (2006). Value-at-Risk prediction: a comparison of alternative strategies. *Journal of Financial Econometrics*, 4(1), 53–89. [Comprehensive VaR method comparison]
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 7.
9. Taleb, N. N. (2007). *The Black Swan*. Random House. [The original critique of VaR and distributional assumptions in finance]
10. JP Morgan/Reuters (1996). *RiskMetrics Technical Document*, 4th ed. [The original VaR framework that popularized daily VaR for banks]

Next Week Preview

The next morning, Alex found an urgent email from Kenji. Attached was a formal letter from the Japan Financial Services Agency: "Dear VolTech Analytics, As part of our annual review of Nippon Life Pension Fund's risk management framework, please provide a comprehensive backtesting report for your VaR model. The report must include: (1) historical VaR breach analysis, (2) the Kupiec unconditional coverage test, (3) the Christoffersen conditional coverage test, and (4) stress testing results under at least three extreme scenarios." Kenji's note at the bottom read: "They want this in two weeks. No pressure." Alex looked at Dr. Lin. She smiled. "Welcome to the world of regulatory compliance. This is where we prove our model isn't just pretty charts — it actually works." Next week: we put our VaR model on trial with ***backtesting and stress testing****.*

Chapter 7

Week 7: VaR Backtesting and Risk Management

Learning Objectives

1. Define VaR backtesting and explain why regulators require it
2. Implement the Kupiec (POF) unconditional coverage test for VaR model validation
3. Implement the Christoffersen conditional coverage test to detect clustered VaR breaches
4. Interpret the Basel traffic light system for VaR model assessment
5. Design and execute stress tests using historical and hypothetical scenarios

7.1 This Week's Story: The Regulator Calls

Scene: VolTech Analytics — Monday, 7:45 AM

Alex had never seen Kenji look this tense. The email from the Japan Financial Services Agency was displayed on the conference room screen, its formal language cutting through the early morning quiet. Dr. Lin had already printed the relevant Basel III guidelines and spread them across the table.

The FSA wants a full backtesting report in two weeks. They specifically asked for the Kupiec test, the Christoffersen test, and stress testing under extreme scenarios. If we fail this review, the pension fund may be required to hold additional capital — which means less money available for investment returns.

Don't panic. This is standard regulatory practice. Every major financial institution goes through this. The key question is simple: does our VaR model do what it claims? If we say the 99% VaR will only be breached 1% of the time, we need to prove it.

How do we "prove" a probability model works?

By looking at the track record. If your 99% VaR was breached on exactly about 1% of days over the past year, that's evidence the model is well-calibrated. If it was breached too often, the model is underestimating risk. If it was never breached, the model might be too conservative — which wastes capital.

7.2 The Logic of Backtesting

What Is VaR Backtesting?

Backtesting compares the model's VaR forecasts against actual realized losses. Define the **hit sequence**:

$$I_t = \begin{cases} 1 & \text{if } r_t < -\text{VaR}_t \quad \text{(breach)} \\ 0 & \text{otherwise} \end{cases}$$

Under a correct model, the hit sequence should satisfy two properties:

1. **Unconditional coverage**: The proportion of breaches should equal $1 - \alpha$. For 99% VaR, expect 1% breaches.
2. **Independence**: Breaches should not cluster. A breach today should not predict a breach tomorrow. Clustering indicates the model is slow to adapt.

A good backtesting framework tests *both* properties.

Why Both Properties Matter

Consider two scenarios over 250 trading days with 99% VaR:

- **Model A**: 3 breaches, spread across January, June, and November. Breach rate = 1.2% — close to expected 1%. Passes unconditional coverage.
- **Model B**: 3 breaches, all in the same week during a market crash. Same breach rate (1.2%), but the clustering reveals the model failed to adapt to rising volatility.

Model A is well-calibrated; Model B has a serious independence problem. The Kupiec test catches Model B only if the total count is wrong. The Christoffersen test catches Model B's clustering even when the total count is correct.

7.3 Test 1: Kupiec Unconditional Coverage

Kupiec POF Test (1995)

Let T be the total number of out-of-sample days and x the number of VaR breaches. The expected breach probability is $p = 1 - \alpha$.
The **Proportion of Failures** (POF) likelihood ratio test:

$$LR_{uc} = -2 \ln \frac{p^x (1-p)^{T-x}}{\hat{p}^x (1-\hat{p})^{T-x}} \tag{7.1}$$

where $\hat{p} = x/T$ is the observed breach rate. Under H_0: $\hat{p} = p$, the statistic $LR_{uc} \sim \chi^2(1)$.
Interpretation:

- Reject H_0 if $LR_{uc} > \chi^2_{1,0.05} = 3.841$ (or p-value < 0.05)
- Rejection means the breach rate is significantly different from the expected rate — the model is miscalibrated
- Two-sided: rejects both too many *and* too few breaches

So if we have 250 days and 99% VaR, we expect 2.5 breaches. What range would pass the test?

Roughly 0 to 6 breaches would pass the Kupiec test at the 5% significance level for $T = 250$. Seven or more breaches, and the model is in trouble. But remember: the Kupiec test has low power with small samples. With only 250 days, it's hard to distinguish between a 1% and a 2% breach rate. That's why regulators typically want at least one year of data.

7.4 Test 2: Christoffersen Conditional Coverage

Christoffersen Test (1998)

This test extends Kupiec by also checking for **independence** of breaches. Define the transition probabilities:

$$\pi_{ij} = P(I_t = j \mid I_{t-1} = i) \quad \text{for } i, j \in \{0, 1\}$$

The independence test statistic:

$$LR_{ind} = -2 \ln \frac{\hat{\pi}^{n_{01}+n_{11}}(1-\hat{\pi})^{n_{00}+n_{10}}}{\hat{\pi}_{01}^{n_{01}}(1-\hat{\pi}_{01})^{n_{00}}\hat{\pi}_{11}^{n_{11}}(1-\hat{\pi}_{11})^{n_{10}}} \tag{7.2}$$

where n_{ij} counts transitions from state i to state j, $\hat{\pi} = (n_{01} + n_{11})/(T-1)$, and $\hat{\pi}_{ij} = n_{ij}/(n_{i0} + n_{i1})$.

The **conditional coverage** test combines both:

$$LR_{cc} = LR_{uc} + LR_{ind} \sim \chi^2(2) \tag{7.3}$$

Reject H_0 if $LR_{cc} > \chi^2_{2,0.05} = 5.991$.

From a data perspective, Kupiec is a one-sample proportion test — you're just comparing the observed breach rate to 1%. Christoffersen adds a Markov transition model on top: if breach indicators are serially correlated, the model's residuals are autocorrelated, and any decent rolling-window estimator should have caught that. Clusters mean the conditional distribution is mis-specified, not just the unconditional one.

7.5 The Basel Traffic Light System

Basel Traffic Light Framework

The Basel Committee uses a simple classification based on the number of VaR breaches over 250 trading days for a 99% VaR model:

Zone	Breaches	Interpretation	Capital Multiplier
Green	0–4	Model acceptable	3.0×
Yellow	5–9	Model questionable	3.4–3.85×
Red	≥ 10	Model rejected	4.0×

A red zone classification means the bank must multiply its VaR by 4.0 (instead of 3.0) to compute regulatory capital — a significant financial penalty. Banks are strongly motivated to stay in the green zone.

So if our model lands in the red zone, the pension fund has to hold 33% more capital? That's money that could be invested.

Exactly. VaR model quality has direct financial consequences. An overly aggressive model saves capital in good times but risks red zone penalties and regulatory scrutiny. The sweet spot is a model that's accurate and slightly conservative.

7.6 Stress Testing

Stress Testing Framework

While backtesting evaluates the model under *normal* conditions, **stress testing** examines performance under *extreme* scenarios. Two main approaches:

1. Historical stress tests: Replay actual crisis periods:

- 2008 Global Financial Crisis (GFC)
- 2020 COVID-19 crash
- 2011 European debt crisis
- 1997 Asian financial crisis

2. Hypothetical stress tests: Design scenarios that haven't occurred but are plausible:

- Sudden 5% market drop in one day
- VIX spike from 15 to 80 (overnight)
- Simultaneous crash in all Asian markets
- Interest rate shock of +300 basis points

For each scenario, compute the portfolio loss and compare it with the VaR estimate. If the loss exceeds VaR, quantify by how much.

Stress testing is where theory meets boardroom reality. Our investors don't care about chi-squared statistics. They want to know: "What happens if 2008 happens again? What happens if something worse happens?" Stress tests give them concrete answers in dollar terms.

7.7 Python Implementation: The Backtesting Suite

Let's build a complete backtesting and stress testing pipeline. We'll test our GJR-GARCH VaR model from Week 6.

Step 1: VaR Backtest Setup

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
from scipy import stats
import warnings
warnings.filterwarnings("ignore")

# Download data and compute returns
sp500 = yf.download("^GSPC", start="2015-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Rolling VaR backtest
window = 1000
alpha = 0.99
n_total = len(returns)
n_oos = n_total - window

# Storage
var_bt = np.full(n_oos, np.nan)
es_bt = np.full(n_oos, np.nan)

print(f"Backtest period: {n_oos} days")
print(f"Expected breaches (99%): {n_oos * 0.01:.1f}")
```

With the data loaded and storage arrays initialized, we can run the rolling window estimation. Each iteration refits the GJR-GARCH model on 1,000 trading days of history and extracts a one-step-ahead forecast — the computational core of the pipeline.

Step 2: Rolling VaR and ES Computation

```python
# Rolling GJR-GARCH VaR (t-distribution)
# Note: this takes ~5-10 minutes
for i in range(n_oos):
    train = returns.iloc[i:i + window]
    try:
        model = arch_model(train, vol="Garch",
                           p=1, o=1, q=1, dist="t")
        res = model.fit(disp="off",
                        show_warning=False)
        # One-step-ahead forecast
        fv = res.forecast(horizon=1)
        cond_var = fv.variance.values[-1, 0]
        cond_vol = np.sqrt(cond_var)
        mu = res.params.get("mu", 0)
        nu = res.params["nu"]

        # Parametric VaR (t-distribution)
        q = stats.t.ppf(1 - alpha, df=nu)
        scale = np.sqrt((nu - 2) / nu)
        var_bt[i] = -(mu + q * scale * cond_vol)

        # Parametric ES (t-distribution)
        pdf_q = stats.t.pdf(q, df=nu)
        es_factor = -(pdf_q / (1 - alpha)) * \
            ((nu + q**2) / (nu - 1))
        es_bt[i] = -(mu + es_factor * scale * cond_vol)
    except Exception:
        var_bt[i] = np.nan
        es_bt[i] = np.nan

var_series = pd.Series(var_bt,
    index=returns.index[window:])
es_series = pd.Series(es_bt,
    index=returns.index[window:])
oos_returns = returns.iloc[window:]

# Identify breaches
```

```python
breaches = oos_returns < -var_series
n_breaches = breaches.sum()
breach_rate = n_breaches / len(oos_returns)
print(f"Breaches: {n_breaches}")
print(f"Breach rate: {breach_rate:.4f} "
      f"(expected: {1-alpha:.4f})")
```

Once the breach series is constructed, we apply the two formal statistical tests discussed earlier. The functions below return structured dictionaries so results can be compared side-by-side.

Step 3: Kupiec and Christoffersen Tests

```python
def kupiec_test(breaches, alpha=0.99):
    """Kupiec POF test for unconditional coverage."""
    hits = breaches.astype(int).values
    T = len(hits)
    x = hits.sum()
    p = 1 - alpha  # expected breach probability

    if x == 0 or x == T:
        return {"LR_uc": np.inf, "p_value": 0.0,
                "breaches": x, "expected": T * p}

    p_hat = x / T
    lr = -2 * (x * np.log(p / p_hat) +
               (T - x) * np.log((1-p) / (1-p_hat)))
    p_val = 1 - stats.chi2.cdf(lr, df=1)
    return {"LR_uc": lr, "p_value": p_val,
            "breaches": int(x),
            "expected": T * p,
            "breach_rate": p_hat}

def christoffersen_test(breaches, alpha=0.99):
    """Christoffersen conditional coverage test."""
    hits = breaches.astype(int).values
    T = len(hits)

    # Count transitions
    n00, n01, n10, n11 = 0, 0, 0, 0
```

```
    for t in range(1, T):
        if hits[t-1] == 0 and hits[t] == 0: n00 += 1
        elif hits[t-1] == 0 and hits[t] == 1: n01 += 1
        elif hits[t-1] == 1 and hits[t] == 0: n10 += 1
        else: n11 += 1

    # Unconditional coverage
    kup = kupiec_test(breaches, alpha)
    lr_uc = kup["LR_uc"]

    # Independence test
    pi = (n01 + n11) / max(T - 1, 1)
    pi01 = n01 / max(n00 + n01, 1)
    pi11 = n11 / max(n10 + n11, 1)

    # Compute in log-domain to avoid numerical underflow
    eps = 1e-10
    log_num = ((n00 + n10) * np.log(max(1 - pi, eps)) +
               (n01 + n11) * np.log(max(pi, eps)))
    log_den = (n00 * np.log(max(1 - pi01, eps)) +
               n01 * np.log(max(pi01, eps)) +
               n10 * np.log(max(1 - pi11, eps)) +
               n11 * np.log(max(pi11, eps)))
    lr_ind = -2 * (log_num - log_den)

    lr_cc = lr_uc + lr_ind
    p_val = 1 - stats.chi2.cdf(lr_cc, df=2)

    return {"LR_uc": lr_uc, "LR_ind": lr_ind,
            "LR_cc": lr_cc, "p_value_cc": p_val,
            "n00": n00, "n01": n01,
            "n10": n10, "n11": n11}

# Run both tests
kup = kupiec_test(breaches)
cc = christoffersen_test(breaches)

print("=== Kupiec Test ===")
```

```python
print(f"Breaches: {kup['breaches']} / "
      f"{len(oos_returns)}")
print(f"Expected: {kup['expected']:.1f}")
print(f"Breach rate: {kup['breach_rate']:.4f}")
print(f"LR_uc = {kup['LR_uc']:.3f}, "
      f"p-value = {kup['p_value']:.4f}")
result = "PASS" if kup["p_value"] > 0.05 else "FAIL"
print(f"Result: {result}")

print("\n=== Christoffersen Test ===")
print(f"LR_uc = {cc['LR_uc']:.3f}")
print(f"LR_ind = {cc['LR_ind']:.3f}")
print(f"LR_cc = {cc['LR_cc']:.3f}, "
      f"p-value = {cc['p_value_cc']:.4f}")
result = "PASS" if cc["p_value_cc"] > 0.05 else "FAIL"
print(f"Result: {result}")
```

With test statistics in hand, the next step maps breach counts to regulatory consequences via the Basel traffic-light lookup.

Step 4: Basel Traffic Light Classification

```python
def basel_traffic_light(breaches_250):
    """Basel traffic light for 250-day window."""
    n = int(breaches_250.sum())
    # Basel multiplier lookup table
    yellow_mult = {5: 3.40, 6: 3.50, 7: 3.65,
                   8: 3.75, 9: 3.85}
    if n <= 4:
        zone = "GREEN"
        mult = 3.0
    elif n <= 9:
        zone = "YELLOW"
        mult = yellow_mult[n]
    else:
        zone = "RED"
        mult = 4.0
    return zone, mult, n

```

```python
# Evaluate last 250 trading days
last_250 = breaches.iloc[-250:]
zone, mult, n = basel_traffic_light(last_250)
print(f"\n=== Basel Traffic Light ===")
print(f"Breaches (last 250 days): {n}")
print(f"Zone: {zone}")
print(f"Capital multiplier: {mult:.2f}x")
```

Regulatory classification done, we now stress-test the current VaR estimate against a menu of historical and hypothetical shocks to understand how far beyond VaR extreme losses can reach.

Step 5: Stress Testing

```python
# Historical stress scenarios
stress_scenarios = {
    "GFC 2008 (worst day)": -0.0947,
    "COVID 2020 (worst day)": -0.1277,
    "Flash Crash 2010": -0.0387,
    "Brexit 2016": -0.0348,
    "VIX spike 2018": -0.0431,
    "Hypothetical -5%": -0.05,
    "Hypothetical -10%": -0.10,
    "Hypothetical -15%": -0.15,
}

# Current VaR and ES (last day)
current_var = var_series.iloc[-1]
current_es = es_series.iloc[-1]
print(f"Current 99% VaR: {current_var:.4f}%")
print(f"Current 99% ES:  {current_es:.4f}%")
print(f"\n{'Scenario':<30} {'Loss':>8} "
      f"{'vs VaR':>8} {'vs ES':>8}")
print("-" * 56)
for name, loss_pct in stress_scenarios.items():
    loss = abs(loss_pct) * 100  # convert to %
    var_ratio = loss / current_var
    es_ratio = loss / current_es
    flag = " !!" if loss > current_var else ""
    print(f"{name:<30} {loss:>7.2f}% "
```

```
            f"{var_ratio:>7.2f}x {es_ratio:>7.2f}x"
            f"{flag}")
```

Finally, we visualize the backtest results:

Step 6: Backtest Visualization

```
fig, axes = plt.subplots(2, 1, figsize=(14, 10),
                         sharex=True)

# Panel 1: Returns with VaR and breach markers
ax1 = axes[0]
ax1.plot(oos_returns.index, oos_returns, color="gray",
         linewidth=0.4, alpha=0.5, label="Returns")
ax1.plot(var_series.index, -var_series, color="#C0392B",
         linewidth=0.8, label="99% VaR (GJR-t)")
ax1.scatter(oos_returns[breaches].index,
            oos_returns[breaches], color="#C0392B",
            s=20, zorder=5, label="VaR breaches")
ax1.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red")
ax1.set_ylabel("Return (%)")
ax1.set_title("VaR Backtest: GJR-GARCH(1,1) "
              "with t-distribution")
ax1.legend(loc="lower left", fontsize=8)
ax1.set_ylim(-15, 12)

# Panel 2: Cumulative breaches
ax2 = axes[1]
cum_breaches = breaches.cumsum()
expected_line = pd.Series(
    np.arange(1, len(breaches) + 1) * 0.01,
    index=breaches.index)
ax2.plot(cum_breaches.index, cum_breaches,
         color="#C0392B", linewidth=1.5,
         label="Actual breaches")
ax2.plot(expected_line.index, expected_line,
         color="#1B3A5C", linewidth=1,
         linestyle="--", label="Expected (1%)")
```

```
ax2.set_ylabel("Cumulative Breaches")
ax2.set_title("Cumulative VaR Breaches vs. Expected")
ax2.legend(loc="upper left")

plt.tight_layout()
plt.savefig("week07_backtest.png", dpi=150)
plt.show()
```

7.8 Application: The FSA Report

Scene: VolTech Analytics — Friday, 4:30 PM

The team reviewed the complete backtesting report before sending it to the FSA. Alex summarized the findings on the whiteboard.

Here's our backtesting summary for the GJR-GARCH VaR model with Student's t-distribution:

1. ***Breach analysis**: Over the full out-of-sample period, our 99% VaR was breached on approximately 1.0–1.5% of days — close to the expected 1%. Most breaches occurred during the initial phase of COVID-19, before the model fully adapted.*
2. ***Kupiec test**: Failed to reject H_0 ($p > 0.05$) — the breach rate is consistent with the 99% confidence level.*
3. ***Christoffersen test**: The independence component shows mild clustering during COVID, but the overall conditional coverage test passes at the 5% level.*
4. ***Basel classification**: Green zone with 2–3 breaches in the most recent 250 days. Capital multiplier remains at $3.0\times$.*
5. ***Stress tests**: Our VaR would be exceeded by GFC-level and COVID-level shocks, but ES provides a more realistic estimate of tail losses.*

The numbers look solid. What about compared to the old Historical Simulation model my fund was using?

Historical Simulation would likely land in the yellow or red zone for any period containing a regime change. The GJR-GARCH model's key advantage is its adaptiveness — it updates daily, so it recovers from a crisis shock much faster.

Excellent work. The FSA report is ready. Kenji, your board should be pleased — we've demonstrated a model that is statistically validated, Basel-compliant, and stress-tested. But I've been thinking about our next proposal to your fund. Our competitors at QuantStar are pitching an AI-driven risk system. If we want to stay ahead, we need to show Kenji's team what AI can add to our framework.

I'm listening. What can AI do that your GARCH model can't?

Let me show you. Next week.

7.9 AI Spotlight: Automated Risk Monitoring

AI-Enhanced Risk Management Systems

Modern risk management is increasingly augmented by AI:

- **Automated VaR breach detection**: ML classifiers that predict likely breach days *before* they happen, using features like VIX changes, credit spreads, and news sentiment. Early warning systems can trigger portfolio hedging.
- **Dynamic stress scenario generation**: Instead of fixed historical scenarios, generative models (GANs, VAEs) can create realistic but novel crisis scenarios by learning the statistical structure of past crises.
- **Regime detection**: Hidden Markov Models or change-point detection algorithms that automatically identify when markets shift from calm to volatile regimes, triggering model parameter updates.
- **NLP for risk narratives**: Large language models that automatically generate human-readable risk reports from quantitative outputs, bridging the gap between quants and board members.

The common thread: AI doesn't replace the core risk framework (VaR, ES, backtesting) — it *enhances* each component with better data, faster adaptation, and richer scenario

analysis.

7.10 Discussion: When Models Fail

Risk Management After the Crisis

1. **The 2008 lesson**: Many banks had green-zone VaR models right up until the GFC. Their VaR models were "backtested and validated." What went wrong, and how can we prevent it?
2. **Model risk vs. market risk**: Is the risk of using a wrong model sometimes greater than the market risk itself? How should a risk manager balance model complexity against model risk?
3. **Regulatory arbitrage**: If banks know the Basel traffic light thresholds, could they design models that deliberately stay in the green zone while underestimating true risk? How might regulators detect this?
4. **Beyond VaR**: The FRTB moves from VaR to ES. But ES is harder to backtest (you need to evaluate the average of rare events). Is this a problem for regulatory oversight?

7.11 Weekly Mission

◎ Mission 7: The FSA Backtesting Report

Objective: Produce a comprehensive VaR backtesting and stress testing report for the Japan FSA.

Deliverables:

1. For both Nikkei 225 and S&P 500:
   - Compute rolling 99% VaR using GJR-GARCH (t-distribution) with a 1000-day window
   - Count breaches and compute the breach rate
   - Run the Kupiec unconditional coverage test
   - Run the Christoffersen conditional coverage test
   - Determine the Basel traffic light zone for the most recent 250 days
2. Create a backtest visualization (returns + VaR + breach markers) for each index
3. Create a cumulative breach plot for each index

4. Run stress tests for at least 5 scenarios (3 historical + 2 hypothetical) and report VaR and ES exceedance ratios
5. Write a 300-word executive summary in the form of a letter to the Japan FSA, covering: model description, backtest results, Basel classification, and stress test conclusions

Bonus Challenge: Comparing VaR Models

Run the full backtesting suite for three VaR methods:

- Historical Simulation (500-day window)
- Parametric VaR with standard GARCH(1,1) and normal distribution
- Parametric VaR with GJR-GARCH(1,1) and t-distribution

Compare their Kupiec test results and Basel traffic light zones. Which model performs best? Does the model that "looks best" on paper also produce the most useful risk estimates in practice?

7.12 Key Takeaways

Key Takeaways

1. **Backtesting validates VaR models**: Compare predicted VaR against actual losses. The hit sequence $I_t \in \{0, 1\}$ should have the correct proportion and no clustering.
2. **Kupiec test**: Tests unconditional coverage — is the overall breach rate equal to $1 - \alpha$? Uses a likelihood ratio statistic with $\chi^2(1)$ distribution.
3. **Christoffersen test**: Tests both coverage *and* independence — are breaches properly spaced or clustered? Uses $\chi^2(2)$ for the joint test.
4. **Basel traffic light**: A practical classification system. Green (0–4 breaches) is acceptable; yellow (5–9) triggers scrutiny; red (≥ 10) requires increased capital.
5. **Stress testing complements backtesting**: Backtesting evaluates normal conditions; stress tests evaluate extreme scenarios. Both are required for regulatory compliance.
6. **GJR-GARCH with t-distribution**: Consistently produces green-zone backtesting results for equity indices, outperforming Historical Simulation and normal-distribution parametric VaR.

7.13 Further Reading

Further Reading

1. Kupiec, P. H. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives*, 3(2), 73–84. [The original POF test]
2. Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review*, 39(4), 841–862. [The conditional coverage test]
3. Basel Committee on Banking Supervision (1996). Supervisory framework for the use of "backtesting" in conjunction with the internal models approach to market risk capital requirements. [The traffic light system]
4. McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management*, 2nd ed. Chapter 9.
5. Berkowitz, J., & O'Brien, J. (2002). How accurate are Value-at-Risk models at commercial banks? *Journal of Finance*, 57(3), 1093–1111. [Real-world VaR performance]
6. Jorion, P. (2007). *Value at Risk*, 3rd ed. McGraw-Hill. Chapters 6–8.
7. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 7.
8. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. Chapters 14–16.

Next Week Preview

As Kenji signed off on the FSA report, David called an all-hands meeting. "I've just had a call with Kenji's investment committee. They're impressed with our risk framework, but they've asked a new question: 'Can AI do this better?'" He looked around the room. "QuantStar is pitching them a pure machine learning risk system — random forests, LSTMs, the whole package. They claim 50% better accuracy than traditional models." Priya's eyes lit up. "Challenge accepted." Dr. Lin raised an eyebrow. "Better accuracy by what measure? Against what benchmark? With what interpretability?" David smiled. "That's exactly what we need to show them. Next week: we build VolTech's AI risk engine and put it head-to-head against everything we've built so far." Next week: ***AI in Finance*** *— where machine learning meets financial risk, and we discover that the future is not AI or GARCH, but AI and GARCH.*

Chapter 8

Week 8: AI in Finance

Learning Objectives

1. Explain how machine learning approaches differ from traditional econometric models for volatility forecasting
2. Implement a Random Forest volatility forecasting model with engineered features
3. Understand the basic architecture and intuition behind LSTM neural networks for time series
4. Conduct a rigorous ML vs. GARCH comparison using the evaluation framework from Week 5
5. Design a hybrid model that combines GARCH structural knowledge with ML flexibility

8.1 This Week's Story: The Final Challenge

Scene: VolTech Analytics — Monday, 8:00 AM

The conference room had been transformed. Priya had set up two screens side by side: the left showed GARCH output, the right showed a neural network training dashboard with loss curves updating in real time. A banner across the whiteboard read: **GARCH vs. AI: The Final Showdown**.

Welcome to the final round. QuantStar claims their ML system beats traditional models by 50%. Today we test that claim — honestly, rigorously, and with the evaluation framework we built. No cherry-picking, no look-ahead bias, no hype.

I want to be clear about what we're testing. The question isn't "Is AI magic?" or "Is GARCH obsolete?" The question is: under what conditions does each approach have an advantage, and how can we combine them intelligently?

My investment committee wants a clear recommendation: should we invest in an AI risk system, keep our GARCH framework, or do both? The answer needs to be backed by evidence, not marketing slides.

Then let's build the evidence. Priya, show us what you've got.

8.2 ML vs. Econometrics: Two Philosophies

Two Approaches to Volatility Modeling

Traditional Econometric Approach (GARCH):

- Start from financial theory (e.g., volatility clustering, leverage effect)
- Specify a parametric model (e.g., GJR-GARCH with t-distribution)
- Estimate a small number of parameters (5–7) via maximum likelihood
- Interpretable: each parameter has economic meaning
- Strong performance with limited data

Machine Learning Approach:

- Start from data, not theory
- Learn complex patterns through flexible function approximation
- Estimate potentially thousands of parameters
- Often a "black box": prediction without explanation
- Requires large datasets to avoid overfitting

Neither is universally superior. The choice depends on the *data regime*, *prediction horizon*, and *interpretability requirements*.

So which one should Nippon Life actually deploy for daily VaR reporting?

That depends on your data and your horizon — which is exactly what the bias-variance tradeoff formalizes.

The Bias-Variance Tradeoff in Finance

GARCH has high **bias** (strong assumptions about the data-generating process) but low **variance** (stable predictions with limited data). ML has low bias (flexible assumptions) but high variance (predictions can be unstable, especially with small datasets).

In finance, data is noisy and regimes change. This often favors the higher-bias, lower-variance approach at *short* horizons. The ML advantage emerges when:

- Rich feature sets are available (VIX, volume, sentiment)
- The prediction horizon is longer (weekly, monthly)
- Nonlinear patterns exist that parametric models cannot capture
- Enough training data is available to regularize the model

8.3 Random Forest for Volatility

Random Forest: Ensemble of Decision Trees

A **Random Forest** builds many decision trees, each trained on a random subset of the data and features. The final prediction is the average across all trees.

Why it works for volatility:

- Captures nonlinear relationships between features and future volatility
- Robust to outliers (trees are adaptive to extreme values)
- Built-in feature importance ranking
- No distributional assumptions

Feature engineering for volatility prediction:

- **Lagged returns**: $r_{t-1}, r_{t-2}, \ldots, r_{t-5}$
- **Lagged squared returns**: $r_{t-1}^2, \ldots, r_{t-5}^2$ (proxy for recent volatility)
- **Rolling volatility**: 5-day, 10-day, 22-day rolling standard deviation
- **Absolute returns**: $|r_{t-1}|, \ldots, |r_{t-5}|$
- **GARCH conditional volatility**: $\hat{\sigma}_t$ from a fitted GARCH model (hybrid feature!)

Notice that last feature: GARCH conditional volatility as an input to the ML model. This is the key to the hybrid approach. We let GARCH summarize its structural knowledge into a single feature, then let the ML model learn whether it can add anything on top.

8.4 LSTM Neural Networks for Time Series

Long Short-Term Memory (LSTM)

LSTM is a type of recurrent neural network designed for sequential data. Key concepts:

- **Cell state** (c_t): A long-range conveyor belt that carries information across many time steps with minimal modification. It is distinct from the **hidden state** (h_t): the cell state is the LSTM's *long-term memory*, updated additively at each step, while the hidden state is its *short-term working memory*, exposed to the next layer and used directly for predictions. Intuitively, c_t remembers that "we are in a high-volatility regime" over weeks, while h_t encodes the most recent return dynamics.
- **Gates**: Three gates control information flow:
  - *Forget gate*: Decides what old information in c_{t-1} to discard
  - *Input gate*: Decides what new information to write into c_t
  - *Output gate*: Derives h_t from c_t, deciding what to expose to the next layer
- **Sequence processing**: Processes returns one time step at a time, building up a representation of the volatility pattern through the interplay of c_t and h_t

Intuition for volatility: The cell state allows LSTM to remember volatility-regime shifts that occurred many steps ago — something a plain RNN forgets due to vanishing gradients. In practice, c_t captures the persistent level of volatility while h_t tracks short-run clustering, mirroring the long-run variance and conditional variance roles in a GARCH model.

So LSTM can discover GARCH-like behavior on its own?

In theory, yes. But here's the catch: GARCH gives you this for free with 5 parameters. LSTM needs thousands of parameters and much more data to learn the same pattern. The question is whether LSTM can learn additional patterns beyond what GARCH captures.

8.5 Python Implementation: The ML Arena

Let's build a complete ML volatility forecasting pipeline and compare it head-to-head with GJR-GARCH.

Step 1: Feature Engineering

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import warnings
warnings.filterwarnings("ignore")

# Download data
sp500 = yf.download("^GSPC", start="2010-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Target: next-day realized volatility (squared return)
target = returns.shift(-1) ** 2

# Feature engineering
features = pd.DataFrame(index=returns.index)
# Lagged returns
for lag in range(1, 6):
    features[f"ret_lag{lag}"] = returns.shift(lag)
# Lagged squared returns
for lag in range(1, 6):
    features[f"ret2_lag{lag}"] = (
        returns.shift(lag) ** 2)
# Rolling volatility
for w in [5, 10, 22]:
    features[f"rvol_{w}d"] = (
        returns.rolling(w).std())
# Absolute returns
for lag in range(1, 4):
    features[f"abs_ret_lag{lag}"] = (
        returns.shift(lag).abs())

```

```python
# Drop NaN rows
data = pd.concat([features, target.rename("target")],
                 axis=1).dropna()
print(f"Features: {len(features.columns)}")
print(f"Observations: {len(data)}")
print(f"Feature list:\n{list(features.columns)}")
```

Step 2: Train-Test Split and GJR-GARCH Baseline

```python
# Time-series split (no shuffling!)
split_date = "2020-01-01"
train = data[data.index < split_date]
test = data[data.index >= split_date]

X_train = train.drop("target", axis=1)
y_train = train["target"]
X_test = test.drop("target", axis=1)
y_test = test["target"]

print(f"Train: {len(train)} obs "
      f"({train.index[0].date()} to "
      f"{train.index[-1].date()})")
print(f"Test:  {len(test)} obs "
      f"({test.index[0].date()} to "
      f"{test.index[-1].date()})")

# GJR-GARCH baseline (full-sample fit, then forecast)
ret_all = returns.loc[data.index]
model = arch_model(ret_all, vol="Garch", p=1, o=1,
                   q=1, dist="t")
res = model.fit(last_obs=split_date, disp="off")
fv = res.forecast(start=split_date, horizon=1)
garch_var = fv.variance.dropna()
garch_fc = garch_var.iloc[:, 0]
# Align with test period
garch_fc = garch_fc.reindex(test.index).dropna()
```

Step 3: Random Forest Model

```python
# Random Forest with tuned hyperparameters
rf = RandomForestRegressor(
    n_estimators=500,
    max_depth=10,
    min_samples_leaf=20,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)
rf_pred = pd.Series(rf.predict(X_test),
                    index=X_test.index)

# Feature importance
importance = pd.Series(
    rf.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

print("Top 10 Features:")
for feat, imp in importance.head(10).items():
    print(f"  {feat:<20} {imp:.4f}")
```

The rolling volatility features dominate the importance ranking. That makes sense — recent volatility is the best predictor of future volatility.

Exactly. And notice how similar that is to GARCH's structure: the β term in GARCH is essentially "yesterday's conditional variance," which is the model's own rolling volatility summary. The Random Forest is rediscovering this pattern from data.

Step 4: Hybrid Model — GARCH + Random Forest

```python
# GARCH variance for hybrid feature:
#   training period  -> in-sample conditional var
#   test period      -> out-of-sample forecast
```

```
# In-sample: fitted conditional variance (training only)
garch_in = res.conditional_volatility ** 2
garch_in = garch_in[garch_in.index < split_date]
# Out-of-sample: forecasted variance (test only)
garch_out = garch_fc          # from Step 2
garch_feature = pd.concat([garch_in, garch_out])
assert not garch_feature.index.duplicated().any(), \
    "Overlap detected between in-sample and forecast"
garch_feature = garch_feature.reindex(data.index)

data_hybrid = data.copy()
data_hybrid["garch_var"] = garch_feature

train_h = data_hybrid[data_hybrid.index < split_date]
test_h = data_hybrid[data_hybrid.index >= split_date]
train_h = train_h.dropna()
test_h = test_h.dropna()

X_train_h = train_h.drop("target", axis=1)
y_train_h = train_h["target"]
X_test_h = test_h.drop("target", axis=1)
y_test_h = test_h["target"]

# Hybrid Random Forest
rf_hybrid = RandomForestRegressor(
    n_estimators=500,
    max_depth=10,
    min_samples_leaf=20,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)
rf_hybrid.fit(X_train_h, y_train_h)
hybrid_pred = pd.Series(rf_hybrid.predict(X_test_h),
                        index=X_test_h.index)

# Check if GARCH feature is important
imp_h = pd.Series(
```

```
    rf_hybrid.feature_importances_,
    index=X_train_h.columns
).sort_values(ascending=False)
garch_rank = list(imp_h.index).index("garch_var") + 1
print(f"\nGARCH feature importance rank: "
      f"#{garch_rank} of {len(imp_h)}")
print(f"GARCH feature importance: "
      f"{imp_h['garch_var']:.4f}")
```

Step 5: Head-to-Head Comparison

```
# Align all forecasts to common index
common_idx = garch_fc.index.intersection(
    rf_pred.index).intersection(hybrid_pred.index)
y_eval = y_test.reindex(common_idx)
garch_eval = garch_fc.reindex(common_idx)
rf_eval = rf_pred.reindex(common_idx)
hybrid_eval = hybrid_pred.reindex(common_idx)

def evaluate(actual, forecast, name):
    """Same three metrics from Week 5."""
    a, f = actual.values, np.maximum(
        forecast.values, 1e-10)
    rmse = np.sqrt(np.mean((a - f) ** 2))
    mae = np.mean(np.abs(a - f))
    qlike = np.mean(np.log(f) + a / f)
    return {"Model": name, "RMSE": rmse,
            "MAE": mae, "QLIKE": qlike}

results = pd.DataFrame([
    evaluate(y_eval, garch_eval, "GJR-GARCH"),
    evaluate(y_eval, rf_eval, "Random Forest"),
    evaluate(y_eval, hybrid_eval,
             "Hybrid (GARCH+RF)"),
]).set_index("Model")

print("\n=== Forecast Comparison ===")
print(results.round(4))
```

```python
print(f"\nBest by RMSE:  {results['RMSE'].idxmin()}")
print(f"Best by MAE:   {results['MAE'].idxmin()}")
print(f"Best by QLIKE: {results['QLIKE'].idxmin()}")
```

Step 6: Visualization

```python
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Forecast comparison
ax1 = axes[0]
ax1.plot(y_eval.index,
         np.sqrt(y_eval * 252), color="gray",
         alpha=0.3, linewidth=0.5,
         label="Realized proxy (ann.)")
ax1.plot(garch_eval.index,
         np.sqrt(garch_eval * 252), color="#1B3A5C",
         linewidth=1, label="GJR-GARCH")
ax1.plot(rf_eval.index,
         np.sqrt(rf_eval * 252), color="#C0392B",
         linewidth=1, alpha=0.7,
         label="Random Forest")
ax1.plot(hybrid_eval.index,
         np.sqrt(hybrid_eval * 252), color="#2D7D5E",
         linewidth=1, alpha=0.7,
         label="Hybrid (GARCH+RF)")
ax1.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red")
ax1.set_ylabel("Annualized Volatility (%)")
ax1.set_title("ML vs. GARCH: Out-of-Sample "
              "Volatility Forecasts")
ax1.legend(loc="upper right", fontsize=8)
ax1.set_ylim(0, 120)

# Panel 2: Feature importance
ax2 = axes[1]
top_features = imp_h.head(10)
colors = ["#D4A843" if "garch" in f else "#1B3A5C"
          for f in top_features.index]
```

```
ax2.barh(range(len(top_features)),
         top_features.values, color=colors)
ax2.set_yticks(range(len(top_features)))
ax2.set_yticklabels(top_features.index, fontsize=9)
ax2.set_xlabel("Feature Importance")
ax2.set_title("Hybrid Model: Top 10 Features "
              "(gold = GARCH)")
ax2.invert_yaxis()

plt.tight_layout()
plt.savefig("week08_ml_vs_garch.png", dpi=150)
plt.show()
```

8.6 Application: The Verdict

Scene: VolTech Board Room — Friday, 10:00 AM

The entire VolTech team, plus Kenji's investment committee (joining by video), gathered for the final presentation. Eight weeks of work came down to this moment.

After extensive testing, here are our conclusions on GARCH vs. ML for daily volatility forecasting:

Table 8.1: GARCH vs. ML: summary of findings[a]

Criterion	GJR-GARCH	Random Forest	Hybrid
1-day RMSE	5.61	5.52	**5.21**
1-day QLIKE	3.65	3.82	**3.51**
Interpretability	High	Low	Medium
Training speed	Fast	Moderate	Moderate
Regulatory fit	Strong	Weak	Medium
Crisis response	Good	Variable	Good

[a]RMSE and QLIKE are representative values for S&P 500 daily squared returns (%). Exact figures depend on sample period; relative ordering is robust.

Three key findings:

1. ***At 1-day horizons, GJR-GARCH is remarkably competitive**. The Random Forest barely improves on it, and sometimes does worse on QLIKE. QuantStar's "50% improvement" claim likely used cherry-picked metrics or look-ahead bias.*
2. ***The hybrid model is the clear winner**. Adding GARCH conditional volatility as a feature to the Random Forest consistently produces the best forecasts. GARCH provides the structural backbone; ML adds the fine-tuning.*
3. ***Interpretability matters for regulatory compliance**. The FSA can audit a GARCH model. A neural network with 10,000 parameters is much harder to explain. The hybrid approach offers a pragmatic middle ground.*

Let me add the academic perspective. Our results are consistent with the literature. Bucci (2020) showed that GARCH remains hard to beat at daily horizons. Christensen, Siggaard, and Veliyev (2023) found that combining ML with econometric features outperforms either approach alone. The future of financial risk modeling is not a battle between AI and traditional methods — it's their integration.

So your recommendation for my fund?

Deploy GJR-GARCH as the primary risk engine for daily VaR and ES reporting — it's fast, interpretable, and Basel-compliant. Layer the hybrid Random Forest model as a supplementary monitoring tool that flags when GARCH may be underestimating risk. This gives you the best of both worlds: regulatory compliance and cutting-edge analytics.

That actually makes sense from a governance perspective. The board would never approve a pure black-box for capital calculations — but a GARCH engine with an ML early-warning layer is something I can take to the investment committee and explain line by line. Consider the contract in motion.

And that, Kenji, is what VolTech delivers: not blind adoption of the latest technology, but intelligent integration based on evidence. Welcome to the future of risk management.

8.7 The FinTech Frontier

AI and FinTech: What's Next

The intersection of AI and finance is evolving rapidly. Key trends for the next decade:

- **Foundation models for finance**: Large language models (GPT, Claude) trained on financial text can extract sentiment, summarize earnings calls, and generate risk narratives. They won't replace quants, but they will augment their productivity.
- **Explainable AI (XAI)**: Regulatory pressure is driving development of interpretable ML models. SHAP values, attention maps, and counterfactual explanations make black boxes more transparent.
- **Real-time risk monitoring**: Streaming ML pipelines that process tick-by-tick data, news feeds, and social media to update risk estimates intraday. Note that high-frequency data introduces its own challenges — market microstructure noise, bid–ask bounce, and asynchronous arrivals require careful preprocessing before any model can be applied reliably.
- **Democratization of quant tools**: Cloud platforms and open-source libraries (like `arch`, `scikit-learn`, `pytorch`) make sophisticated risk analytics accessible to smaller firms and individual investors.
- **Regulatory technology (RegTech)**: AI systems that automate compliance reporting, anomaly detection, and audit trails. The FSA report we built manually this semester will likely be auto-generated within a few years.

The professionals who thrive will be those who understand *both* the financial theory and the AI tools — exactly the combination this course has built.

8.8 Discussion: The Future of Finance

AI, Ethics, and the Future

1. **Job displacement**: Will AI replace financial analysts and risk managers? Or will it create new roles that require different skills? How should you prepare?
2. **Algorithmic bias**: If an AI risk model is trained on historical data, it inherits the biases of that data (e.g., underweighting emerging market risk). How can we detect and mitigate this?
3. **Systemic risk**: If every bank uses the same AI risk model, they might all react the same way to market stress — amplifying crashes instead of dampening them. Is AI-driven

herding a real threat?

4. **Looking back**: You've spent 8 weeks building a risk management system from scratch. What was the most surprising thing you learned? What would you do differently if you started the project again?

8.9 Weekly Mission

◎ Mission 8: The VolTech AI Risk Report

Objective: Build and evaluate an ML volatility forecasting model, and write the final report for Kenji's investment committee.

Deliverables:

1. Build a Random Forest model for the Nikkei 225 and S&P 500:
   - Engineer at least 15 features (lagged returns, rolling volatility, absolute returns)
   - Use a time-series split (pre-2020 train, 2020+ test)
   - Report feature importance rankings
2. Build a hybrid model by adding GJR-GARCH conditional volatility as an additional feature
3. Compare GJR-GARCH, Random Forest, and Hybrid models using RMSE, MAE, and QLIKE
4. Create a comparison visualization (as in Step 6)
5. Write a 500-word final report (as a memo from VolTech to Nippon Life Pension Fund):
   - Executive summary of all 8 weeks
   - Model recommendation with evidence
   - Risk management framework overview
   - Future roadmap and AI integration plan

🏆 Bonus Challenge: Build a Simple LSTM

For those comfortable with deep learning, implement a simple LSTM volatility forecasting model:

- Use PyTorch or TensorFlow
- Input: sequence of 22 past returns
- Output: next-day squared return prediction

- Compare with GJR-GARCH and Random Forest
- How much training data does the LSTM need before it becomes competitive?

Hint: Start with a single-layer LSTM with 32 hidden units. Regularize heavily (dropout = 0.3) to prevent overfitting.

8.10 Key Takeaways

Key Takeaways

1. **ML and GARCH are complementary, not competing**: GARCH provides structural knowledge (volatility clustering, leverage effect) in a parsimonious form. ML adds flexibility to capture patterns beyond what GARCH assumes.
2. **GARCH is hard to beat at daily horizons**: With only 5–7 parameters, GJR-GARCH captures the dominant dynamics of daily volatility. The ML advantage grows with richer features and longer horizons.
3. **Feature engineering is the key to ML success**: The Random Forest's performance depends heavily on the features you provide. The most important features (rolling volatility, lagged squared returns) mirror GARCH's structure.
4. **Hybrid models win**: Adding GARCH conditional volatility as an ML feature consistently produces the best out-of-sample forecasts. This is the practical recommendation: use GARCH as a feature, not a competitor.
5. **Interpretability has economic value**: Regulatory compliance requires model explainability. A model that can't be audited may produce better forecasts but worse business outcomes.
6. **The future is integration**: The most effective risk management systems will combine econometric structure, ML flexibility, and human judgment. This course has equipped you with the foundation for all three.

8.11 Further Reading

Further Reading

1. Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3), 502–531.
2. Christensen, K., Siggaard, M., & Veliyev, B. (2023). A machine learning approach to volatility forecasting. *Journal of Financial Econometrics*, 21(5), 1680–1727.
3. Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273. [The benchmark paper for ML in finance]
4. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. [The original LSTM paper]
5. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
6. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS*. [SHAP values for model interpretability]
7. Dixon, M. F., Halperin, I., & Bilokon, P. (2020). *Machine Learning in Finance: From Theory to Practice*. Springer.
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 12.

Formula Appendix

This appendix collects all key formulas from the textbook for quick reference.

Week 1: Return Distributions

Simple Return

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{8.1}$$

Log Return

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(1 + R_t) \tag{8.2}$$

Skewness

$$S = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^3 \tag{8.3}$$

Excess Kurtosis

$$K = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^4 - 3 \tag{8.4}$$

Jarque–Bera Test

$$JB = \frac{T}{6}\left(S^2 + \frac{K^2}{4}\right) \sim \chi^2(2) \tag{8.5}$$

Week 2: Measuring Volatility

Rolling Window Volatility

$$\hat{\sigma}_t^{\text{roll}} = \sqrt{\frac{1}{N-1}\sum_{i=0}^{N-1}(r_{t-i} - \bar{r}_{t,N})^2} \tag{8.6}$$

Annualized: multiply by $\sqrt{252}$.

EWMA Variance

$$\hat{\sigma}_t^2 = \lambda\,\hat{\sigma}_{t-1}^2 + (1-\lambda)\,r_{t-1}^2 \tag{8.7}$$

RiskMetrics standard: $\lambda = 0.94$. Effective window $\approx 1/(1-\lambda)$ days.

Realized Volatility

$$RV_t = \sum_{j=1}^{M} r_{t,j}^2 \tag{8.8}$$

where $r_{t,j}$ are M intraday returns on day t.

Week 3: GARCH

Return Decomposition

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t\,z_t, \quad z_t \sim D(0,1) \tag{8.9}$$

ARCH(q) Model

$$\sigma_t^2 = \omega + \sum_{i=1}^{q}\alpha_i\,\varepsilon_{t-i}^2 \tag{8.10}$$

GARCH(1,1) Model

$$\sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2 \tag{8.11}$$

Constraints: $\omega > 0$, $\alpha \geq 0$, $\beta \geq 0$, $\alpha + \beta < 1$.

Long-Run Variance

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta} \tag{8.12}$$

Volatility Half-Life

$$H = \frac{\ln(0.5)}{\ln(\alpha + \beta)} \tag{8.13}$$

Ljung–Box Test

$$Q(m) = T(T+2) \sum_{k=1}^{m} \frac{\hat{\rho}_k^2}{T-k} \sim \chi^2(m) \tag{8.14}$$

Standardized Residuals

$$\hat{z}_t = \frac{r_t - \hat{\mu}}{\hat{\sigma}_t} \tag{8.15}$$

Week 4: Asymmetric Volatility

GJR-GARCH(1,1)

$$\sigma_t^2 = \omega + \alpha\, \varepsilon_{t-1}^2 + \gamma\, \varepsilon_{t-1}^2\, I_{[\varepsilon_{t-1}<0]} + \beta\, \sigma_{t-1}^2 \tag{8.16}$$

$\gamma > 0$ captures the leverage effect. Stationarity: $\alpha + \beta + \gamma/2 < 1$.

News Impact Curve

$$\sigma_t^2(\varepsilon_{t-1}) = \begin{cases} \omega + (\alpha + \gamma)\, \varepsilon_{t-1}^2 + \beta\, \bar{\sigma}^2 & \text{if } \varepsilon_{t-1} < 0 \\ \omega + \alpha\, \varepsilon_{t-1}^2 + \beta\, \bar{\sigma}^2 & \text{if } \varepsilon_{t-1} \geq 0 \end{cases} \tag{8.17}$$

EGARCH(1,1)

$$\ln \sigma_t^2 = \omega + \alpha(|z_{t-1}| - E|z_{t-1}|) + \gamma\, z_{t-1} + \beta\, \ln \sigma_{t-1}^2 \tag{8.18}$$

Models log-variance; no positivity constraints needed.

Likelihood Ratio Test

$$LR = 2(\ell_{\text{GJR}} - \ell_{\text{GARCH}}) \sim \chi^2(1) \tag{8.19}$$

Reject GARCH in favor of GJR if $LR > 3.84$.

Week 5: Volatility Forecasting

Loss Functions

$$\text{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(\hat{\sigma}_t^2 - \sigma_t^{*2})^2} \tag{8.20}$$

$$\text{MAE} = \frac{1}{T}\sum_{t=1}^{T}|\hat{\sigma}_t^2 - \sigma_t^{*2}| \tag{8.21}$$

$$\text{QLIKE} = \frac{1}{T}\sum_{t=1}^{T}\left(\ln\hat{\sigma}_t^2 + \frac{\sigma_t^{*2}}{\hat{\sigma}_t^2}\right) \tag{8.22}$$

where σ_t^{*2} is the realized volatility proxy.

Mincer–Zarnowitz Regression

$$\sigma_t^{*2} = a + b\,\hat{\sigma}_t^2 + u_t \tag{8.23}$$

Optimal forecast: $H_0 : a = 0,\ b = 1$. Joint F-test.

Week 6: Value-at-Risk

VaR Definition

$$P(L_t > \text{VaR}_\alpha) = 1 - \alpha \tag{8.24}$$

Parametric VaR (Normal)

$$\text{VaR}_\alpha = -(\mu + z_{1-\alpha}\cdot\sigma_t) \tag{8.25}$$

Parametric VaR (Student-t)

$$\text{VaR}_\alpha = -\left(\mu + t_{1-\alpha,\nu} \cdot \sqrt{\frac{\nu-2}{\nu}} \cdot \sigma_t\right) \tag{8.26}$$

Scale factor $\sqrt{(\nu-2)/\nu}$ converts standard t to unit variance.

Expected Shortfall / CVaR

$$\text{ES}_\alpha = E[L_t \mid L_t > \text{VaR}_\alpha] \tag{8.27}$$

ES is a *coherent* risk measure; VaR is not (fails subadditivity).

Week 7: VaR Backtesting

Kupiec POF Test

$$LR_{\text{POF}} = 2\left[x \ln\left(\frac{x/T}{1-\alpha}\right) + (T-x)\ln\left(\frac{1-x/T}{\alpha}\right)\right] \sim \chi^2(1) \tag{8.28}$$

x = number of VaR breaches in T observations.

Christoffersen Conditional Coverage

$$LR_{\text{ind}} = 2\left[\sum_{i,j} n_{ij} \ln\left(\frac{\hat{\pi}_{ij}}{\hat{\pi}_j}\right)\right] \sim \chi^2(1) \tag{8.29}$$

Tests whether breaches are independent (no clustering).

$$LR_{\text{CC}} = LR_{\text{POF}} + LR_{\text{ind}} \sim \chi^2(2) \tag{8.30}$$

Basel Traffic Light System

Zone	Breaches (250 days)	Action
Green	0–4	Model accepted
Yellow	5–9	Capital surcharge ($k = 3.4$–3.85)
Red	10+	Model rejected ($k \geq 4.0$)

Week 8: AI in Finance

Random Forest Prediction

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x}) \tag{8.31}$$

Average of B decision trees, each trained on a bootstrap sample with random feature subsets ($\sqrt{p}$ features per split).

Bias-Variance Decomposition

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise} \tag{8.32}$$

GARCH: high bias, low variance. ML: low bias, high variance.

Glossary

Key terms and definitions used throughout this textbook. Terms are listed alphabetically with the week number where they are first introduced.

Term	Definition	Week
ARCH Model	Autoregressive Conditional Heteroscedasticity; models variance as a function of past squared errors.	3
Backtesting	The process of evaluating a risk model's accuracy by comparing its predictions with realized outcomes over a historical period.	7
Basel Traffic Light	A regulatory framework classifying VaR models as Green (0–4 breaches), Yellow (5–9), or Red (10+) based on 250-day backtest results.	7
Bias-Variance Tradeoff	The balance between model simplicity (high bias) and flexibility (high variance); central to comparing GARCH and ML approaches.	8
Christoffersen Test	A conditional coverage test that checks whether VaR breaches are both correctly sized and independently distributed.	7
Conditional Variance	The variance of a random variable given information up to time $t-1$; the core quantity modeled by GARCH.	3
CVaR (Expected Shortfall)	The expected loss given that loss exceeds VaR; also called Conditional VaR. A coherent risk measure.	6
Decay Factor (λ)	The smoothing parameter in EWMA; controls how quickly old observations lose influence. RiskMetrics standard: $\lambda = 0.94$.	2

Term	Definition	Week
Diebold-Mariano Test	A statistical test for comparing the predictive accuracy of two competing forecast models.	5
EGARCH	Exponential GARCH; models log-variance to capture asymmetric effects without positivity constraints.	4
EWMA	Exponentially Weighted Moving Average; a volatility estimator that gives more weight to recent observations.	2
Excess Kurtosis	Kurtosis minus 3; measures the heaviness of tails relative to a normal distribution. $K > 0$ indicates fat tails.	1
Expected Shortfall	See CVaR. The average loss in the worst $(1 - \alpha)\%$ of cases.	6
Fat Tails	A property of distributions where extreme events occur more frequently than predicted by the normal distribution.	1
Feature Engineering	The process of creating informative input variables for ML models from raw data (e.g., lagged returns, rolling volatility).	8
GARCH(1,1)	Generalized Autoregressive Conditional Heteroscedasticity; the most widely used volatility model in finance.	3
GJR-GARCH	A GARCH extension by Glosten, Jagannathan, and Runkle that captures the leverage effect via an indicator variable.	4
Half-Life	The number of days for a volatility shock to decay to half its initial impact; $H = \ln(0.5)/\ln(\alpha + \beta)$.	3
Historical Simulation	A VaR method that uses the empirical distribution of past returns; non-parametric and distribution-free.	6

Term	Definition	Week
Hybrid Model	A model that combines GARCH structural output (as a feature) with ML flexibility for improved forecasting.	8
Implied Volatility	The volatility level embedded in option prices; forward-looking, unlike realized volatility.	2
Integrated Variance	The true (latent) variance of a continuous-time price process over a period; estimated by realized volatility.	2
Jarque–Bera Test	A statistical test for normality based on skewness and kurtosis. $JB \sim \chi^2(2)$ under H_0.	1
Kupiec Test	A test for VaR model accuracy based on the proportion of failures (POF); uses a likelihood ratio statistic.	7
Leptokurtic	A distribution with excess kurtosis > 0; has heavier tails than the normal distribution.	1
Leverage Effect	The empirical observation that negative returns increase volatility more than positive returns of equal magnitude.	4
Ljung–Box Test	A portmanteau test for autocorrelation in a time series; used to detect ARCH effects in squared returns.	3
Log Return	Natural logarithm of the price ratio; preferred in financial modeling for its additive property.	1
Long-Run Variance	The unconditional variance implied by a GARCH model: $\bar{\sigma}^2 = \omega/(1-\alpha-\beta)$.	3
LSTM	Long Short-Term Memory; a recurrent neural network architecture designed for sequential data with long-range dependencies.	8
Maximum Likelihood	A parameter estimation method that finds values maximizing the probability of observing the data.	3

Term	Definition	Week
Mean Reversion	The tendency of volatility to return to a long-run average after being displaced by shocks.	2
Microstructure Noise	Distortions in high-frequency price data caused by bid-ask bounce, discrete tick sizes, and other market frictions.	2
Mincer-Zarnowitz Regression	A forecast evaluation regression: $\sigma_t^{*2} = a + b\hat{\sigma}_t^2 + u_t$. Optimal forecast: $a = 0, b = 1$.	5
Monte Carlo VaR	A VaR method that simulates thousands of return paths from a fitted model to estimate the loss distribution.	6
News Impact Curve	A plot showing how positive and negative shocks of different magnitudes affect conditional variance.	4
Out-of-Sample	Forecast evaluation using data not used in model estimation; the gold standard for predictive assessment.	5
Parametric VaR	A VaR method assuming returns follow a specific distribution (normal or Student-t) with time-varying variance.	6
Persistence	The sum $\alpha + \beta$ in GARCH (or $\alpha + \beta + \gamma/2$ in GJR); measures how slowly volatility shocks decay.	3
QLIKE	Quasi-likelihood loss function; robust to noise in the volatility proxy and preferred by forecasting literature.	5
QQ Plot	Quantile–Quantile plot; a graphical tool for comparing a sample distribution against a theoretical distribution.	1
Random Forest	An ensemble ML method that averages predictions from many decision trees trained on random subsets of data and features.	8

Term	Definition	Week
Realized Volatility	The sum of squared intraday returns; a model-free estimate of daily integrated variance.	2
RiskMetrics	J.P. Morgan's 1996 risk measurement framework; popularized EWMA with $\lambda = 0.94$ for daily data.	2
Rolling Window	A volatility estimator using the standard deviation of the most recent N returns with equal weights.	2
Skewness	A measure of the asymmetry of a probability distribution. $S < 0$ for most equity return series.	1
Standardized Residuals	The residuals $\hat{z}_t = (r_t - \hat{\mu})/\hat{\sigma}_t$ from a GARCH model; should be i.i.d. if the model is correct.	3
Stress Testing	Evaluating a risk model's performance under hypothetical extreme scenarios (historical or user-defined).	7
Student-t Distribution	A heavy-tailed distribution controlled by degrees of freedom ν; commonly used as the error distribution in GARCH.	1
Stylized Facts	Robust empirical patterns observed across financial markets: clustering, mean reversion, asymmetry, long memory, co-movement.	2
Subadditivity	A property of coherent risk measures: $\rho(X + Y) \leq \rho(X) + \rho(Y)$. VaR can violate this; ES satisfies it.	6
Value-at-Risk (VaR)	The maximum expected loss over a given time horizon at a specified confidence level.	6
Variance Risk Premium	The difference between implied volatility and realized volatility; compensation for bearing volatility risk.	2
VIX	CBOE Volatility Index; a measure of the market's expectation of 30-day volatility implied by S&P 500 options.	2

Term	Definition	Week
Volatility Clustering	The empirical observation that large changes in prices tend to be followed by large changes, and small by small.	3
Volatility Smile/Smirk	The U-shaped (or skewed) pattern of implied volatility across option strike prices; evidence of non-normality.	2